

*Digital Comprehensive Summaries of Uppsala Dissertations
from the Faculty of Science and Technology 2632*

Symbolic Execution for Testing Conformance of Network Protocol Implementations

HOOMAN ASADIAN



ACTA UNIVERSITATIS
UPSALIENSIS
2026



UPPSALA
UNIVERSITET

Dissertation presented at Uppsala University to be publicly examined in 10132, Högssalen, Ångström, Regementsvägen 10, Uppsala, Friday, 20 March 2026 at 09:15 for the degree of Doctor of Philosophy. The examination will be conducted in English. Faculty examiner: Associate professor Erik Poll (Institute for Computing and Information Science, at the Faculty of Science of Radboud University).

Abstract

Asadian, H. 2026. Symbolic Execution for Testing Conformance of Network Protocol Implementations. *Digital Comprehensive Summaries of Uppsala Dissertations from the Faculty of Science and Technology* 2632. 79 pp. Uppsala: Acta Universitatis Upsaliensis. ISBN 978-91-513-2728-0.

Testing network protocol implementations for conformance with their specifications is challenging due to protocol statefulness, complex dependencies across packet sequences, and large input spaces that hinder exhaustive exploration. Subtle deviations from specifications can result in interoperability issues or security vulnerabilities, yet often escape detection by conventional testing techniques. This thesis investigates how symbolic execution and related techniques can be adapted to address these challenges in a systematic and practical manner.

The first contribution of the thesis is a requirement-driven symbolic execution approach for testing stateful network protocol implementations against their specifications. The approach guides symbolic execution toward execution paths that are relevant for violating protocol requirements expressed over sequences of packets. By selectively making packet fields symbolic and constraining exploration to requirement-relevant paths, the approach enables the detection of violations that depend on subtle corner-case values.

The second contribution is a monitor-based symbolic execution technique that externalizes requirement checking from the system under test. Protocol requirements are encoded in lightweight monitors that observe packet exchanges, maintain interaction state, and guide symbolic exploration without intrusive modifications to protocol implementations. This design improves modularity, supports reuse of requirements across implementations and versions, and enables early detection of requirement violations.

The third contribution explores differential symbolic execution as an oracle-free testing technique. Instead of relying on explicitly encoded requirements, the approach compares observable behaviors across implementations or versions under identical symbolic inputs to identify discrepancies that may indicate nonconformance.

The final contribution examines requirement-based testing of IoT protocol implementations using fuzzing and symbolic execution, with CoAP as a case study. The study analyzes how both techniques can be adapted to check protocol requirements and compares their effectiveness and practical trade-offs in exposing specification violations.

The proposed techniques are evaluated on real-world implementations of widely deployed protocols, including DTLS, QUIC, and CoAP. The evaluation demonstrates that the proposed techniques uncover numerous previously unknown bugs, specification violations, and interoperability issues.

Keywords: Symbolic Execution, Conformance Testing, Network Protocol Implementations, DTLS, QUIC, CoAP

Hooman Asadian, Department of Information Technology, Computer Systems, Box 337, Uppsala University, SE-75105 Uppsala, Sweden.

© Hooman Asadian 2026

ISSN 1651-6214

ISBN 978-91-513-2728-0

URN urn:nbn:se:uu:diva-577919 (<http://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-577919>)

Dedicated to my parents Hanieh & Mohammad

List of papers

This thesis is based on the following papers, which are referred to in the text by their Roman numerals.

- I Applying Symbolic Execution to Test Implementations of a Network Protocol Against its Specification**
Hooman Asadian, Paul Fiterău-Broștean, Bengt Jonsson, and Konstantinos Sagonas. In *IEEE Conference on Software Testing, Verification and Validation (ICST 2022)*. 70-81.
<https://doi.org/10.1109/ICST53961.2022.00019>
- II Monitor-based Testing of Network Protocol Implementations Using Symbolic Execution**
Hooman Asadian, Paul Fiterău-Broștean, Bengt Jonsson, and Konstantinos Sagonas. In *Proceedings of the 19th International Conference on Availability, Reliability and Security ARES '24*. ACM, Article 17, 1-12. <https://doi.org/10.1145/3664476.3664521>
- III Uncovering Flaws in Network Protocol Implementations Using Differential Symbolic Execution**
Hooman Asadian, Paul Fiterău-Broștean, Bengt Jonsson, and Konstantinos Sagonas. Draft
- IV Testing IoT Protocol Requirements Using Fuzzing and Symbolic Execution: Application to CoAP**
Hooman Asadian, Paul Fiterău-Broștean, Bengt Jonsson, and Konstantinos Sagonas. In *IEEE Conference on Standards for Communications and Networking (CSCN 2024)*. 48-54.
<https://doi.org/10.1109/CSCN63874.2024.10849709>

Reprints were made with permission from the publishers.

Contents

1	Introduction	9
1.1	Network Protocols	10
1.2	Specification vs. Implementation	11
1.3	Protocol Conformance Testing	12
1.4	Research Challenges	13
2	Background	15
2.1	Traditional Testing Methods	15
2.2	Modern Automated Approaches for Protocol Testing	15
2.3	Symbolic Execution	17
3	From Specifications to Oracles: Requirement-Driven Symbolic Execution	21
3.1	Technique	21
3.1.1	Extracting Specification Requirements	21
3.1.2	Symbolic Execution	23
3.1.3	Test Case Construction and Validation	25
3.2	Evaluation	26
3.3	Limitations	26
4	Monitor-based Symbolic Execution for Stateful Protocols	28
4.1	Technique	28
4.1.1	Monitors	28
4.1.2	Creating a Test Harness	30
4.1.3	Encoding Monitors as Programs	31
4.1.4	Invoking Monitors	31
4.1.5	Developing Protocol-specific Infrastructure	32
4.1.6	Applying Small Modifications to the SUT	32
4.2	Implementation	32
4.2.1	Generic Infrastructure	32
4.2.2	Protocol-specific Infrastructure	33
4.3	Evaluation	34
4.4	Limitations	35
5	Beyond Oracles: Differential Symbolic Execution	37
5.1	Technique	38
5.1.1	Creating a Test Harness	38
5.1.2	Running Symbolic Execution	39

5.1.3	Finding and Analyzing Discrepancies	41
5.1.4	Selecting Fields for Symbolic Execution	43
5.2	Implementation	45
5.2.1	Creating a Test Harness	45
5.2.2	Augmentations to Symbolic Execution	45
5.2.3	Finding and Analyzing Discrepancies	47
5.2.4	Optional SUT Adaptations	48
5.2.5	Evaluation	48
6	Two Paths to Conformance Testing: Fuzzing and Symbolic Execution	51
6.1	Technique	51
6.1.1	Test Harness Creation	51
6.1.2	Encoding Protocol Requirements as Assertions	52
6.1.3	Testing Requirements Using Fuzzing	54
6.1.4	Testing Requirements Using Symbolic Execution	55
6.2	Evaluation	56
7	Related Work	58
7.1	Specification-Based Protocol Testing	58
7.2	Fuzzing for Network Protocol Implementations	59
7.3	Symbolic Execution for Protocol Implementations	60
8	Conclusion	63
9	Future Work	64
10	Contributions	67
10.1	Paper I	67
10.2	Paper II	67
10.3	Paper III	68
10.4	Paper IV	68
10.5	Personal Contributions	69
11	Sammanfattning På Svenska	70
12	Acknowledgements	72
	References	74

1. Introduction

Reliable and secure communication has become a fundamental requirement for every modern information system. Software components that handle communication, whether they are network stacks in operating systems or protocol libraries used in IoT devices, must operate in accordance with their specifications. When they do not, even subtle deviations can lead to a range of problems, including interoperability failures, functional errors, and severe security vulnerabilities. Incidents such as Heartbleed [60] and the TLS POODLE downgrade vulnerability [48, 61] demonstrate that even minor deviations can have catastrophic consequences. The practical lesson is that adherence to specifications is essential for both security and reliability.

Testing protocol implementations is difficult for two main reasons. First, modern protocols are usually stateful, and correct processing of a single packet often depends on a potentially long and intricate sequence of preceding packets. Second, protocol messages are highly parameterized, containing numerous interdependent fields whose values in some cases influence the interpretation of other fields and the protocol’s operational state. This creates a vast input space in which subtle combinations of parameters may expose corner cases or requirement violations that are rarely triggered in normal operation. These characteristics make classical unit testing and ad hoc fuzzing insufficient for comprehensive validation. Unit tests rarely explore adversarial message sequences, while naively applied fuzzing struggles to cover the complex dependencies and stateful interactions of protocol messages. Consequently, many implementation faults remain unnoticed until they are discovered in the wild or exploited.

Several established techniques exist for the systematic testing of software. Among these, *symbolic execution (SE)* has proved to be particularly effective. SE is a white-box technique that analyzes programs in which some inputs or variables are designated as *symbolic*, and explores the code paths that are feasible for some assignment to these symbolic inputs, thereby reasoning over the entire space of their possible values. In the context of network protocols, this means that an implementation can be tested not only against individual packets but also against entire classes of message sequences that may trigger specification violations.

This thesis investigates how SE can be adapted and extended to test the conformance of real-world network protocol implementations against their specifications. The focus is on developing techniques and tools that make SE effective in testing the implementation of stateful, complex protocols, where both

the structure of individual packets and their sequence of exchanges determine correct behavior.

Applying SE in this context first requires an understanding of how network protocols are structured and how their specifications define correct behavior.

1.1 Network Protocols

A network protocol defines a set of rules that govern how distributed software and hardware systems communicate. They specify the format and semantics of messages exchanged between entities and the order in which these messages must appear to achieve correct and reliable communication. Protocols operate at different layers of the communication stack, from link and transport layers to application layer, and examples include TCP [25], TLS [58], DTLS [59], QUIC [36], and CoAP [66]. Although protocols widely vary in purpose and complexity, all share the same fundamental goal of providing a well-defined interface that enables independent implementations to interoperate correctly.

Most network protocols are *stateful systems* in which each communicating entity maintains an internal state that tracks the progress of an interaction, and the processing of an incoming message depends on this state. For example, handshake messages establish parameters such as cryptographic keys, protocol versions, and sequence numbers that influence how subsequent messages are interpreted.

Protocol messages are also *highly parameterized*. They consist of multiple fields such as lengths, flags, versions, and sequence numbers that are often interdependent. These dependencies determine how a message is parsed and processed. For example, a length field defines the number of bytes that follow, a flag bit can change the message type, and a sequence number links the message to previous communication in the interaction.

Together, statefulness and parameterized message structures make protocol implementations particularly challenging to test, since the correctness of behavior depends both on message sequences and on complex combinations of parameter values.

Protocols may differ in purpose and environment. An IoT device, a web server, and a client application can all implement the same protocol under different operational constraints. Despite these variations, each implementation must conform to the same rules of interaction for this particular protocol to ensure interoperability. These rules and expected behaviors are formally defined in protocol specifications, which are introduced in the next section.

1.2 Specification vs. Implementation

A protocol specification describes the syntax and semantics of messages, the constraints on field values, the permitted ordering of messages, and the state transitions that occur during communication. For internet protocols, these specifications are published as *Requests for Comments (RFCs)* by the Internet Engineering Task Force (IETF). RFCs serve as the primary reference for implementers and define both mandatory and optional requirements through keywords such as *MUST*, *SHOULD*, and *MAY*. For example, RFC 6347 [59, p. 9] mandates that “implementations *MUST* either abandon an association or rehandshake prior to allowing the sequence number to wrap“ illustrating how specifications define concrete behaviors to preserve reliability and correctness. While a specification captures the abstract rules of communication, it is written in natural language and often includes explanatory text, loosely defined conditions, ambiguous phrasing, or even contradictory statements.

An implementation, by contrast, is a concrete realization of the specification in code. It translates the abstract protocol rules into data structures, algorithms, message parsing, and processing logic. Different implementations may be written in different programming languages, follow different architectural designs, or make use of varying libraries. In practice, developers often face trade-offs between strict conformance and other goals such as performance and compatibility. These trade-offs, combined with variations in interpretation and design choices, can result in subtle deviations from the intended protocol behavior.

Deviations between specification and implementation can appear in various forms. Some are benign and affect only non-essential functionality, while others may compromise interoperability or security. For example, an implementation might accept invalid message parameters, reuse sequence numbers incorrectly, or fail to enforce version-negotiation requirements. Such inconsistencies may lead to communication failures, denial-of-service vulnerabilities, or protocol downgrade attacks. Even when a deviation does not result in a direct vulnerability, it may still lead to unpredictable or non-standard behavior when interacting with other implementations.

Maintaining conformance between specification and implementation is particularly difficult as protocols evolve. New versions and extensions are published as additional RFCs, while clarifications and errata are issued to amend existing documents. Older implementations must remain compatible with the newer versions, and developers frequently rely on reference implementations, example code, or informal test suites that themselves may not fully adhere to the specification. Over time, such de facto standards can diverge from the official specification, making conformance testing necessary to ensure consistent behavior across implementations.

1.3 Protocol Conformance Testing

Protocol conformance testing is the process of assessing whether an implementation behaves according to the requirements defined in the protocol specification. Its goal is to ensure that the observable behavior of a system matches the behavior prescribed by the specification, thereby ensuring interoperability and correctness across independent implementations. Unlike functional testing, which focuses on whether a component produces the correct output for a given input, conformance testing evaluates compliance with externally defined standards.

Traditional approaches to conformance testing are often *black-box* methods, where the implementation is evaluated solely through its input-output behavior. Testers provide sequences of protocol messages as input and observe the resulting outputs, comparing them to the behaviors prescribed by the specification. Test cases are commonly derived manually from the specification, describing valid and invalid message exchanges together with oracles that determine whether the observed responses conform to the specification. While black-box approaches closely mirror real-world communication, their effectiveness is limited by the coverage and completeness of the test suite. Constructing such suites manually is time-consuming, and covering all possible states and message combinations is rarely achievable in practice.

Model-Based Testing (MBT) introduces a more systematic framework for black-box conformance testing. In MBT, an abstract behavioral model of the specification, such as a finite-state machine that describes the system's behavior in terms of inputs and outputs, is used to automatically generate test cases. Each generated sequence represents a potential interaction between protocol entities, and the observed outcomes are compared against the model's expected behavior to assess conformance. Model-based approaches improve coverage and make it possible to reason formally about test completeness. However, they rely on the availability of accurate formal models that capture the semantics of the specification. For many protocols, deriving such models from lengthy, natural-language RFCs remains a complex, error-prone, and often manual process.

White-box approaches incorporate detailed knowledge of the implementation, such as source code, control flow, and data dependency, to analyze its behavior against the specification. Within this category, SE has emerged as a particularly effective approach. SE executes the program symbolically rather than concretely, treating selected inputs as symbolic variables that represent many possible values. By reasoning about the conditions under which different execution paths are taken, SE can systematically explore a program's behavior and identify inputs that cause requirement violations. Symbolic execution is well suited for analyzing highly parameterized message structures, as it can reason about constraints on input fields and systematically explore combinations of parameter values that affect control flow. When extended to

handle sequences of interactions, it can also be applied to the analysis of stateful protocols, where both message content and interaction order determine correctness.

When requirements from the specification are expressed as assertions in the code or checked externally through monitors, SE can automatically detect deviations from expected behavior. It can also produce concrete test cases that reproduce identified issues, making them verifiable and repeatable. Symbolic execution therefore provides a systematic basis for testing protocol conformance, although applying it effectively to real-world protocols introduces several technical challenges discussed in the next section.

1.4 Research Challenges

Despite providing a systematic foundation for testing protocol conformance, SE faces several challenges when applied to real-world network protocol implementations. These challenges arise from both the technical complexity of SE itself and the characteristics of protocol implementations.

Path explosion

Symbolic execution in theory explores all feasible execution paths in a program. In practice, exploring every path is impossible since the number of paths grows exponentially with the program size and can even be infinite in the presence of unbounded loops or recursive calls. With highly parameterized messages and stateful logic, path explosion is unavoidable in network protocol implementations and must be accounted for in any realistic application of SE for testing conformance, as each new message and state transition can create additional branching conditions that expand the symbolic search space.

Stateful behavior

Network protocols often maintain internal state across sequences of messages, and the interpretation of an incoming message depends on the outcome of earlier exchanges. Classical SE is designed for stateless programs, in which each input is processed independently of all previous inputs. In protocols, however, validating certain requirements depends on observing specific sequences of message exchanges rather than isolated interactions. Applying SE to network protocol implementations therefore requires reasoning about how earlier symbolic inputs influence subsequent behavior.

Modeling network communication

Communication between protocol entities occurs through socket calls that handle the exchange of messages over the network. Classical SE, however, cannot directly reason about these interactions, since they take place outside the control of the program under analysis. Therefore, interactions performed

through sockets must either be modeled or abstracted to enable symbolic analysis of protocol implementations.

Formalization of requirements

Symbolic execution relies on oracles to determine whether observed behaviors conform to the protocol specification. Network protocol specifications, however, are written in natural language, and their ambiguities, cross-references, and conditional statements make it difficult to translate them into formal properties that can be checked automatically. As a result, defining precise conformance criteria that capture the intent of the specification remains a major obstacle to applying SE effectively in protocol testing.

The challenges outlined above define the scope of this thesis and motivate the research presented in the following chapters. The remainder of the thesis is organized as follows. Chapter 2 provides background on software testing and SE, describing both traditional and modern approaches to protocol testing. Chapters 3 to 6 summarize the research papers included in this thesis, each addressing a specific aspect of applying SE to protocol conformance testing. Chapter 7 reviews related work, Chapter 8 presents the overall conclusions drawn from this research, Chapter 9 outlines potential directions for future work, and Chapter 10 summarizes the main contributions of the thesis.

2. Background

2.1 Traditional Testing Methods

Testing has long been a central technique for assessing software correctness. Traditional approaches operate by exercising selected program behaviors under representative inputs and observing whether the outputs match the expectations defined by the relevant specification. In practice, this usually involves a hierarchy of testing activities. Unit tests target individual functions or modules in isolation, while integration and system tests examine interactions among components and validate end-to-end functionality in realistic settings. Regression tests ensure that previously correct behavior is preserved as the software evolves.

These techniques remain highly useful, yet their effectiveness is inherently limited by the scope of the selected test cases. Most software systems have large input spaces and complex control flow, so manually designed test suites typically cover only a fraction of the possible behaviors. Corner cases and rarely exercised paths often remain untested, allowing subtle flaws to stay undetected.

Even with modern advances, testing remains inherently incomplete, as test cases, whether constructed manually or generated automatically, explore only a subset of the behaviors a program may exhibit in practice. Testing can therefore increase confidence in correctness by revealing flaws in the observed executions, but it cannot guarantee their absence in real-world software systems.

While this fundamental limitation applies to all testing techniques, traditional approaches face additional practical constraints. Manually crafted tests scale poorly to complex systems, provide limited coverage of hard-to-reach behaviors, and require substantial effort to maintain as software evolves. These practical challenges motivated the development of more systematic and automated testing approaches, particularly in domains where correctness depends on structured inputs, ordered sequences of operations, or adherence to detailed specifications.

The next subsection introduces such automated methods in the context of protocol testing.

2.2 Modern Automated Approaches for Protocol Testing

As software systems grew in complexity, testing practices increasingly incorporated automation to improve coverage and reduce reliance on manually

crafted test suites. Automated techniques aim to explore program behavior more systematically, exercise hard-to-reach code paths, and reduce the effort required to maintain large test suites as software evolves. These goals are particularly relevant for protocol implementations, where correctness depends on structured message formats, precise constraints on message order, and stateful interactions.

A widely adopted automated technique is *Model-Based Testing* [10, 72], which derives test cases from an abstract behavioral model such as a finite-state machine. When sufficiently accurate models are available, this approach provides a systematic way to explore a broad range of message sequences and protocol states. Each generated sequence represents a potential interaction between protocol entities, and the observed behavior of the implementation is compared against the model’s expected behavior to assess conformance. However, constructing accurate models from natural-language specifications is labor-intensive, and inaccuracies can directly affect the resulting tests.

Fuzz testing offers a complementary strategy based on automatically generating large numbers of inputs, often by mutating valid messages or message sequences. Coverage-guided fuzzers [80, 27] prioritize inputs that trigger previously unexplored control-flow branches and have proven effective at exposing robustness issues such as crashes and memory-safety violations. Traditional fuzzers treat each input largely in isolation, which makes them less effective for exploring deeper protocol behaviors that depend on long-lived internal state or structured message flows. Recent state-aware fuzzers [52, 3] mitigate this limitation by incorporating message types and simple feedback on server responsiveness, enabling more targeted exploration of protocol state. However, their notion of state remains coarse-grained and heuristic, and they cannot direct the fuzzer toward exercising or violating specific semantic rules defined in the protocol specification, which reduces their utility for conformance testing.

Differential testing [43, 15, 13, 14] provides another automated technique suitable for protocols. By exercising multiple independent implementations with the same inputs and comparing their outputs, it can reveal behavioral differences in how implementations process those inputs. The inputs may vary not only in the sequence of messages but also in the specific values carried by individual fields, either of which can drive implementations into different behaviors. Such differences do not automatically indicate flaws, since protocol specifications often allow multiple conforming responses. The specification should therefore be consulted to determine whether a discrepancy is permitted or whether one implementation violates the expected behavior. The effectiveness of differential testing depends heavily on the richness of the test suite, because discrepancies can only be observed for the interactions that are actually exercised. Although differential testing avoids the need for an explicit behavioral model, it relies on the availability of multiple mature implementations.

Several other automated techniques have also been explored in the domain of protocol testing. Grammar-based input generation [24, 2] constructs syntactically valid messages from protocol grammars, allowing systematic exploration of field combinations, encodings, and optional structures that mutation-based fuzzing may miss. Model learning techniques [28, 30, 33] infer an implementation’s state machine from observed interactions and use the inferred model for targeted test generation or conformance checks. State fuzzing techniques [7, 21, 31, 32] combine inferred state models with fuzzing to guide mutations toward specific protocol states or transitions, enabling the exploration of behaviors that depend on multi-step interactions rather than isolated messages. Together, these approaches extend the behaviors that automated testing can exercise and complement model-based and mutation-based techniques.

Despite these advances, automated approaches still struggle to exercise certain behaviors, especially when those behaviors depend on specific branching choices or arise only under narrowly constrained conditions. Because input generation alone cannot always guide execution toward these areas, it is often useful to complement such techniques with approaches that reason directly about program logic. The next section introduces symbolic execution, which executes programs with symbolic inputs to explore feasible execution paths.

2.3 Symbolic Execution

Symbolic execution (SE) [37] is a white-box technique that analyzes programs in which some inputs or variables are designated as symbolic, and explores all code paths that are feasible for some assignments to these symbolic variables, thereby reasoning over the entire space of their possible values. During symbolic execution, the SE engine maintains two key structures for each explored path: a symbolic store (σ), which maps program variables to symbolic expressions representing their current values, and a path condition (PC), which accumulates the logical constraints that must hold along the path being explored.

In practice, SE engines such as KLEE [11] operate incrementally. As the program executes, each instruction is evaluated while tracking how its result depends on symbolic inputs: assignments update the symbolic store, memory loads retrieve the current symbolic expression of the addressed location, and conditional branches may fork execution when their predicates involve symbolic data. At each branch, the engine conjoins the branch predicate with the current path condition and consults an SMT solver to determine whether the branch is feasible. Infeasible branches are pruned immediately, while feasible ones continue with their updated symbolic state.

To illustrate, consider the simple C function in Listing 1, which is intended to return the larger of two integers provided as input. Although the function successfully compiles and may appear correct, it contains a subtle flaw: when

a equals b, the variable `result` is returned without being initialized. Reading an uninitialized variable in C constitutes *undefined behavior*, which yields arbitrary values or may cause a crash depending on memory state. This type of flaw is common in real-world systems and is notoriously difficult to detect through conventional testing.

```

1  int max(int a, int b) {
2      int result;
3      if (a > b)
4          result = a;
5      else if (a < b)
6          result = b;
7      // Missing assignment when a == b
8      return result;
9  }

```

Listing 1: Function returning the maximum of two integers.

Symbolic execution begins by initializing symbolic values a_0 and b_0 . The initial symbolic state is:

$$\sigma_0 = \{a \mapsto a_0, b \mapsto b_0\},$$

$$PC_0 = \text{true}.$$

The conditional statements in Listing 1 yield three feasible execution paths. When $a_0 > b_0$, corresponding to the if branch on line 3, the program returns a (Path 1). When $a_0 < b_0$, corresponding to the else if branch on line 5, it returns b (Path 2). Finally, when $a_0 = b_0$, execution skips both assignments, returning an uninitialized `result` (Path 3).

PATH 1: $PC_1 = (a_0 > b_0) \Rightarrow \text{return } a_0,$

PATH 2: $PC_2 = (a_0 < b_0) \Rightarrow \text{return } b_0,$

PATH 3: $PC_3 = (a_0 = b_0) \Rightarrow \text{uninitialized read of result}.$

For compactness, the symbolic state corresponding to the faulty path can be summarized as:

Symbolic state on Path 3

$$\sigma_3 = \{a \mapsto a_0, b \mapsto b_0, \text{result} \mapsto \perp\},$$

$$PC_3 = (a_0 = b_0).$$

When KLEE explores Path 3, the variable `result` has no prior assignment in the symbolic state. When execution reaches the `return result` statement,

the engine evaluates a load from the symbolic memory. Because `result` has no defined symbolic value, this load produces an *uninitialized read*, which KLEE reports immediately.

Although the solver is consulted incrementally at each branch to determine feasibility, KLEE also invokes the solver at the end of each feasible path to construct a concrete assignment (a witness) that satisfies that path condition. The paths, conditions, and corresponding witnesses are summarized in Table 2.1.

Path	Path Condition	Example Witness
1	$a_0 > b_0$	$(a, b) = (5, 3)$
2	$a_0 < b_0$	$(a, b) = (2, 7)$
3	$a_0 = b_0$	$(a, b) = (4, 4)$

Table 2.1. Feasible paths, path conditions, and corresponding witnesses for the program in Listing 1.

Finally, executing the program with the concrete values $(a, b) = (4, 4)$, the witness for Path 3 reveals the flaw: an uninitialized read of `result`. Symbolic execution engines typically report such behavior as an uninitialized variable read accompanied by a counterexample witness. The boxed state above clearly exposes the root cause: the symbolic store records $\text{result} \mapsto \perp$ (undefined) while the path condition PC_3 remains satisfiable.

Beyond this small example, SE provides several advantages as a testing technique. Its systematic exploration of feasible execution paths, combined with the use of SMT solvers to reason about constraints over symbolic variables, enables it to uncover subtle errors that are difficult to expose through conventional testing. The witnesses produced by the solver allow precise and reproducible test cases, and assertions or specification rules can be embedded directly into the tested program to serve as correctness oracles.

Several mature SE engines have demonstrated the practical value of the technique. KLEE [11] has been widely used to analyze system libraries and embedded utilities, achieving high coverage and uncovering numerous real-world bugs. S2E [17] extends SE to full-system environments, allowing analysis of complex binaries that interact with operating systems services and hardware abstractions.

Despite its strengths, SE also faces several well-known limitations.

Path explosion. The number of feasible execution paths in a program typically grows exponentially with the number of branches, a common phenomenon known as the path-explosion problem.

Constraint complexity. Constraint solving becomes increasingly expensive when programs manipulate pointer-heavy data structures, arrays with symbolic indices, or arithmetic expressions whose satisfiability requires non-linear reasoning.

Environment modeling. Real-world programs interact with their environment through file systems, network sockets, timers, and other operating-system services. Because symbolic execution is a white-box technique, it can only reason about behavior that is executed inside the symbolic interpreter. External services operate outside this scope and return concrete data such as file contents, incoming packets, or timer values that the engine cannot control or fork. For example, a network read depends on the exact bytes delivered by the OS, which cannot be treated symbolically without replacing the socket interface with a model that produces symbolic data. Some SE engines [11] therefore fall back on *concretization*, supplying fixed concrete values for such interactions when no model is available. While this allows execution to proceed, concretization restricts exploration to a single concrete behavior and may cause the engine to miss feasible paths that depend on alternative inputs. Symbolic models avoid this loss of coverage but are difficult to implement comprehensively and can significantly affect scalability.

Taken together, these challenges are amplified in the context of network protocol implementations. Reaching specific protocol states often requires carefully orchestrated sequences of inputs, some of which depend on subtle constraints encoded in the specification. Classical SE provides no mechanism to guide execution toward such message sequences, to represent protocol-level state transitions, or to validate conformance rules that are not directly reflected in the control flow of the implementation. Addressing these limitations requires extending SE with ways to control incoming protocol messages, to represent protocol state transitions, and to relate program paths to behaviors described in the specification. These extensions form the basis of the techniques developed in the remainder of this thesis.

3. From Specifications to Oracles: Requirement-Driven Symbolic Execution

Network protocols rely on precise specifications that describe how communicating parties should construct messages, how state is expected to evolve during an interaction, and how error conditions must be handled. These specifications are expressed in RFC documents, where natural language is used to formulate requirements. Many of these requirements involve relationships across multiple packets. For example, a field in one message may depend on a value in a previous message, or a sequence must follow a specific ordering that is valid for the selected key exchange method.

Testing such requirements is challenging. Traditional fuzzers concentrate on triggering crashes and often operate without awareness of protocol state. Standard symbolic execution (SE) tools are powerful for analyzing single inputs, yet they do not directly address multi-packet interactions or stateful requirements. When applied naively to protocol implementations, these tools either fail to explore relevant paths or become overwhelmed by the path explosion caused by large input spaces.

The work presented in this chapter introduces a technique that applies SE in a requirement-driven manner. The aim is to systematically search for inputs that violate specific RFC requirements and to produce concrete packet sequences that expose the deviation in the implementation under test. The objective is not limited to detecting crashes. It also focuses on revealing non-conformance flaws that may affect security or interoperability.

3.1 Technique

3.1.1 Extracting Specification Requirements

The first step is to extract protocol requirements from the RFC text and express them as logical predicates over the packets exchanged during a protocol session. RFCs typically use particular keywords such as "MUST", "MUST NOT", "REQUIRED", "SHALL", and "SHALL NOT" to indicate the strictness of a requirement [9]. These statements can be formulated into predicates that describe how packets must be structured and how they relate to one another.

Protocol specifications contain requirements of different kinds. Some requirements constrain the validity of individual packets or sequences of packets, specifying conditions under which inputs must be rejected. Other requirements relate received inputs to the outputs that an implementation is expected

to generate in response, for example by constraining fields in outgoing messages based on earlier inputs in the protocol session. In both cases, requirements often involve relationships across multiple packets and cannot be expressed as predicates over a single message in isolation.

To express such predicates precisely, we adopt a simple model of interaction. For DTLS, a session is viewed as a finite sequence of records received by one protocol party, which we denote by R . We write $r, r' \in R$ to range over individual records in the sequence. Equality between records refers to their position in the sequence: $r = r'$ means that they are the same occurrence of a record, not merely two records with identical contents. Each record is a structured object containing fields such as type, epoch, and sequence_number. For example, $r.\text{sequence_number}$ is used to refer to the sequence number of a record r .

As an example of an input-output requirement, consider the following rule from the DTLS RFC [59, p. 17] and its errata (ID: 5186), which concerns the handling of message sequence numbers during cookie exchanges:

In order to avoid sequence number duplication in case of multiple cookie exchanges, the server MUST use the message_seq in the ClientHello as the message_seq in its initial ServerHello.

Letting $\text{resp}(r, i)$ denote the i th output record generated in response to an input record r , this requirement can be expressed as:

$$\begin{aligned} \forall r \in R: r.\text{msg_type} = \text{client_hello} \implies \\ (\text{resp}(r, 1).\text{msg_type} = \text{server_hello} \implies \\ \text{resp}(r, 1).\text{message_seq} = r.\text{message_seq}) \end{aligned} \quad (3.1)$$

This predicate captures a dependency between an incoming record and a subsequent outgoing record and illustrates how input-output relationships specified in the RFC can be formalized. This formulation relies on the fact that the `msg_type` field of a record indicates the type of the enclosed (and possibly fragmented) handshake message.

A simple yet representative example of an input-validity requirement appears in the DTLS RFC [59, p. 13], which specifies the uniqueness of record sequence numbers within a session. Each DTLS record contains a 48-bit `sequence_number` field that uniquely identifies the record within a session. This field is incremented for every record sent, usually starting from zero. The receiver uses sequence numbers to process records in the correct order and to detect duplicates, since any two records with the same `sequence_number` indicate a replay. The requirement is formulated in the RFC as follows:

For each received record, the receiver MUST verify that the record contains a sequence number that does not duplicate the sequence number of any other record received during the life of this session.

This requirement serves as a protection against replayed records. For a set of records R received during a DTLS session, the requirement can be expressed in predicate logic as:

$$\forall r, r' \in R : r \neq r' \implies r.\text{sequence_number} \neq r'.\text{sequence_number} \quad (3.2)$$

This formula captures the essential rule that no two records may share the same sequence number. We will use this requirement as a running example in the remainder of this chapter to illustrate how formalized requirements are tested using SE.

A further example of an input-validity requirement that relates multiple records concerns the DTLS epoch field. The epoch identifies which set of session keys were used to protect a record, and the RFC specifies that its value should be initialized with zero and increase exactly when a *ChangeCipherSpec* is received [59, p. 8]. Let r_i denote the i th record in the sequence. For n received records in a DTLS session, we capture this requirement with the following formula:

$$\begin{aligned} &\forall i \in \mathbb{N}, 2 \leq i \leq n: \\ &\quad (r_1.\text{epoch} = 0) \wedge \\ &\quad (\text{if } r_{i-1}.\text{type} = \text{change_cipher_spec} \text{ then } r_i.\text{epoch} = r_{i-1}.\text{epoch} + 1 \\ &\quad \text{else } r_i.\text{epoch} = r_{i-1}.\text{epoch}) \end{aligned}$$

This formula specifies that the epoch of the first record is zero, and that for all subsequent records the value of the epoch field is either one greater than that of the previous record or equal to it, depending on whether the type field of the previous record is *change_cipher_spec*.

Once formulated, these predicates can be used to restrict SE to those parts of the input space where a violation of the requirement may occur.

Although the logical predicates are not executed directly, formulating the requirements in this way is still essential. RFC text is written in natural language and often describes relationships across multiple packets, message types, or stages of a handshake. Writing these requirements as predicates makes their meaning precise and exposes exactly which fields participate in them. This precision is necessary because SE must focus on the fields that matter for the requirement and explore the conditions under which the requirement may be violated. In short, the logical formulation defines the requirement in a clear and unambiguous way, and this definition is what guides the later testing steps.

3.1.2 Symbolic Execution

Symbolic execution is applied to each tested implementation through a dedicated test harness that replays a pre-captured sequence of records from a valid DTLS handshake and passes them one at a time to the implementation under

test. Each record is parsed into a structured representation, which enables SE to access and manipulate selected fields. The harness provides a deterministic environment by controlling the order of record processing and by removing nondeterministic behavior such as cookie verification or timeouts.

Symbolic execution requires the implementation under test to behave in a fully deterministic manner. Any nondeterministic behavior such as random number generation, retransmissions, or time-based actions would cause the symbolic executor to branch unnecessarily or to follow paths that differ from those observed during record capture. For this reason, the harness must execute deterministically so that the same control flow is obtained both when capturing the handshake and when symbolically executing it. A deterministic setup ensures that any variation observed by SE arises solely from symbolic fields. Pre-captured records are used for the same reason. A full DTLS handshake contains many fields that are irrelevant for a given requirement, and making entire records symbolic would cause path explosion. By replaying a fixed valid sequence and making symbolic only the fields involved in the requirement, we restrict SE to the part of the input space where a violation may occur. Parsing records into structured representations further supports this goal by allowing the analysis to target individual protocol fields rather than byte arrays.

The requirements introduced in the previous section fall into two classes. The first class consists of input-validity requirements, which specify conditions that each received record must satisfy. These requirements describe how the implementation should reject malformed inputs. When checking an input-validity requirement, SE makes the relevant fields symbolic and introduces assumptions that correspond to the negation of the requirement. This directs the analysis toward inputs under which the requirement may be violated, and an assertion is used to detect if the implementation processes such invalid inputs in a forbidden way. The second class consists of input-output requirements. These requirements describe how an implementation must respond to valid inputs, for example by producing an output message whose fields satisfy a specified relation. For these requirements, SE assumes that the input is valid and places the requirement itself in an assertion that checks the correctness of the generated output.

For input-validity requirements, SE should concentrate on requirement-violating inputs rather than on the vast space of valid inputs, so the harness introduces assumptions that encode the negation of the requirement. These assumptions restrict SE to inputs under which the requirement may be violated. The final step is to check whether the implementation uses the invalid input in a forbidden way. Protocols differ in how malformed inputs must be handled. Some specifications require discarding the input, others mandate generating an alert, and others require immediate termination of the connection. In DTLS, the general rule [59, p. 14] for handling invalid records requires discarding them and states that implementations may generate a fatal-level alert message.

For DTLS, we therefore approximate forbidden behavior by checking whether the handshake nonetheless progresses. A failing assertion is inserted at the point where the implementation reaches the code that completes the handshake. If SE reaches this point, it means that the implementation advanced through the handshake despite the invalid input, which indicates a violation of the requirement.

The predicate in Equation (3.2) formulates an input-validity requirement that states that no two records in a session share the same sequence number. To apply our technique to evaluate this requirement for a server, the harness loads the sequence of pre-captured records that a DTLS server would receive during its handshake and marks the client-side `sequence_number` fields as symbolic. All other fields remain concrete.

An assume statement is then introduced that indicates that at least two symbolic sequence numbers are equal. This directs SE to explore precisely those assignments that violate uniqueness. To evaluate the effect of this invalid input, a failing assertion (e.g., `assert(false)`) is placed at the code location that indicates successful completion of the handshake. If SE reaches this assertion, it means that the implementation completed the handshake even though it received a record with a duplicate sequence number. The assertion then fails, and the SE engine reports the concrete assignments that triggered this behavior.

In our implementation, handshake completion is detected by instrumenting the code at the point where the implementation expects to receive and process the *Finished* message. This point marks the transition into the encrypted phase of DTLS and therefore indicates that the handshake has completed successfully. A failing assertion is inserted at this location. If SE reaches this assertion, it means that the implementation progressed through the handshake despite having received an invalid input earlier in the sequence. This provides a uniform and lightweight method for detecting forbidden progress across different DTLS implementations without requiring detailed inspection of their internal state machines.

3.1.3 Test Case Construction and Validation

When SE discovers a violating execution path, the engine provides concrete values for all symbolic fields involved. To construct a test case, these values are substituted back into the structured record representations used by the test harness, while all other fields retain their original concrete values from the pre-captured handshake. The resulting records are then serialized into byte arrays that follow the exact wire format of the protocol.

For the sequence number requirement, this means that SE produces a concrete assignment in which two records share the same sequence number. These values replace the symbolic fields in the corresponding records, creating a con-

crete record sequence that instantiates the violation of the predicate in Equation (3.2).

These concrete record sequences are replayed against the unmodified implementation outside the SE environment. Validation is done by observing whether the unmodified implementation accepts the sequence and completes the handshake. Since handshake completion is used as an approximation for protocol progress, a successful handshake in the presence of an invalid record constitutes a confirmed violation of the corresponding requirement. In the sequence number example, if the server completes its handshake despite the duplicate sequence number, the requirement is violated and the test case is confirmed.

3.2 Evaluation

The technique was applied to four implementations of the DTLS protocol. These included OpenSSL, MbedTLS, and two variants of TinyDTLS. A collection of sixteen requirements from the DTLS RFC [59] were formulated and tested. These requirements ranged from record layer constraints, concerning content type, version fields, and epoch handling, to handshake layer rules related to message ordering, fragmentation, and message sequence numbers.

The evaluation revealed a total of *thirty six* unique bugs, seven of which were classified as vulnerabilities. Several issues were memory related, including out of bounds accesses during message reassembly. Others concerned logical violations of the specification, such as incorrect validation of version fields, improper handling of repeated handshake messages, and failures to respect message ordering rules. Many issues were identified within a short symbolic exploration time, and all of them were acknowledged and fixed by the developers.

In addition to uncovering new issues, our technique also reproduced a previously known exploitable vulnerability in OpenSSL’s function for reassembling fragmented messages. This reproduction confirms the power and promise of our technique.

For a detailed description of the technique’s implementation, the full set of input-validity and input-output requirements we evaluated, and a comprehensive analysis of the issues we discovered, the reader is referred to Paper I.

3.3 Limitations

The technique presented in this chapter has two main limitations that motivate the next contribution of this thesis. The first limitation is that the code responsible for checking requirements must be directly inserted into the source code

of the implementation under test. This involves adding calls to a shared library at specific locations in the implementation. Since these locations differ between implementations and between versions, the process requires a detailed understanding of the code base and must be repeated whenever the implementation changes. This reduces the portability of the technique and increases the manual effort required to apply it.

The second limitation concerns the reliance on completing a protocol interaction. In the case of DTLS, this means that SE must reach either the successful completion of the handshake or a definite failure triggered by the invalid input. As a result, SE may be forced to explore many paths that lie far beyond the part of the execution that is relevant to the requirement being tested. This unnecessarily increases the symbolic search space and can significantly slow down the detection of violating inputs.

These two limitations motivate the need for a technique that avoids instrumentation of the implementation and that can detect requirement violations without relying on the completion of a full protocol exchange. The next chapter introduces a monitor-based approach that addresses both issues by moving requirement checking outside the implementation and by enabling earlier detection of violation points.

4. Monitor-based Symbolic Execution for Stateful Protocols

Testing stateful network protocol implementations is challenging because the processing of each packet often depends on a potentially long sequence of previously exchanged packets. As discussed in Chapter 3, detecting requirement violations using symbolic execution (SE) becomes challenging when checks must be inserted in many locations inside an implementation or when violations surface only after the completion of an entire interaction. This chapter introduces a monitor-based SE technique that moves requirement checking outside the system under test (SUT). The technique addresses limitations identified in Section 3.3 by enabling early detection of requirement violations and by avoiding intrusive instrumentation of protocol implementations.

The technique combines SE with external monitors. A monitor is a software component that is invoked at specific points during protocol interaction as packets are delivered, observes the packets exchanged between protocol parties (e.g., client and server), and maintains information about the interaction state. Based on this information, the monitor can check whether packets sent by the SUT conform to the protocol specification. Monitors may also designate selected packet fields as symbolic, which allows SE to explore a full range of potentially violating inputs. Since monitors are separated from the SUT, and depend only on the protocol (and not a specific implementation), the same set of monitors can be reused across different implementations and versions of a protocol. This separation of requirement logic from implementation code improves reuse, reduces maintenance effort, and allows the technique to scale across protocol implementations.

The technique consists of several components. We begin by introducing monitors, which form its core.

4.1 Technique

4.1.1 Monitors

Monitors serve as the central abstraction of the technique. A monitor is synchronously invoked during the protocol interaction to observe packets exchanged between protocol parties. Based on these observations, it maintains its own internal state and checks whether the SUT behaves according to the protocol specification. To cover all potentially requirement-violating values

in input fields, the monitor may designate selected fields of incoming packets as symbolic. An SE engine will then explore all execution paths that can be reached for any values of these fields, including values that expose requirement violations. In addition, SE will discover memory errors, crashes, and other runtime problems in the SUT. A monitor therefore acts as an external component that inspects and manipulates communication while leaving the SUT unmodified. When an assertion in the monitor fails, the corresponding execution path is terminated and a violating packet assignment is reported.

We represent monitors as state machines. Each monitor has a control state that indicates where it is in the requirement checking process and may contain a set of auxiliary variables that track the parts of the interaction relevant for the requirement. In this representation, each transition is written with a condition above a horizontal bar and an action below it. The condition specifies when the transition applies and is expressed in terms of the currently observed packet P . When the condition holds, the monitor performs the transition and executes the statement below the bar. These statements may update auxiliary variables, introduce symbolic values for selected packet fields, restrict these values using assumptions, or check requirements using assertions.

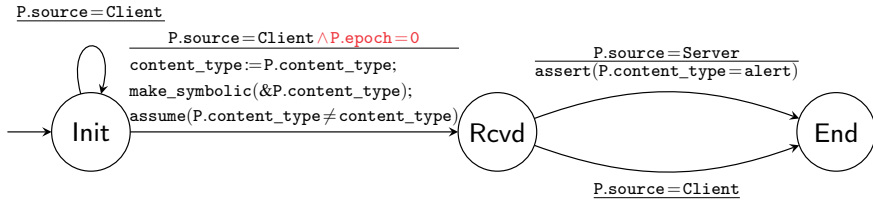


Figure 4.1. Monitor for the Matching Content Type requirement of DTLS.

The monitor in Figure 4.1 checks that the DTLS server correctly handles records whose content type does not match the enclosed fragment.

According to the TLS specification [22, p. 19], the content type field in the record layer determines the higher-level protocol used to process the enclosed fragment. The content type field is defined as a one-byte value drawn from a fixed set, one of which corresponds to *alert*. A mismatch between the content type and the enclosed fragment therefore renders a record invalid. The DTLS specification further prescribes how such invalid records must be handled. In particular, it states that invalid records should be silently discarded and that implementations which choose to generate an alert must generate a fatal alert [59, p. 14]. This requirement can be checked by observing how a server reacts to records with invalid content type values.

Leaving aside the optional conjunct on the epoch field, shown in red, the monitor operates as follows. In the initial state *Init*, it passively observes client-generated records and may nondeterministically select one of them to invalidate by making its *content_type* field symbolic and constraining it to differ

from its original value. This nondeterministic choice allows SE to explore violations introduced at arbitrary points in the interaction, effectively checking the requirement for all client-generated records. When such an invalidation is introduced, the monitor transitions to the state *Rcvd*.

Once in state *Rcvd*, the monitor waits for the next packet sent by the server. If the server responds by generating an *Alert*, the requirement is satisfied and the monitor transitions to the terminal state *End*. If the server instead continues processing the invalid record and produces a packet with a different content type, the assertion on the corresponding transition fails and the violation is reported. If a client packet is observed while in state *Rcvd*, the monitor also transitions to *End*, reflecting the case where the server silently discards the invalid record. The terminal state *End* ensures that the monitor performs a single check per execution path and avoids repeated invalidations within the same interaction.

The additional guard ($P.\text{epoch} = 0$) restricts this manipulation to records that are not yet protected by encryption. In DTLS, records with epoch zero carry plaintext fragments, whereas records with nonzero epoch values contain encrypted payloads. Although the content type field itself is not encrypted, it influences subsequent cryptographic processing when the epoch is nonzero. Restricting symbolic manipulation to records whose epoch is zero therefore avoids requiring SE to reason about expensive cryptographic operations.

Although the example in this section focuses on DTLS, we applied the same monitor-based technique to QUIC by encoding requirements over packet headers, frames, and connection state.

4.1.2 Creating a Test Harness

The technique requires a test harness that executes the protocol parties and invokes a monitor whenever a packet is received by one of them. The harness is executed symbolically by the SE engine and is designed as a uniform component that can be equipped with any arbitrary monitor. It is independent of the particular requirement being tested. The extent of the protocol interaction that the harness performs depends on the monitor: it may involve only a simple message exchange in stateless protocols, or it may require the parties to complete a full handshake and exchange application data. Whenever one protocol party sends a packet, the harness makes this packet available to the receiving party and invokes the monitor on a structured representation of the packet before the SUT processes it. In this way, the monitor can inspect the packet, introduce symbolic values, or impose additional assumptions before the SUT observes the input.

When using KLEE as the SE engine, the harness must consolidate all involved protocol parties into a single-threaded process and coordinate their interactions explicitly. Each party takes turns processing the packets available

to it, and the harness advances the protocol interaction by alternating between parties until the interaction terminates or a monitor assertion fails.

4.1.3 Encoding Monitors as Programs

Monitors are represented as state machines, similar to the one displayed in Figure 4.1, but to be usable during SE they must be encoded as executable programs. Each monitor is therefore translated into a small program that maintains its control state, stores auxiliary variables, and evaluates transitions whenever a packet is delivered. Since we use KLEE as the SE engine, we are constrained to use a programming language that can be compiled to LLVM bitcode. In our implementation, monitors are written in C.

The programmatic representation of a monitor preserves the structure of the underlying state machine. The control state is implemented as an enumeration, and the monitor keeps a data structure that holds this state along with any local variables it needs. Each transition is implemented as a conditional block that checks whether its guard is satisfied by the currently observed packet. If the guard holds, the corresponding action is executed. Actions may update auxiliary variables, introduce symbolic values, add assumptions, or check assertions.

Whenever the state machine contains a nondeterministic choice between multiple outgoing transitions, the translation introduces a symbolic variable that ranges over the available alternatives. When this symbolic value is evaluated, KLEE forks execution, allowing the SE engine to explore each possible outcome of the nondeterministic choice.

If no transition is applicable, the monitor takes no action and the interaction proceeds. In this case, SE continues without imposing additional constraints on the packet or the SUT.

4.1.4 Invoking Monitors

Once a monitor has been encoded as an executable program, it must be invoked at the appropriate points during SE. The test harness orchestrates the protocol interaction and ensures that every packet delivered from one protocol party to another is first presented to the monitor. In practice, this is realized by implementing the monitor as a callback function that is invoked just before the packet is delivered to the protocol party. The callback receives a structured representation of the packet as its parameter.

Monitor invocation is synchronous: the SUT does not observe a packet until the monitor has completed its evaluation. This ordering is essential, because the monitor may introduce symbolic values, apply assumptions, or raise assertions that influence how SE proceeds. If a transition introduces symbolic values or assumptions, the SE engine forks execution to explore the resulting

paths. If an assertion fails, the SE engine terminates that path and reports the violating packet assignment.

As a consequence of this online invocation model, packets do not need to be pre-captured or replayed from execution traces. Monitors are executed directly on packets as they flow between protocol parties, eliminating the need for a separate trace collection and replay infrastructure. This design also simplifies testing across different protocol or implementation configurations, as changes to configuration parameters do not require regenerating or adapting traces.

4.1.5 Developing Protocol-specific Infrastructure

Monitors check requirements over protocol fields such as content type, sequence number, and epoch. To facilitate this, the technique advocates developing protocol-specific infrastructure that exposes packet contents in a structured form suitable for inspection and manipulation by monitors. Conceptually, this infrastructure translates packets between their wire format and an internal representation before and after monitor execution.

4.1.6 Applying Small Modifications to the SUT

For some SUTs, it is beneficial to apply small changes that simplify the application of SE. When a protocol transmits packets in encrypted form, disabling encryption can simplify the analysis. This avoids the cost of performing encryption and decryption and prevents the symbolic executor from having to reason about cryptographic operations. A similar consideration applies to timing-related behavior. Many protocol implementations trigger retransmissions or abort the interaction if no response is received within a certain time window. To prevent such behavior from interfering with symbolic exploration, the timeout can be set to its maximum value or disabled entirely.

4.2 Implementation

This section describes how the monitor-based SE technique introduced above is realized in practice.

4.2.1 Generic Infrastructure

Many protocols exchange packets over POSIX sockets using an API provided by an operating system library [68]. Our generic infrastructure provides a custom implementation of these APIs that acts as a proxy and redirects communication between the SUT and its peers. More concretely, functions that send packets, such as `sendto`, are replaced by functions that place the packet into a

shared queue. Similarly, functions that receive packets, such as `recvfrom`, are replaced by functions that remove the first packet from the queue and invoke the monitor before returning the packet to the caller.

Processing an incoming packet follows three steps. First, the packet at the head of the queue is parsed using protocol-specific infrastructure and stored in a structured packet object `P`. Second, the monitor is invoked with `P` as input. Third, after the monitor completes its evaluation, the packet, whose fields may have been manipulated, is serialized back into its wire format and returned to the SUT.

A separate packet queue is maintained for each direction of communication between protocol parties, for example from client to server and from server to client. The `bind` function is redirected to an initialization function that creates the packet queues. All other library functions remain unchanged.

4.2.2 Protocol-specific Infrastructure

Our implementation consists of the following protocol-specific components.

Set of Monitors

Monitors are encoded as C functions using the translation described in Section 4.1.3. Each function takes as parameter a pointer to the data structure that represents the currently observed protocol packet, and accesses packet fields by selecting the corresponding fields in this structure.

Figure 4.2 shows the C implementation of the monitor for the Matching Content Type DTLS requirement for servers (shown in Figure 4.1). The code closely follows the structure of the abstract state machine. It first declares an enumeration that encodes the monitor's control states and a static variable that stores the current state. The body of the monitor function consists of a sequence of conditional blocks, each corresponding to a transition of the abstract state machine. Within each block, the guard of the transition is evaluated over the fields of the current packet, and, if the guard holds, the associated actions are executed before the monitor moves to the target state.

The only difference from the abstract representation is the self-loop transition in the initial state that is handled by an infrastructure that supports non-determinism between two transitions. To this end, each packet processed during an interaction is assigned an index, and a symbolic variable ranging over possible packet indices is introduced. A non-self-loop transition is taken only if its guard holds and the index of the current packet matches the symbolic index; otherwise, the self-loop is taken. Since the index is symbolic, the symbolic execution engine explores executions in which the monitor remains in a given control state for an arbitrary number of packets before taking the transition.

```

1  enum STATE { INIT, RCVD, END };
2
3  static STATE curr_state = INIT;
4
5  void matching_content_type_server(RECORD *P) {
6      if (curr_state == INIT && P.source == CLIENT &&
7          byte_to_int(P->epoch, sizeof(P->epoch)) == 0) {
8          uint8_t content_type = P->content_type;
9
10         kleener_make_symbolic(&P->content_type,
11             sizeof(P->content_type), "content-type");
12         klee_assume(P->content_type != content_type);
13         curr_state = RCVD;
14     }
15     else if (curr_state == RCVD && P.source == SERVER) {
16         if (P->content_type != Alert_REC)
17             assert(0 && "Mismatching Content Type");
18         curr_state = END;
19     }
20     else if (curr_state == RCVD && P.source == CLIENT)
21         curr_state = END;
22     else if (curr_state == END) return;
23 }

```

Figure 4.2. The C implementation of the monitor of Figure 4.1.

Parser and Serializer

Packets exchanged by protocol implementations are typically represented as byte arrays. To simplify the development of monitors, we provide a protocol-specific parser that translates received packets into structured data objects before invoking the monitor. This allows monitors to inspect and manipulate individual packet fields directly. After the monitor has completed its execution, a serializer converts the possibly modified data structure back into its wire format before it is delivered to the receiving protocol party. These parser and serializer components are optional, but they significantly reduce the complexity of monitor implementations by avoiding the need to manipulate raw byte offsets within packet buffers.

In some cases, small SUT-specific adjustments are applied to simplify SE, such as disabling encryption or extending timeout values. These changes are not required for correctness, but they reduce analysis complexity and avoid symbolic reasoning over cryptographic operations or timing-related behavior.

4.3 Evaluation

The technique was evaluated on multiple stateful network protocol implementations, focusing primarily on DTLS and QUIC. The evaluation considered

both client-side and server-side implementations and covered requirements drawn directly from the corresponding RFC specifications [59, 22, 36]. These requirements included protocol-specific constraints at the packet and record level, such as content type handling and epoch processing in DTLS, as well as QUIC-specific checks related to frame validity, packet number handling, and error signaling, in addition to higher-level protocol rules such as message ordering and error handling.

The results demonstrate that the monitor-based technique is effective at detecting both logical specification violations and low-level implementation errors. Across the evaluated implementations, the technique uncovered *twenty two* previously unknown bugs, including incorrect handling of malformed records, missing validation checks, and improper reactions to protocol violations. Notably, all issues previously identified using the technique presented in Chapter 3 were rediscovered considerably faster and with substantially less manual effort. This improvement is primarily due to the fact that requirement checking is performed externally by monitors and can be triggered as soon as a violating packet is observed. As a result, violations can be detected without requiring the protocol handshake to complete.

Overall, the evaluation indicates that the monitor-based symbolic execution technique presented in this chapter provides a practical and efficient means of testing stateful protocol implementations for conformance to their specifications. For a detailed description of the DTLS and QUIC requirements and their monitors, the experimental setup, and the complete set of discovered issues, the reader is referred to Paper II.

4.4 Limitations

The effectiveness of the monitor-based SE technique fundamentally depends on the quality and completeness of the specification-based oracle encoded by the monitors. Each monitor represents a manually extracted requirement from the protocol specification, and violations can only be detected for those behaviors that are explicitly captured by such monitors. Consequently, any requirement that is omitted, misunderstood, or encoded imprecisely will not be checked during SE.

This limitation is particularly challenging for complex network protocols such as DTLS and QUIC, whose specifications are lengthy, intricate, and in some cases ambiguous. Manually identifying and encoding all relevant requirements and corner cases as executable monitors is labor-intensive and error-prone. Even when SE is used to explore a wide range of inputs, the analysis remains inherently incomplete, as it is constrained by the coverage of the specification-based oracle. Bugs that manifest in behaviors not explicitly encoded by the oracle may therefore remain undetected.

Moreover, some protocol behaviors are not governed by strict, explicitly stated rules, but instead arise from underspecified parts of the standard. In these cases, different implementations may legitimately make different design choices while still remaining compliant with the specification. Although specification-based monitors can be written to allow for multiple compliant behaviors, accurately capturing the full space of behaviors permitted by a protocol specification is often difficult. RFCs frequently contain such underspecified aspects, and the set of behaviors that are considered compliant may not be precisely characterized. As a result, constructing monitors that faithfully reflect all allowed behaviors can be challenging, and some compliant but divergent implementation behaviors may not be detected.

These limitations highlight that, while monitor-based SE is effective for detecting violations of explicitly stated requirements, it does not address the fundamental incompleteness of specification-based oracles. This motivates the need for complementary approaches that can uncover protocol nonconformances without relying on explicitly specified oracles.

5. Beyond Oracles: Differential Symbolic Execution

The techniques presented in the previous chapters rely on the existence of an explicit oracle that characterizes correct protocol behavior. In Chapter 3, correctness is defined through logical predicates extracted from protocol specifications and checked by instrumenting the implementation under test. In Chapter 4, this notion is preserved, but the responsibility for checking requirements is moved outside the implementation and delegated to external monitors. In both cases, symbolic execution (SE) is guided by a specification-based oracle that encodes how an implementation is expected to behave.

As discussed in Section 4.4, the effectiveness of specification-based oracles fundamentally depends on the completeness and precision of the requirements extracted from the protocol specification. Violations can only be detected for behaviors that are explicitly captured by monitors, and any requirement that is omitted, misunderstood, or encoded imprecisely remains unchecked. This dependence is particularly challenging for complex protocols such as DTLS and QUIC, whose specifications are lengthy, intricate, and in some cases ambiguous. Even when SE explores a wide range of inputs, the analysis remains inherently incomplete, as it is constrained by the coverage of the encoded oracle.

In addition, not all protocol behaviors are defined by strict, explicitly stated rules. Many aspects of protocol behavior arise from underspecified parts of the specification, where multiple implementation choices may be considered compliant. Accurately capturing the full space of such permitted behaviors in the form of executable monitors is often difficult. As a result, implementations may exhibit divergent behaviors that do not violate any explicitly stated requirement and therefore remain undetected by specification-based checking, even though such discrepancies may indicate subtle inconsistencies, corner cases, or potential interoperability issues.

This chapter considers protocol testing scenarios that arise precisely in the presence of such incomplete or underspecified oracles. Rather than checking an implementation against an explicit set of specification-derived requirements, we adopt a comparative perspective and analyze how multiple implementations behave when exposed to the same inputs. The underlying assumption is that independently developed implementations of the same protocol are expected to exhibit equivalent externally observable behavior when processing identical packet sequences. Deviations from this common behavior may indicate implementation errors, inconsistent interpretations of the specification, or behaviors that are not explicitly characterized by the specification.

To explore such discrepancies systematically, this chapter introduces a differential SE technique for network protocol implementations. The technique symbolically executes multiple implementations on shared symbolic packet inputs and observes their externally visible behavior. When SE produces execution paths whose responses differ across implementations for a common input, the subsequent analysis reports a behavioral discrepancy and extracts concrete packet values that induce it. At this stage, no judgment is made about which behavior is correct. The resulting packet sequences are instead used as test cases whose effects are analyzed with reference to the protocol specification to determine whether the discrepancy corresponds to a violation, an underspecified behavior, or a permissible implementation choice.

The implementation of the differential SE technique builds directly on the monitor-based infrastructure introduced in Chapter 4. The same test harness, packet parsing infrastructure, and SE setup are reused. The key difference lies in how observations are interpreted. Monitors are no longer used to check specification requirements, but instead to observe packet exchanges and record execution outcomes that are later analyzed to identify behavioral discrepancies across implementations. This reuse allows differential SE to be integrated into the existing framework while addressing aspects of protocol behaviors that are difficult to capture using specification-based oracles alone.

5.1 Technique

5.1.1 Creating a Test Harness

Differential SE is performed within a test harness that orchestrates a predefined interaction between a protocol implementation under test and another protocol party. As in the monitor-based technique, the purpose of the test harness is to provide a controlled and deterministic execution environment. In the differential setting, the harness additionally ensures that multiple implementations are exposed to equivalent inputs.

For each system under test (SUT), a separate test harness is constructed. Each harness connects two components. The first component is the SUT, which is the protocol implementation whose behavior is being analyzed. The second component is another protocol party, which we refer to as the *facilitator*. The facilitator provides the inputs that drive the protocol interaction forward. Depending on the protocol and the role of the SUT, this interaction may involve completing a handshake or, more generally, exchanging a predefined sequence of packets between protocol parties.

The facilitator implements the complementary protocol role to that of the SUT. For example, in a client/server protocol such as DTLS, if the SUTs are server implementations, the facilitator is a client implementation. For a given protocol role of the SUT, the same facilitator implementation is used for all tested SUTs. Although a separate harness is constructed for each SUT, these

harnesses share the same structure and differ only in the concrete SUT that is being tested. Reusing the same facilitator ensures that all SUTs are subjected to identical inputs and interaction patterns, and that any observed behavioral differences are due to differences in the SUT implementations rather than from variations in their peers.

To illustrate this setup, we use a running example based on the sequence number handling requirement discussed in Chapter 3. Suppose we wish to compare the behavior of the DTLS server implementations of Mbed TLS and wolfSSL. Applying differential SE requires constructing two test harnesses, one for each server implementation. Both harnesses use the same DTLS client implementation as the facilitator, for example a client from the Mbed TLS library, although other choices such as OpenSSL are equally possible. Each facilitator drives the interaction by sending the same sequence of handshake messages to its corresponding server. Figure 5.1 shows the architecture of one of the test harnesses used in this running example.

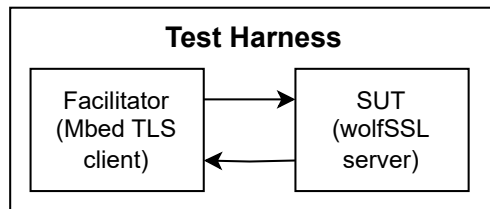


Figure 5.1. Architecture of the test harness used in the running example to test a DTLS server implementation.

5.1.2 Running Symbolic Execution

Assuming that multiple protocol implementations are available, basic differential testing can already be performed. Concretely, the SUT can be executed on a predefined interaction using the corresponding test harness, selected response packets can be observed, and pairwise differences can be flagged. Such testing can reveal discrepancies for specific packet sequences, but it remains limited to concrete interactions that are manually selected. Leveraging the power of SE extends this basic form of differential testing by systematically exploring variations in selected packet fields and by uncovering behaviors that arise only for specific input values and that may later cause discrepancies.

Differential SE is performed by executing each test harness under an SE engine. As in the monitor-based technique, the harness executes in a controlled and deterministic environment. However, rather than relying on assertions derived from specification requirements, SE is used to explore how different implementations respond to symbolic variations in protocol inputs. Selected

fields of packets generated by the facilitator are made symbolic, while all other fields retain their original concrete values.

The SE engine explores the code paths that are feasible for some assignment to the symbolic fields. For each explored execution path, the engine records the externally observable outputs produced by the SUT. These outputs include packets sent by the SUT or the absence of a response after processing a packet whose field was made symbolic. Technically, outputs are recorded as symbolic expressions that capture how they depend on the symbolic input fields. This representation allows the analysis to reason about output behavior over a large input space without enumerating all concrete assignments explicitly.

Differential SE is conducted in a campaign-based manner. For each pair of SUTs, multiple differential SE campaigns are executed. Each campaign analyzes a specific protocol interaction and makes a specific set of fields symbolic. For example, in DTLS, one campaign may focus on the handshake for a specific cipher suite and investigate how the implementation responds to different values of the record sequence number in input, while another campaign may target a different cipher suite.

Monitors play an important but fundamentally different role in the differential setting than in the specification-based technique introduced in Chapter 4. In differential SE, monitors do not encode protocol requirements and do not check assertions. Instead, they are used to observe packet exchanges and to record selected responses of each implementation during SE. In particular, monitors record packets generated by the SUT or the absence of a response and associate these responses with the symbolic inputs that led to them. This information is later used to compare the behavior of different implementations.

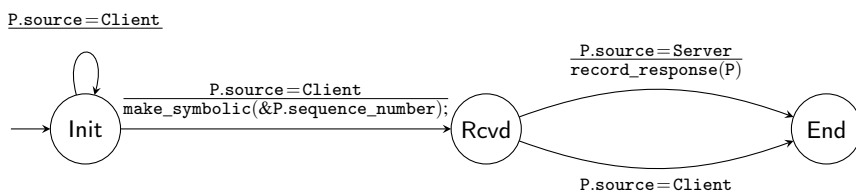


Figure 5.2. Monitor recording a DTLS server’s responses for different values of sequence_number.

In the running example, we apply this technique to find differences between the Mbed TLS and wolfSSL servers. To make the role of monitors in the differential setting concrete, Figure 5.2 shows the monitor used in the running example to observe DTLS server responses for different values of the sequence_number field. The monitor is invoked on every packet exchanged during the interaction. In the starting state Init, it passively observes packets sent by the client. At a nondeterministically chosen client packet, the monitor makes the sequence_number field symbolic and transitions to the state

Rcvd. From this point onward, the monitor waits for the next packet sent by the server. When such a packet is observed, the monitor records the server's response using the `record_response` function and transitions to the terminal state `End`. If another client packet is observed instead, the monitor assumes that the server has generated no response and transitions to `End`. Reaching the state `End` terminates the current path and prompts the SE engine to record the corresponding path condition.

Running SE with such a monitor produces, for each SUT, a set of explored paths. To compare the behavior of different SUTs, it is necessary to ensure that a symbolic field is introduced at the same point in the protocol interaction. To this end, we associate each symbolic field with a specific position in the packet sequence exchanged during the interaction. This position, which we refer to as the *index*, identifies the point during the interaction at which a packet field is made symbolic.

The *index* is not tracked explicitly by the monitor. Instead, the underlying infrastructure assigns a concrete counter value to each packet processed during the interaction and introduces a symbolic variable that selects the packet on which the monitor will act. This mechanism is the same as the one used in the monitor-based technique described in Section 4.2.2. Both the selected index and the symbolic field become part of the path condition recorded by SE. For every explored path, the SE engine records this path condition together with the response observed for the packet in which the field was made symbolic.

Finally, to limit the number of explored paths and to reduce the manual effort required for analysis, each differential SE campaign employs a single monitor that makes one instance of a protocol field symbolic. All other instances of the same field in the interaction remain fixed to their concrete values. This design choice isolates the effect of a single symbolic input at a well-defined interaction point, which simplifies both SE and the interpretation of discrepancies.

While it is possible to make multiple fields symbolic within a single campaign, doing so significantly increases the number of explored paths and makes it harder to attribute observed discrepancies to specific symbolic fields. We therefore restrict each campaign to a single symbolic field instance in order to keep the analysis manageable and the resulting discrepancies easier to interpret.

5.1.3 Finding and Analyzing Discrepancies

Given two SUTs, the goal of differential SE is to identify common inputs that cause the SUTs to produce different responses when a symbolic field is introduced at the same interaction point. Discrepancy detection is performed as a separate, offline analysis phase that operates on the results produced by SE. The process consists of three main steps, which we describe below.

Deriving Path-Output Sets.

For each SUT, SE explores a set of execution paths. For every explored path, the SE engine records the path condition together with the response observed for the packet in which the field was made symbolic. We represent each such record as a *Path-Output* pair of the form $\langle pc, o \rangle$, where pc denotes the path condition and o denotes the recorded response. Both pc and o are parameterized by the symbolic field and the packet index. Let S_I denote the set of Path-Output pairs generated by the SE campaign for implementation I .

Deriving Discrepancies Between Implementations.

Given the sets S_I and $S_{I'}$ for two implementations I and I' , we compare these sets to determine whether there exists a common input under which the two implementations produce different responses. Because each Path-Output pair is associated with a specific packet index, comparisons are restricted to pairs that correspond to the same interaction point, ensuring that symbolic fields are introduced at the same point in the interaction.

If a concrete input causes I and I' to produce different responses, then there exist a Path-Output pair $\langle pc, o \rangle \in S_I$ and a Path-Output pair $\langle pc', o' \rangle \in S_{I'}$ such that both path conditions are satisfied by the same input, while the resulting responses to this specific input differ. When such a concrete input instantiates the symbolic field, the formula

$$(pc \wedge pc') \wedge (o \neq o') \quad (5.1)$$

is satisfied.

The conjunct $(pc \wedge pc')$ ensures that such a common assignment to the symbolic input fields satisfies both path conditions, while the conjunct $(o \neq o')$ ensures that this assignment leads to different responses. For each pair of Path-Output pairs $\langle pc, o \rangle$ and $\langle pc', o' \rangle$ with the same packet index, we check the satisfiability of Formula (5.1) using an SMT solver. If the formula is satisfiable, we ask the solver to produce a valuation for the symbolic input field and the corresponding packet index. Each such valuation, together with the differing responses o and o' , constitutes a *discrepancy witness*. Our technique produces a witness for every pair of Path-Output pairs that satisfies Formula (5.1).

Analyzing Discrepancies for Bugs.

For each discrepancy witness, the protocol specification is consulted to interpret the observed behaviors. At this stage, the RFC serves as a reference for interpretation rather than as an executable oracle. Several outcomes are possible: both implementations may be compliant with the specification, both may be non-compliant, or the discrepancy may arise from underspecified or ambiguous parts of the protocol where multiple behaviors are permitted.

If the analysis indicates that one implementation is non-compliant while the other behaves in accordance with the specification, we validate the discrepancy

by executing the test harness on the unmodified implementation. The witness provides both the value of the symbolic field and the index of the packet in which it appears. During validation, the corresponding packet is modified to contain the witness value, and the interaction is replayed to confirm that the divergent behavior can be reproduced. If confirmed, the non-conformance is reported to the developers.

We conclude this section by instantiating the above procedure for the running example involving the `sequence_number` field. For Mbed TLS, 35 Path-Output pairs are generated, while for wolfSSL, 28 Path-Output pairs are generated. Instantiating Formula (5.1) for these Path-Output pairs yields two satisfying assignments, each forming a discrepancy witness. Each witness contains a concrete value for `sequence_number`, the index of the packet containing it, and the corresponding responses produced by the two SUTs.

One of the witnesses indicates that, upon receiving the first *ClientHello* packet with a `sequence_number` slightly larger than 2^{47} , Mbed TLS responds with a *HelloVerifyRequest*, indicating acceptance of the packet, while wolfSSL produces no response and appears to silently discard it. According to the DTLS specification [59, p. 8], record sequence numbers are 48-bit values and must remain strictly less than 2^{48} ; the sequence number produced by the witness satisfies this numeric constraint. Consulting the specification reveals that Mbed TLS handles this packet correctly, whereas wolfSSL incorrectly rejects a packet whose sequence number is large but still valid. This discrepancy exposes a bug in wolfSSL that can lead to interoperability problems.

5.1.4 Selecting Fields for Symbolic Execution

The choice of which protocol fields to make symbolic, and which fields of response packets to record, has a significant impact on both the effectiveness and the practicality of differential SE. Making too many fields symbolic, or recording overly detailed responses, can quickly lead to an explosion in the number of explored paths and identified discrepancies, many of which may not correspond to meaningful specification violations. In this section, we outline general guidelines for selecting input and output fields, following the technique adopted in our implementation.

Selecting Fields to Make Symbolic in Input Packets

In principle, our technique can be applied to any protocol field, including fields such as `sequence_number`, whose incorrect handling may have serious security or functional implications. In practice, however, performance considerations and limitations of SE necessitate a more selective approach. In particular, fields that are fed into cryptographic operations are often difficult to handle symbolically. Symbolic execution engines typically struggle with cryptographic functions, and accessing encrypted fields within monitors may

require additional infrastructure, such as packet decryption. To avoid these complications, one option is to restrict symbolic manipulation to fields that are transmitted in plaintext. In DTLS, for example, the `sequence_number` field is not encrypted and can therefore be accessed and manipulated without decrypting records.

Another option is to disable encryption altogether. This can be achieved either through configuration settings supported by the implementation under test or by applying small, targeted modifications to the SUT. In our DTLS experiments, we disabled encryption using configuration options provided by the tested implementations.

Selecting Fields to Record in Response Packets

Similarly to input fields, one could in principle record entire response packets in order to maximize the number of discrepancies that can be identified. In practice, however, recording all response fields tends to produce a large number of discrepancies that are unlikely to correspond to bugs and that significantly increase the manual effort required for analysis. For this reason, our technique selectively excludes certain classes of response fields from consideration.

The first class consists of fields that contain nonces or other randomly generated values. Recording such fields would almost always lead to trivial discrepancies, since independently generated random values are expected to differ across implementations. For example, we do not record the random field in the DTLS *ServerHello* message, as this field contains a nonce.

The second class consists of fields whose values primarily reflect differences in supported features or configuration options across implementations. Such differences are often unavoidable and do not necessarily indicate incorrect behavior. Recording these fields would therefore result in discrepancies that are unlikely to be representative of actual nonconformance issues. As an example, we do not record the `cipher_suites` field in the DTLS *ServerHello* message, since its value depends on the set of cryptographic algorithms supported by the implementation, which varies widely across SUTs.

The third class consists of fields whose values depend on fields belonging to the first or second class. The same reasons for excluding those fields apply here as well. For instance, the DTLS *Finished* message contains the `verify_data` field, which is computed as a hash over several protocol fields, including random and `cipher_suites`. We therefore exclude this field from analysis.

In our implementation, the selection and recording of response fields is handled by a dedicated output-capture component. This component is responsible for extracting the chosen fields from response packets and associating them with the corresponding path conditions. We describe this component in the next section.

5.2 Implementation

This section describes how the differential SE technique introduced in the previous sections is realized in practice. The implementation builds directly on the monitor-based infrastructure presented in Chapter 4. In particular, the same packet interception mechanism, monitor invocation model, and protocol-specific parsing infrastructure are used. The main differences lie in how monitors are used, how protocol responses are recorded, and how the results of SE are analyzed to identify discrepancies across implementations.

We first describe the construction of the test harness. We then outline the extensions to the SE setup that enable the generation of Path-Output pairs. Finally, we describe how discrepancies are identified and discuss optional adaptations to the SUT that improve SE performance.

5.2.1 Creating a Test Harness

For each SUT, SE is performed within a dedicated test harness consisting of the SUT and a facilitator, as described in Section 5.1.1.

All components execute within a single Unix process. This design choice is dictated by the SE engine used in our implementation, KLEE [11], which supports only single-threaded execution. As in the monitor-based technique presented in Chapter 4, the harness mediates all communication between the facilitator and the SUT; the concrete interception and packet-handling mechanisms are described in Section 5.2.2.

Protocol implementations are typically distributed as libraries accompanied by example programs. We construct test harnesses by adapting these example programs. In the DTLS setting, we start from the sample client and server programs provided by each library and refactor them into facilitator and SUT components. To ensure that all SUTs are exposed to identical inputs, the same facilitator implementation is used across all tested SUTs. Only the SUT component varies between harnesses. Although a separate harness is required for each library, a harness can typically be reused across multiple versions of the same implementation with little or no modification.

For DTLS, each harness executes a handshake between the facilitator and the SUT and then exchanges a small piece of application data. Execution proceeds in a loop that terminates when the handshake has completed on both sides and exactly one record containing application data has been exchanged in each direction. The loop also terminates if a fatal alert is generated or if an iteration produces no new packet.

5.2.2 Augmentations to Symbolic Execution

Our technique requires extending the SE setup so that protocol responses can be recorded and compared across implementations. These extensions reuse

large parts of the monitor-based infrastructure described in Chapter 4, while adapting it to support this technique.

Common Infrastructure

As in the monitor-based technique (Section 4.2.1), packet exchange between the facilitator and the SUT is mediated by a proxy that intercepts POSIX socket calls. When executed under KLEE, the harness invokes custom implementations of socket functions such as `sendto` and `recvfrom`, which enqueue and dequeue packet payloads from in-memory queues. Each direction of communication maintains a separate queue.

Whenever a packet is dequeued, it is first parsed into a structured representation using protocol-specific infrastructure. A monitor is then invoked on this representation. After the monitor completes its execution, the possibly modified packet is serialized back into its wire format and delivered to the receiving party. This execution model is unchanged from the monitor-based framework and is reused without modification.

Protocol-Specific Infrastructure

Beyond the common infrastructure, our technique relies on several protocol-specific components. These include a set of monitors, an output-capture function, and optional parser and serializer components.

Set of Monitors

Monitors in the differential setting are considerably simpler than those used for specification-based testing. Their role is not to evaluate protocol requirements or check assertions, but to select fields for symbolic exploration and to trigger the recording of responses.

Each monitor is implemented as a C function that operates on a structured packet representation. When invoked, the monitor may make a specific packet field symbolic and then wait for a corresponding response from the SUT.

Figure 5.3 shows the C implementation of the monitor used in the running example to explore variations of the DTLS `sequence_number` field. The monitor maintains a simple control state. In the initial state, it observes client packets and nondeterministically selects one packet on which to act. When that packet is encountered, the `sequence_number` field is made symbolic. The monitor then waits for the next packet generated by the server and invokes the output-capture function (`capture_dtls_output`) to record the response.

Similar to the monitor-based framework, nondeterministic selection of the packet index is implemented by the infrastructure rather than explicitly encoded in the monitor.

Output-Capture Function

Output recording is implemented via a dedicated output-capture function that is invoked by the monitor when a response from the SUT is observed. Con-

```

1  enum STATE { INIT, RCVD, END };
2
3  static STATE curr_state = INIT;
4
5  void sequence_number_diff_testing_server(RECORD *P) {
6      if (curr_state == INIT && P.source == CLIENT) {
7          kleener_make_symbolic(&P->sequence_number,
8                               sizeof(P->sequence_number), "sequence_number");
9          curr_state = RCVD;
10     }
11     else if (curr_state == RCVD && P.source == SERVER) {
12         capture_dtls_output(P);
13         curr_state = END;
14     }
15     else if (curr_state == RCVD && P.source == CLIENT)
16         curr_state = END;
17 }

```

Figure 5.3. The C implementation of the monitor of fig. 5.2.

ceptually, this function operates on the structured representation of a protocol message produced by the SUT, extracts a selected set of response fields, and records their values as the observable output for the current execution path. The choice of which fields to record is protocol-specific and follows the guidelines described in Section 5.1.4.

For DTLS, the output-capture function inspects the parsed DTLS record produced by the SUT and captures selected fields such as message types or alert codes, while excluding fields whose values are nondeterministic or depend on configuration. Analogous output-capture functions are implemented for other protocols by selecting and recording the corresponding protocol-specific response fields.

Parser and Serializer

As in the monitor-based technique, optional parser and serializer components are used to translate between raw packet bytes and structured protocol representations. These components simplify the implementation of monitors and output-capture functions by allowing direct access to protocol fields. They are reused unchanged from the earlier framework.

5.2.3 Finding and Analyzing Discrepancies

Executing the test harness under SE produces a set of Path-Output pairs for each tested SUT. Each pair consists of an SMT formula encoding the path condition and a symbolic representation of the selected response fields recorded by the output-capture function.

For each SE campaign and each tested SUT, KLEE produces one constraint file per explored path, containing the path condition, and a corresponding file that encodes the recorded response. Both artifacts are expressed in SMT-LIBv2 format. To identify discrepancies, we compare the Path-Output pairs generated for different SUTs using an offline analysis script.

For every pair of SUTs, the script iterates over pairs of paths that correspond to the same packet index and checks the satisfiability of the discrepancy condition represented by Formula (5.1) using the SMT solver Z3. If the condition is satisfiable, the solver produces a concrete assignment to the symbolic variables that causes the two SUTs to exhibit different responses. This assignment, together with the differing responses, yields a discrepancy witness.

Because discrepancy witnesses are analyzed manually, the script applies a simple deduplication strategy. A witness is reported only if no previously reported witness exists with the same packet index and the same pair of observed responses. This filtering significantly reduces the number of discrepancies that must be inspected while preserving representative cases. In our running example, this strategy reduces the number of reported discrepancies from ten to two.

5.2.4 Optional SUT Adaptations

As in the monitor-based technique described in Chapter 4, we apply a small set of optional adaptations to the SUT to improve performance without affecting the analysis.

In particular, we suppress time-based behavior such as retransmissions and timeouts, either through configuration options provided by the implementation or through minor code modifications when necessary. We also avoid expensive cryptographic operations during SE. For DTLS, this is achieved by configuring the SUT and facilitator to disable encryption after the handshake.

5.2.5 Evaluation

We evaluated our technique to answer four main questions. First, how does the technique compare to the monitor-based, specification-driven approach in terms of bug-finding effectiveness. Second, how does the required testing effort compare, and how can this effort be optimized. Third, can the technique uncover previously unknown bugs in recent versions of DTLS implementations. Finally, can the technique be used to detect regressions across implementation versions.

Comparison with Monitor-Based Specification Testing

The monitor-based technique presented in Chapter 4 relies on explicitly encoded specification requirements to detect violations. As a result, its effective-

ness depends directly on the completeness and precision of the specification-based oracle. Differential SE removes this dependency by comparing the externally observable behavior of multiple implementations under identical symbolic inputs.

In our evaluation, differential SE uncovered nonconformance issues that were not detected by the monitor-based approach. Several of these nonconformances arose in parts of the protocol that are underspecified or loosely defined in the RFC, making them difficult to encode accurately as specification-based monitors. While the monitor-based technique excels at detecting clear violations of explicitly stated requirements, differential SE proved more effective at revealing subtle semantic differences and corner cases that manifest only through comparison. The two techniques are therefore complementary rather than redundant.

Testing Effort and Optimization

Differential SE requires less manual effort to define protocol oracles, as it does not require extracting and encoding individual specification requirements. Instead, the main manual effort lies in selecting protocol interactions, choosing fields to make symbolic, and interpreting the discrepancies produced by the analysis.

The symbolic exploration cost is comparable to that of the monitor-based technique, since both approaches rely on similar test harnesses and SE infrastructure. However, differential analysis may initially produce a larger number of candidate discrepancies. We mitigate this by restricting each SE campaign to a single symbolic field instance and by applying deduplication during discrepancy analysis. These optimizations significantly reduce the number of discrepancies that require manual inspection and keep the overall testing effort manageable.

Detection of Previously Unknown Bugs

We applied differential SE to several widely used DTLS server implementations. The evaluation uncovered nine distinct bugs, five of which were previously unknown, including deviations from the protocol specification that can lead to interoperability issues. Importantly, these bugs were found in up-to-date versions of the implementations, demonstrating that the technique remains effective even as implementations evolve.

Several of the discovered issues arise from subtle semantic differences in how implementations interpret and enforce protocol rules. Such issues are difficult to capture using specification-based testing alone, particularly when the relevant aspects of the specification are underspecified or permit multiple interpretations. By systematically comparing behaviors across implementations, differential SE exposes these discrepancies and provides concrete witnesses for further analysis.

Regression Detection

Differential SE can also be used to detect regressions by comparing different versions of the same implementation. By treating two versions of a library as separate SUTs and executing identical SE campaigns, the technique can reveal behavioral changes introduced by code modifications. This allows developers to identify unintended regressions or semantic changes, even when such changes do not violate explicitly encoded specification requirements.

Overall, the evaluation demonstrates that differential SE is an effective and practical complement to specification-based SE. By removing reliance on specification-based oracles, the technique enables the discovery of protocol nonconformances and semantic discrepancies that are difficult to detect using explicit requirement checks alone. A detailed description of the experimental setup, tested implementations and versions, and the full set of discovered issues is provided in Paper III.

Building on the techniques presented in earlier chapters, this chapter explored differential SE as a means of testing network protocol implementations without relying on specification-based oracles. By comparing the externally observable behavior of multiple implementations under symbolic inputs, the technique complements the requirement-driven and monitor-based approaches presented earlier in this thesis.

Symbolic execution enables systematic exploration of protocol behavior and precise reasoning about corner cases. Fuzzing offers a complementary perspective by enabling lightweight, scalable exploration of large input spaces. The final chapter examines these techniques side by side by applying them to the same set of protocol requirements and implementations. Using the Constrained Application Protocol (CoAP) as a case study, it analyzes their effectiveness in identifying requirement violations in resource-constrained environments.

6. Two Paths to Conformance Testing: Fuzzing and Symbolic Execution

Network protocol conformance can be assessed through different testing techniques, each offering distinct strengths and limitations. The previous chapters have focused on symbolic execution (SE) as a principled technique for systematically exploring protocol behavior and for reasoning about requirement violations that arise only under narrowly constrained conditions. By progressively extending SE with requirement-driven oracles, external monitors, and differential analysis, we demonstrated how it can be applied effectively to complex, stateful protocols such as DTLS and QUIC.

However, SE is not the only technique used in practice to test protocol implementations. Coverage-guided fuzzing has become a widely adopted approach for testing networked systems, particularly in settings where scalability and low analysis overhead are important. Fuzzers excel at exercising large input spaces quickly and can expose robustness issues as well as logical flaws that manifest through externally observable behavior. At the same time, fuzzing is not typically employed to check protocol requirements or to systematically target specific semantic conditions.

This chapter applies the requirement-centric perspective developed in earlier chapters to both techniques and compares their effectiveness when applied to the same requirements and implementations. In particular, it examines how fuzzing and SE differ in their ability to expose requirement violations in the context of an IoT protocol.

6.1 Technique

6.1.1 Test Harness Creation

Testing protocol conformance requires a controlled execution environment in which interactions with a protocol entity can be reproduced and manipulated systematically. As in the previous chapters, this is achieved through a dedicated test harness that mediates communication between the system under test (SUT) and its peer and that provides full control over the messages exchanged during a protocol interaction.

The construction of the harness begins by capturing a representative interaction with the protocol entity. For CoAP, this interaction consists of a sequence of request and response messages exchanged between a client and a

server. Packet capture is performed by configuring sample programs provided with the SUT to execute a valid interaction and recording the packets sent to the protocol entity using standard network analysis tools such as TCPdump or Wireshark. The resulting packet sequence serves as a concrete reference interaction that can be replayed during testing.

During testing, the harness advances the interaction step by step by replaying the pre-captured packets. At each step, the harness invokes the appropriate API of the SUT to deliver an incoming message and to obtain the corresponding response. Responses produced by the SUT are recorded and used by the harness to determine how the interaction proceeds. This execution model ensures that the protocol logic exercised during testing follows the same high-level interaction pattern as the captured execution, while allowing individual message fields to be varied or constrained as required by the testing technique.

The test harness should be designed to support all protocol requirements considered for testing. Depending on the nature of the requirements, this may involve executing a single interaction that exercises multiple protocol features or running several distinct interactions that target different parts of the protocol. In either case, the harness provides a uniform mechanism for delivering inputs, observing outputs, and controlling progress through the protocol interaction.

A key design goal of the harness is to enable both fuzzing and SE with minimal adaptation. The same harness structure is used in both settings, with differences limited to how input messages are generated and how responses are analyzed. For fuzzing, the harness supplies mutated variants of the captured packets, while for SE it designates selected fields as symbolic and relies on the symbolic executor to explore feasible execution paths. By sharing a common harness, both techniques are applied to identical interaction patterns and protocol requirements, enabling a meaningful comparison of their effectiveness in exposing requirement violations.

6.1.2 Encoding Protocol Requirements as Assertions

Network protocol specifications define requirements not only over individual messages but also over sequences of messages exchanged between protocol entities. For CoAP, such requirements are specified across several RFC documents, most notably RFC 7252 [66] and RFC 7959 [8]. Once a set of requirements has been extracted from these specifications, they can be translated into executable assertions that are evaluated during a protocol interaction.

In this work, protocol requirements are encoded as assertions over the packets exchanged during an interaction. These assertions are inserted at points in the SUT where response messages are generated, allowing the observed behavior of the implementation to be checked against the specification at run-time. In the CoAP implementations considered, assertions were inserted into

the functions responsible for transmitting response messages. These functions already operate on structured representations of outgoing packets, which allows assertions to directly inspect response fields. Fields of the corresponding incoming packet are accessed through a global pointer that is maintained by the test harness and updated at each interaction step.

We illustrate this approach by presenting the encoding of two representative CoAP requirements.

Message ID Echo.

The CoAP specification mandates that the Message ID of certain request messages must be echoed in the corresponding response. RFC 7252 [66, p. 24] states:

The Message ID is a 16-bit unsigned integer that is generated by the sender of a Confirmable or Non-confirmable message and included in the CoAP header. The Message ID MUST be echoed in the Acknowledgement or Reset message by the recipient.

Let p_{in} denote the parsed incoming packet and p_{out} the response packet generated by the SUT. The requirement can be encoded by the following assertion, which is evaluated when the response packet is constructed:

$$\text{assert } (((p_{in}.type = \text{CON} \vee p_{in}.type = \text{NON}) \wedge (p_{out}.type = \text{ACK} \vee p_{out}.type = \text{RST})) \implies p_{in}.message_id = p_{out}.message_id)$$

This assertion ensures that whenever the SUT generates an acknowledgement or reset in response to a Confirmable or Non-confirmable request, the Message ID is correctly echoed. During testing, the assertion is evaluated at each interaction step, and any violation immediately indicates nonconformance with the specification.

Further Request Block Size.

Some CoAP requirements relate the handling of a request to information conveyed in earlier responses. RFC 7959 [8, p. 14] specifies such a relationship for block-wise transfers:

In response to a request with a payload (e.g., a PUT or POST transfer), the block size given in the Block1 Option indicates the block size preference of the server for this resource. ... the client SHOULD heed the preference indicated and, for all further blocks, use the block size preferred by the server or a smaller one.

To encode this requirement, the assertion logic explicitly maintains state across packets. A boolean variable `first_response` indicates whether the current

response is the first generated by the server, and a variable `preferred_size` stores the block size advertised by the server. The requirement is checked using the following logic:

```

if (first_response ∧ pin.code ∈ REQ_CODES)
    preferred_size := BlockSize(pout)
    first_response := false
elseif (¬first_response ∧ pin.code ∈ REQ_CODES)
    assert((BlockSize(pin) > preferred_size) ⇒
        pout.code ∈ ERR_CODES)

```

The assertion is evaluated only for subsequent relevant requests, after the server’s preferred block size has been established by the first relevant response. In the formulation presented in Paper IV, these scoping conditions are implicit, whereas they are made explicit here for clarity.

Here, `REQ_CODES` represents the set of CoAP request codes. The function `BlockSize` extracts the block size specified by the `Block1` option of a packet, and `ERR_CODES` denotes the set of CoAP error response codes. The assertion checks that if a client continues to send request blocks larger than the server’s advertised preference, the server responds with an error.

While the RFC does not prescribe a specific error response for this case, requiring an error response provides a reasonable approximation of the intended behavior and allows violations to be detected systematically.

Once the SUT has been extended with such assertions, they serve as executable representations of protocol requirements. Both fuzzing and SE can then be applied to explore whether the implementation can be driven into executions that violate these assertions, thereby revealing instances of protocol nonconformance.

6.1.3 Testing Requirements Using Fuzzing

While coverage-guided fuzzing is traditionally used to discover crashes and robustness issues, it can be adapted to support requirement-driven testing when combined with explicit assertions. In this work, we use fuzzing not as a robustness testing technique, but as a mechanism for exploring executions that may violate protocol requirements encoded as runtime assertions. Standard fuzzing infrastructure can be leveraged for this purpose without modifying the fuzzer itself, by generating inputs that may trigger violations of the encoded assertions. In our evaluation, we use AFL++ [27], a state-of-the-art fuzzer that incorporates techniques from recent research and provides efficient feedback-driven input generation.

Fuzzing is performed by executing the test harness described earlier. As initial seeds, we use valid, pre-captured packets obtained from protocol interactions executed using the sample programs provided with each implementation. These packets serve as well-formed starting points from which the fuzzer

generates mutated inputs. During a fuzzing campaign, the fuzzer repeatedly executes the harness and supplies mutated variations of the incoming packets to the SUT. All encoded assertions are evaluated during each execution, and any violation is immediately detected.

Some protocol requirements, particularly those related to block-wise transfer in CoAP, depend on the processing of multiple packets and therefore require that protocol state be built across several interaction steps before an assertion can be checked. Standard fuzzers are not designed to reason about such stateful interactions. To accommodate these requirements, we represent the packets of a complete interaction as a single contiguous input buffer, with each packet placed at a fixed offset. During execution, the harness delivers packets to the SUT by reading from the appropriate offsets within this buffer. Since the fuzzer is free to arbitrarily mutate the input buffer, packet boundaries are not guaranteed to remain intact. As a result, some mutated inputs may cause packet boundaries to shift or packets to become malformed. In such cases, the harness may fail to parse the input correctly or the SUT may immediately reject the input, and these executions do not contribute to requirement checking. In our experiments, this behavior did not prevent the fuzzer from generating valid test cases that successfully exercised multi-packet interactions and triggered assertion violations. When packet parsing succeeds, this design allows the fuzzer to mutate a single input buffer while still exercising a multi-packet protocol interaction. As a result, state-dependent requirements can be tested using standard fuzzing infrastructure without modifying the fuzzer itself.

Taking into account the inherent randomness of fuzzing, campaigns are repeated multiple times to increase confidence in their ability to expose violations. When the fuzzer triggers an assertion violation, it stores the corresponding mutated input together with metadata such as the time of discovery and the number of mutations applied. These concrete test cases can then be analyzed to determine which packet fields and values caused the requirement violation and to reproduce the behavior on the unmodified implementation.

Overall, fuzzing provides a lightweight and scalable means of exploring large portions of the input space and of exercising multiple requirements simultaneously. However, its effectiveness depends on the fuzzer's ability to reach inputs that satisfy the specific conditions under which assertions fail. In the next subsection, we contrast this behavior with SE, which reasons explicitly about such conditions and can systematically target individual requirements.

6.1.4 Testing Requirements Using Symbolic Execution

The approach for testing requirements using SE employed in this work closely follows the requirement-driven SE technique introduced in Chapter 3, using the same test harness structure and execution model. The main differences

in this work concern the protocol under test and the specific requirements considered, rather than the underlying SE methodology. To test protocol requirements using SE, we employ KLEE [11], which we use as the SE engine throughout this thesis. As in the fuzzing-based approach, SE is performed by executing the test harness described earlier. However, unlike fuzzing, SE targets individual requirements explicitly and is therefore performed in separate runs, with each run checking exactly one requirement.

In each SE run, KLEE executes the test harness using a pre-captured sequence of valid packets that define the interaction under test. Before a packet is processed by the protocol party, the fields relevant to the requirement being checked are designated as symbolic, while all other fields remain concrete. This selective symbolic treatment restricts exploration to the part of the input space that is relevant for the requirement and avoids unnecessary path exploration.

As in the earlier chapters, packets are parsed into structured representations to allow symbolic manipulation of individual protocol fields, and are serialized back into byte buffers before being processed by the protocol implementation.

As an example, to test the Message ID Echo requirement, the `message_id` and `type` fields of the incoming packet are made symbolic. KLEE then explores all feasible execution paths that arise from different assignments to these fields. If there exists an execution path along which the corresponding assertion is violated, KLEE reports the violation and produces a concrete assignment for the symbolic fields that triggers it. This assignment constitutes a concrete test case that demonstrates nonconformance with the requirement.

In this setting, a test case corresponds to the specific values assigned to the symbolic protocol fields, rather than to an entire packet sequence. These values can be substituted back into the original packet structure and replayed against the unmodified implementation to reproduce the violation. Symbolic execution therefore provides a systematic means of determining whether a requirement can be violated at all and, if so, of generating precise witnesses that expose the violation.

6.2 Evaluation

The purpose of the evaluation is to assess how fuzzing and SE behave in practice when applied to the same protocol requirements within a shared testing framework. The evaluation focused on a set of representative protocol requirements extracted from the CoAP specification and its extensions.

Overall, our findings demonstrate that both fuzzing and SE are effective at exposing protocol nonconformances when requirements are encoded explicitly as assertions. Across the evaluated requirements, both fuzzing and SE identified the same set of nine distinct nonconformances in the tested implementations. All identified issues were reported to the respective developers,

and most have been fixed at the time of writing. These results indicate that assertion-based testing is capable of uncovering concrete and actionable specification violations in real-world CoAP implementations.

In terms of effectiveness at detecting violations, we did not observe a substantial difference between the two techniques. For the evaluated requirements, fuzzing and SE were able to identify violations for the same subsets of requirements in each implementation. Moreover, violations were typically discovered quickly. Fuzzing consistently generated inputs that triggered assertion failures within seconds for most requirements, while SE also produced violating inputs within comparable time frames in the majority of cases. This suggests that, when requirement violations are reachable through relatively shallow protocol interactions, both techniques are well suited to exposing them.

The main distinction between fuzzing and SE becomes apparent when considering requirements that an implementation respects. Fuzzing is inherently probabilistic and cannot provide evidence that a requirement is upheld beyond the absence of observed violations within a given time budget. Symbolic execution, by contrast, explores program paths systematically and can, in principle, establish that no execution violating a requirement exists within the explored search space. In practice, this ability is limited by path explosion and solver performance. For some requirements, SE was able to complete exploration and thus provide stronger assurance of correctness. For others, exploration did not complete within the imposed time limits.

Taken together, these observations suggest a complementary relationship between the two techniques. Fuzzing offers a lightweight and scalable way to uncover specification violations quickly, making it well suited for early testing. Symbolic execution, while more computationally expensive, provides deeper insight into the conditions under which violations occur and can offer stronger guarantees when exploration completes. Our experience indicates that combining both techniques within a single assertion-based framework allows developers to benefit from the strengths of each while mitigating their respective limitations. A detailed description of the experimental setup, the evaluated implementations, and the complete set of discovered issues is provided in Paper IV.

7. Related Work

This chapter reviews related work on techniques for testing protocol implementations, focusing on methods that test network protocol implementations.

7.1 Specification-Based Protocol Testing

Ensuring that network protocol implementations conform to their specifications has long been an important goal, motivated by the fact that even subtle deviations can lead to interoperability issues or security vulnerabilities. Traditional approaches for testing conformance are typically specification-driven and rely on abstract models or explicitly encoded properties derived from specification documents such as RFCs.

Model-based Testing (MBT) is a well-established approach in this space, where an abstract behavioral model of the protocol is either manually constructed or inferred through learning techniques [10, 72, 70]. This model is then used to systematically generate test inputs, which are applied to an implementation, usually in a black-box setting. A wide range of modeling formalisms and tooling support has been proposed, including automata-based specifications and domain-specific languages [75, 45].

Recent work has demonstrated the applicability of MBT to realistic networked and distributed systems. Li et al. [38] derive executable tests for networked applications from formal models, while MBT has also been applied to IoT protocols such as BLE [23] and MQTT [69], uncovering conformance issues in implementations of these protocols.

While MBT can provide strong guarantees when accurate models are available, constructing and maintaining such models for complex evolving protocols remains labor intensive.

Property-based testing (PBT) offers a related but often lighter-weight alternative [4, 42, 40, 41]. Instead of a full behavioral model, PBT relies on properties that inputs and outputs should satisfy, combined with automated input generation. The high-level abstraction and built-in generators provided by PBT frameworks often make them easier to apply than MBT. However, as with MBT, the effectiveness of PBT depends on the completeness and precision of the manually specified properties.

Both MBT and PBT are commonly applied in black-box testing scenarios. As a result, they are limited in their ability to guide exploration toward specific internal program states or code paths that may expose subtle requirement

violations. In contrast, white-box techniques can leverage internal program structure to search more directly for inputs that trigger specification violations, especially in the presence of complex control flow and stateful behavior.

7.2 Fuzzing for Network Protocol Implementations

Fuzz testing has established itself as one of the most effective techniques for uncovering software defects by repeatedly executing a program on mutated inputs and observing unintended behavior, particularly crashes and memory errors. Modern coverage-guided greybox fuzzers, such as AFL [80] and its successor AFL++ [27], use lightweight instrumentation to guide input generation toward previously unexplored branches and have achieved considerable success across a wide range of software systems.

Fuzzing has also been applied to network and IoT protocol implementations. For example, fuzzers have been used to test IPv6 network stacks in constrained environments [54], as well as protocol implementations such as CoAP [39, 46, 81].

A key challenge for fuzzing protocol implementations is statefulness. Many protocols require specific sequences of messages to reach deep states in the implementation, and naive fuzzing often fails to explore such interactions. To address this challenge, several extensions to greybox fuzzing have been proposed, including AFLNET [52], StateAFL [49], SGFuzz [6], SnapFuzz [3] and LibAFLstar [19]. These approaches incorporate protocol state awareness, for instance by tracking message sequences or using lightweight protocol models, in order to improve coverage in stateful systems. Empirical evaluations show that such techniques significantly outperform traditional greybox fuzzers on network protocol implementations, enabling deeper exploration of protocol logic and leading to the discovery of previously unknown bugs across a range of stateful protocols. Nevertheless, their effectiveness is primarily measured in terms of coverage and crash discovery, and they do not explicitly target violations of semantic protocol requirements derived from specifications.

Despite these advances, fuzzing-based techniques generally offer no guarantees about the absence of protocol-level violations beyond the detection of runtime errors such as crashes, assertion failures, or hangs. Even when stateful fuzzing reaches deep protocol states, violations that do not manifest as immediate runtime failures may remain undetected.

Protocol state fuzzing based on model learning represents a related line of work. By inferring abstract state machine models, often in the form of Mealy machines, such techniques can uncover logical flaws in control flow and message ordering [55, 73]. This approach has been successfully applied to a variety of protocols, including DTLS [31, 16], TCP [28, 30], SSH [33], OpenVPN [18], IPSec [35], QUIC [26, 56], and EDHOC [62]. These tech-

niques are effective at exposing inconsistencies in protocol state machines, but they typically operate in a black-box manner and do not directly reason about individual protocol requirements or packet-level field constraints.

7.3 Symbolic Execution for Protocol Implementations

Symbolic execution (SE) was introduced in the 1970s as a program analysis technique that explores program behavior by treating selected inputs as symbolic rather than concrete values [37]. Advances in constraint solving and tool support over the past two decades have turned SE into a practical and powerful testing technique, capable of uncovering deep bugs and security vulnerabilities that are difficult to expose using random or coverage-guided testing alone [12, 11, 17].

Applying SE to network protocol implementations poses challenges that are not present in traditional single-input programs. Protocol implementations are often stateful, process sequences of messages rather than isolated inputs, and interact with other protocol parties and the environment. As a result, SE must account for both the values of packet fields and the order in which packets are exchanged.

Several approaches apply SE to protocol implementations by designating specific fields in incoming packets as symbolic, thereby enabling exploration of code paths reachable under different packet contents [64, 65, 50, 57, 67, 71, 78]. In these approaches, SE is typically used to uncover generic runtime errors, such as crashes or memory safety violations, or to detect interoperability issues between protocol parties. For example, Pedrosa et al. [50] use SE to characterize messages that can be sent by one protocol party but rejected as non-compliant by another, while Rath et al. [57] apply SE to explore interoperability scenarios in QUIC implementations.

A recurring difficulty in these settings is protocol statefulness. To reach deep states, an implementation must process a specific sequence of packets, and the correctness of processing a given packet may depend on information accumulated from earlier messages. To address this, some approaches incorporate explicit models of protocol behavior. Wen et al. [77] employ model learning to infer a protocol state machine, which is then provided as additional input to SE, while Tempel et al. [71] show that taking protocol state into account can significantly increase test coverage. These techniques improve path exploration but still focus primarily on coverage and runtime error detection rather than checking compliance with protocol specifications.

Other lines of work symbolically execute multiple protocol parties jointly in order to explore interactions under different synchronization and failure scenarios. KleeNet [64, 65] extends SE to distributed settings by modeling packet loss and node failures using symbolic variables, allowing the exploration of diverse execution scenarios. Similar ideas appear in SymNet [63],

which applies SE to integration testing of protocol implementations. These approaches are effective at identifying interaction and interoperability issues that arise during joint execution of protocol parties, but have limited ability to expose flaws caused by adversarial or malformed inputs, since inputs are typically generated by protocol-compliant peers.

More recent work combines SE and specification-based testing by checking protocol requirements derived from protocol specification documents. SymboxNet [67] first applies SE to generate concrete test inputs that explore a wide range of code paths, and then replays these inputs on an unmodified implementation while monitoring the resulting input-output behavior against specification-derived rules. While this separation allows requirements to be checked without modifying the system under test, SE in this setting may still miss requirement-violating inputs when violating and non-violating inputs exercise the same code path.

Several approaches address this limitation by guiding SE explicitly toward requirement-relevant behaviors. One line of work, exemplified by Paper I, embeds assertions and assumptions directly into the SUT's code to constrain SE toward inputs that may violate protocol requirements, enabling systematic exploration of requirement-specific corner cases. Subsequent work, represented by Paper II, externalizes this requirement-checking logic into monitors that observe packet exchanges and interact with the SE engine, reducing the need for intrusive modifications to the implementation. These approaches demonstrate that SE can be effectively adapted from a coverage-oriented testing technique into a tool for checking semantic protocol requirements.

Overall, SE offers strong capabilities for systematically exploring protocol implementations under a wide range of inputs. However, its effectiveness depends on how statefulness, nondeterminism, and correctness criteria are incorporated into the analysis. Unguided SE tends to emphasize code coverage and runtime errors, while exposing semantic protocol violations often requires more targeted approaches, including explicit consideration of protocol requirements or alternative notions of correctness.

An alternative to specification-based correctness checking is differential testing, which detects discrepancies by comparing the behavior of multiple implementations under the same inputs. Differential testing has been widely used to uncover semantic bugs in protocol implementations without requiring a complete specification oracle. Differential symbolic execution, introduced by Person et al. [51], extends this idea by enabling systematic exploration of inputs that lead to behavioral differences across executions.

Several works have applied differential SE by comparing the behavior of multiple implementations rather than checking conformance against a single specification. Chau et al. [13, 14] use SE to derive path constraints for certificate validation logic and compare them across implementations to identify discrepancies. RFCcert [15] combines specification-derived rules with SE to generate test cases that expose differences between X.509 validators.

Other approaches apply differential analysis to restricted protocol components or employ offline comparison of execution models, such as ParDiff [82] and SymTCP [76]. These techniques demonstrate the effectiveness of differential reasoning, but typically operate on isolated components or abstracted models rather than complete, stateful protocol interactions.

Together, these lines of work illustrate how advances in SE have progressively expanded the scope of protocol testing, from coverage-oriented exploration to requirement-aware and differential analysis of stateful protocol implementations.

8. Conclusion

Testing network protocol implementations for conformance with their specifications remains challenging due to protocol statefulness, complex dependencies across packet sequences, and large input spaces that are difficult to explore exhaustively. Even minor deviations from specifications can lead to interoperability issues or security vulnerabilities, yet such deviations often escape detection by conventional testing techniques that rely on concrete test cases or incomplete oracles.

This thesis investigated how symbolic execution can be adapted to systematically and practically address the challenges of testing stateful protocol implementations. It introduced a requirement-driven approach that uses symbolic execution to explore behaviors relevant to specification requirements, enabling the detection of violations that depend on specific corner case values of protocol fields. Building on this foundation, the thesis presented a monitor-based testing technique that externalizes requirement checking from the system under test, allowing protocol requirements to be expressed independently of specific implementations and reused across implementations and versions. The thesis also explored differential symbolic execution as a complementary approach that removes the need for an explicitly specified oracle by comparing observable behaviors across implementations, thereby enabling the detection of discrepancies that may indicate nonconformance issues even when explicit requirements have not been encoded. As a comparative baseline, the thesis also empirically studied the use of coverage-guided fuzzing for testing protocol requirements and contrasted its effectiveness with symbolic execution in the context of CoAP implementations, highlighting the complementary strengths and limitations of the two techniques.

The proposed techniques were evaluated on multiple real-world implementations of the DTLS, CoAP, and QUIC protocols. Across these studies, the techniques uncovered 72 previously unknown specification violations. These findings were validated in practice, with the majority of the reported issues fixed by developers. The discovery of previously unknown bugs in widely deployed protocol implementations indicates that the proposed techniques are applicable beyond controlled academic settings and can be effectively used to analyze production-quality software. Overall, the results demonstrate that symbolic execution, when guided by protocol requirements and combined with monitor-based and differential analysis, can systematically expose semantic deviations from protocol specifications that are difficult to detect using conventional testing techniques.

9. Future Work

The work presented in this thesis demonstrates that requirement-driven and differential symbolic execution can effectively expose subtle nonconformance issues in stateful protocol implementations. While the proposed techniques proved effective, several promising directions remain for extending their applicability, degree of automation, and scalability. In particular, recent advances in artificial intelligence and machine learning offer opportunities to further reduce manual effort and to improve the practicality of protocol testing.

Automated Extraction of Protocol Requirements Using NLP

Protocol requirements are currently identified by manually inspecting specification documents such as RFCs. Although the specification text can sometimes be ambiguous and even contradictory, advances in natural language processing open the possibility of automating part of this process. Future work could explore the use of NLP techniques to identify normative statements, extract candidate requirements, and map them to structured representations suitable for monitor construction, building on existing work on NLP-assisted requirements extraction from standards and specification documents [34, 79].

Automating the process of requirement extraction would lower the barrier to applying requirement-driven testing to new protocols and help ensure more systematic coverage of protocol specifications.

A High-Level Domain-Specific Language for Monitor Specification

In the current framework, monitors are implemented directly as executable code that encodes state machines, symbolic field declarations, and assertion logic. While this approach offers precise control over monitor behavior, it requires writing low-level code and familiarity with implementation details. A promising direction for future work is the design of a high-level domain-specific language (DSL) for formulating monitors. Related work has already explored more structured ways of expressing properties over sequences of packets, for example through pattern-based specification languages used to guide symbolic execution and trace checking, as in SymbexNet [67].

Such a DSL would provide a declarative and protocol-aware abstraction for specifying monitor behavior, including state transitions, packet predicates, symbolic field selection, assumptions, and assertions. Monitors expressed in the DSL could then be compiled automatically into executable monitors compatible with the existing symbolic execution infrastructure. This would preserve the benefits of external, reusable monitors while significantly reducing manual effort and implementation complexity.

Applicability to Additional Protocol Classes

The evaluation in this thesis already spans multiple protocol classes, including transport-layer protocols as well as application-layer protocols such as CoAP. An important direction for future work is to further extend this evaluation across a broader and more diverse set of protocol classes, including industrial automation protocols, routing and discovery protocols, and emerging application-layer protocols.

While CoAP demonstrates that the approach is applicable beyond transport-layer protocols, additional protocol classes may exhibit different forms of statefulness, looser or more ambiguous specifications, and higher variability across implementations. Evaluating the proposed techniques in these settings would help uncover their strengths and limitations more precisely, and may motivate strategies tailored to different protocol families.

Deeper Comparative Analysis of Fuzzing and Symbolic Execution

This thesis presents an empirical comparison of fuzzing and symbolic execution for testing protocol requirements, demonstrating that both techniques can effectively expose nonconformances, but with different strengths and limitations. A natural direction for future work is to broaden this comparison by considering a wider range of protocols and a more diverse set of requirement types.

Future work could investigate whether certain classes of requirement violations are systematically more difficult for fuzzing to discover. In particular, violations that depend on specific corner case values, long-range dependencies across packet sequences, or precise relationships between multiple protocol fields may exercise the same code paths as conforming executions and therefore evade detection by coverage-guided fuzzers. A broader evaluation across additional protocols and diverse requirement types would help clarify the conditions under which fuzzing or symbolic execution is more effective.

Continuous Automated Semantic Regression Testing

The thesis demonstrated that differential symbolic execution can be used to detect behavioral regressions by comparing different versions of the same implementation. A natural extension is to integrate this capability into continuous integration workflows.

Future work could investigate lightweight, incremental symbolic execution strategies that allow protocol implementations to be tested automatically as part of a development pipeline. Such an integration would enable early detection of semantic regressions and unintended behavioral changes, strengthening protocol robustness throughout the development life cycle.

Reducing Manual Effort in Discrepancy Classification

While the differential testing approach effectively detects discrepancies, determining whether a discrepancy represents a true nonconformance, a benign

difference, or an underspecified behavior still requires manual inspection of the specification. Reducing this manual effort is an important challenge for scaling the approach.

AI-assisted discrepancy classification could help address this issue by learning from previously analyzed discrepancies. Models could cluster similar discrepancies, prioritize those likely to correspond to genuine violations, or flag cases that are consistent with known ambiguities in the specification.

10. Contributions

10.1 Paper I

Paper I presents a requirement-driven approach for using symbolic execution to test stateful network protocol implementations against their specifications. The paper shows how symbolic execution can be tailored to explore execution paths that are relevant for violating protocol requirements formulated over sequences of packets, rather than single inputs. By selectively making protocol fields symbolic and constraining exploration to requirement-relevant execution paths, the approach enables the detection of violations that arise only under specific protocol states and corner case values.

The paper evaluates the proposed methodology on widely used DTLS implementations. The results demonstrate its effectiveness in practice, leading to the discovery of 36 previously unknown bugs in mature and extensively tested implementations. This confirms that requirement-driven symbolic execution can uncover subtle specification violations that are difficult to detect using existing testing techniques.

In addition, the paper provides publicly available replication material for all experiments, supporting reproducibility and facilitating further research on symbolic execution-based testing of network protocols.

10.2 Paper II

Paper II introduces a monitor-based symbolic execution technique for testing stateful network protocol implementations against their specifications. The key contribution is the adaptation of the concept of monitors for symbolic execution, in which monitors external to the system under test encode protocol requirements, maintain interaction state, and guide symbolic execution by introducing symbolic protocol fields, assumptions, and assertions. By keeping monitors external to the system under test, the technique avoids intrusive instrumentation and allows requirements to be reused across different implementations and versions of the same protocol.

The paper evaluates the proposed technique on multiple real-world protocol implementations and compares it against a state-of-the-art requirement-driven symbolic execution approach. The comparison shows that the monitor-based technique exposes requirement violations considerably faster and with significantly less implementation effort. In addition, the evaluation demonstrates

that the technique is effective in practice, uncovering 22 previously unknown bugs in DTLS and QUIC implementations.

Finally, the paper provides a publicly available testing infrastructure, including reusable test harnesses and sets of monitors for DTLS and QUIC. This infrastructure supports replication of the experimental results and facilitates extending the approach to additional protocols and requirements.

10.3 Paper III

Paper III introduces a differential symbolic execution technique for testing stateful network protocol implementations without relying on an explicitly specified oracle. The key idea is to compare observable behaviors across protocol implementations under the same symbolic inputs, enabling the detection of nonconformance issues that are difficult to capture using explicit specification-based checks.

The paper evaluates the proposed technique on several widely used DTLS server implementations. The evaluation reveals multiple discrepancies in the handling of protocol edge cases, leading to the discovery of nine distinct bugs, five of which were previously unknown. These results demonstrate that differential symbolic execution can effectively expose subtle behavioral differences that may indicate specification violations.

In addition, the paper compares the proposed approach with the technique presented in Paper II. The results show that differential symbolic execution is able to detect all bugs found by the specification-based approach, while also uncovering additional issues that are missed when testing relies on an explicit oracle.

Finally, the paper provides a publicly available infrastructure to support replication of the experiments and to facilitate extending the approach to other network protocols.

10.4 Paper IV

Paper IV investigates the application of fuzzing and symbolic execution to testing CoAP protocol implementations against their specifications. The paper provides a systematic comparison of fuzzing and symbolic execution with respect to their effectiveness in exposing requirement violations, as well as the time required to do so. The results highlight complementary strengths and limitations of the two techniques when applied to protocol conformance testing.

As a result of this evaluation, the paper improves the quality of two CoAP implementations by exposing nine previously unknown bugs. These findings demonstrate that both fuzzing and symbolic execution can be effective tech-

niques for improving the reliability and correctness of IoT protocol implementations when appropriately adapted.

Finally, the paper provides practical guidance on how fuzzing and symbolic execution can be applied to IoT protocol testing. These insights help inform the selection and configuration of testing techniques for protocols with limited resources, stateful interactions, and complex specification requirements.

10.5 Personal Contributions

All papers included in this thesis are joint efforts with co-authors. My contributions were consistent across Papers I–IV, and are summarized below.

I am the primary author of all four papers. The overall research directions and methodologies were developed jointly with the co-authors. I designed and implemented the proposed approaches. Experimental evaluations were conducted collaboratively with the co-authors. I analyzed the results, prepared and submitted the corresponding bug reports, and communicated the identified bugs to the developers. All papers were written collaboratively with the co-authors.

11. Sammanfattning På Svenska

Internet spelar en central roll i dagens samhälle, och möjliggör att vi kan kommunicera och åtnjuta viktiga tjänster som t.ex. surfning, bankärenden och sociala medier. Internet bygger på ett antal nätverksprotokoll, som är implementerade i enheter anslutna till internet. Sådana implementeringar kan innehålla buggar, bland vilka en del, så kallade sårbarheter, kan utnyttjas för att orsaka skada. Ett exempel är den nu ökända Heartbleed-sårbarheten i en vida spridd implementering av TLS-protokollet, som ledde till att flera miljoner patienters data blottades. För att förebygga och åtgärda sådana sårbarheter har tekniker utvecklats för att säkra implementeringar av nätverksprotokoll, varav den dominerande är testning.

Att testa implementationer av nätverksprotokoll mot sina specifikationer är en utmanande uppgift på grund av protokollens tillståndsberoende beteende, komplexa beroenden mellan paketsekvenser och stora indatarymder. Även små avvikelser från en specifikation kan leda till interoperabilitetsproblem eller allvarliga säkerhetssårbarheter, men sådana fel upptäcks ofta inte med traditionella testmetoder som bygger på konkreta testfall eller ofullständiga testorakel. Symbolisk exekvering är en testningsteknik som adresserar problemet med stora indatarymder. Den analyserar program i vilka (några av) variabler är *symboliska*, utforskar de kodvägar som är möjliga för specifika värden av de symboliska variablerna, och genererar konkreta för vilka exekveringen följer specifika kodvägar. Denna avhandling undersöker hur symbolisk exekvering kan anpassas för att systematiskt och praktiskt testa implementationer av tillståndsberoende nätverksprotokoll.

Avhandlingens första bidrag är en kravdriven metod baserad på symbolisk exekvering, i vilken krav hämtas från protokollets specifikation och uttrycks som egenskaper hos paketsekvenser, samt realiserar som assertions inlagda i den testade implementeringens källkod. Testningen förbereds genom att spela in och spara en sekvens indatapaket i en interaktion mellan den testade implementeringen och dess motpart. Denna sekvens av indatapaket används sedan som indata under den symboliska exekveringen. Den symboliska exekveringen styrs mot exekveringsvägar som är relevanta för att bryta mot dessa krav. Genom att selektivt göra delar av indatapaket symboliska och begränsa exekveringen till kravrelevanta scenarier möjliggörs upptäckt av subtila fel som endast uppstår för specifika variabelvärden eller hörnfall.

Det andra bidraget är en monitorbaserad testteknik i vilken implementering och kontroll av krav separeras från det system som testas. Detta görs med hjälp av monitorer: dessa är komponenter i testomgivningen som är separerade från

den testade implementeringen. Protokollkrav specificeras i monitorer; dessa observerar paketutbyten, upprätthåller tillståndsinformation, styr den symboliska exekveringen, och kontrollerar krav vid tillämpliga punkter i exekveringen, utan att kräva genomgripande modifieringar av implementeringens källkod. Genom att protokollkraven inte sätts in i den testade implementeringens källkrav (som i det första bidraget) utan i återanvändbara monitorer, så förbättras modularitet, möjliggörs återanvändning av krav mellan olika implementationer och versioner, samt förenklas utvidgning till nya protokollkrav.

Det tredje bidraget utforskar differentiell symbolisk exekvering som en testmetod som inte använder explicit formulerade krav. I stället för att kontrollera efterlevnad mot en formellt specificerad kravuppsättning jämförs det observerbara beteendet mellan olika implementationer eller versioner av samma protokoll under identiska symboliska indatasekvenser. Differentiell testning är en etablerad teknik inom vanlig (icke-symbolisk) testning. I bidraget visar vi hur denna teknik kan anpassas för symbolisk exekvering av kommunikationsprotokoll. Anpassningen bygger till vissa delar på den monitorbaserade tekniken som utvecklats i det andra bidraget. Avvikelser i beteende kan därigenom avslöja potentiella fel eller specifikationsbrott även när explicita krav inte har formulerats.

I det fjärde bidraget studeras kravbaserad testning av IoT-protokoll genom en fallstudie utförd på IoT-protokollet CoAP. Studien är en empirisk jämförelse mellan täckningsstyrd fuzzing och symbolisk exekvering, utförd på CoAP. Den bygger på den kravdrivna testmetoden som utvecklats i avhandlingens första bidrag. Bidraget visar hur denna metod även kan användas för att testa mot protokollkrav genom täckningsstyrd fuzzing. Analysen belyser teknikernas kompletterande styrkor och svagheter vid testning av protokollkrav.

De föreslagna teknikerna har utvärderats på verkliga implementationer av de vitt använda protokollen DTLS, QUIC och CoAP. Som resultat av studierna har ett stort antal tidigare okända buggar och brott mot specifikationskrav upptäckts, av vilka majoriteten har bekräftats och åtgärdats av utvecklarna. Vidare har arbetet resulterat i en allmänt tillgänglig infrastruktur med återanvändbara testomgivningar och monitorer för de testade protokollen, som kan användas för att reproducera våra resultat och bygga vidare på dem. Resultaten visar att kravdriven, monitorbaserad och differentiell symbolisk exekvering kan avslöja semantiska avvikelser från protokollspecifikationer som är svåra att upptäcka med konventionella testmetoder, och att teknikerna är praktiskt tillämpbara även på produktionsmogen programvara.

12. Acknowledgements

I would like to begin by expressing my sincere gratitude to my main supervisor, Bengt Jonsson. Throughout my doctoral studies, he was always approachable and willing to help, offering thoughtful guidance and constructive feedback whenever it was needed. At the same time, he placed great trust in me and gave me the freedom to explore ideas, take risks, and shape my own research direction without micromanagement. This balance of support, trust, and independence was deeply appreciated and played a crucial role in my development as a researcher.

I would like to warmly thank Konstantinos Sagonas, a mentor, close collaborator, and co-author, for his exceptional attention to detail and his dedication to high-quality research. He has an impressive ability to spot even the smallest issues, and his careful scrutiny consistently strengthened our work. At the same time, he combined rigor with kindness, making discussions both demanding and genuinely enjoyable. I frequently turned to him for his opinion to ensure that nothing important had been overlooked, and his insight and thoroughness had a substantial impact on the quality of this thesis. Beyond his academic guidance, he was also consistently supportive and helpful with practical, everyday matters, which made the entire process smoother and more manageable.

I am deeply grateful to my second supervisor, Paul Fiterău-Broștean, for his constant support throughout my PhD. From the very beginning, he was always there to help with everything from navigating practical matters, such as banking and administrative issues, to engaging in daily discussions about research ideas and technical details. Beyond his academic guidance, his generosity with his time, his encouragement, and his friendship made the PhD journey far more enjoyable.

I would also like to thank several senior colleagues at the IT Department for their support and guidance throughout my PhD. In particular, I am grateful to Mohamed Faouzi Atig, Deputy Head of the IT Department, as well as Philipp Rümmer and Parosh Aziz Abdulla, for their help with structuring my PhD studies and for their advice on navigating departmental responsibilities and long-term planning. Their support was invaluable and greatly contributed to making my doctoral studies run smoothly.

I would also like to thank my officemates and friends, Magnus and Sarbojit, who were both knowledgeable colleagues and genuinely pleasant companions. Our conversations, ranging from research discussions to broader reflections on life, made everyday work more enjoyable. I am also grateful to my colleagues

Clément, Fredrik, and Stephan, whose presence and interactions contributed to a friendly and supportive environment and made my PhD experience more pleasant overall.

I would also like to thank my friends in Uppsala who made life during my PhD both richer and more enjoyable. Their companionship, support, and shared moments provided a welcome balance to academic life. In particular, I am grateful to, in lexicographic order, Alireza, Ashkan, Behnaz, Ehsan, Kasperi, Lan, Mahdi, Parnian, Rashid, Sajjad, Sheida, Sina, Soheil, and Sou-san for making my time in Uppsala more pleasant and memorable.

I would also like to thank my fiancée, Sara, for her unwavering support, kindness, and care throughout the final years of my PhD. We got to know each other during the last two years of my doctoral studies, and during that time she consistently helped me manage stress and find calm during difficult moments. Her presence was a source of love, warmth, and light, especially during the long and dark Swedish winters. Her support, understanding, and companionship meant more to me than I can express.

Finally, I would like to express my deepest gratitude to my parents, Hanieh and Mohammad, and to my brother, Armin. Their constant love, support, and encouragement have been with me throughout this journey and in every stage of my life. I would not be who I am today without them, and this work would not have been possible without their unwavering belief in me and my path.

Thank you for taking the time to read these paragraphs. If you were part of my PhD journey and were not mentioned, I apologize for any unintended omission.

References

- [1] Fides Aarts, Joeri de Ruiter, and Erik Poll. Formal models of bank cards for free. In *4th International Workshop on Security Testing, SECTEST 2013*, pages 461–468, Los Alamitos, CA, USA, 2013. IEEE Computer Society.
- [2] Dave Aitel. An introduction to spike, the fuzzer creation kit. <https://blackhat.com/presentations/bh-usa-02/bh-us-02-aitel-spike.ppt>, 2002.
- [3] Anastasios Andronidis and Cristian Cadar. SnapFuzz: High-throughput fuzzing of network applications. In *31st ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA '22*, pages 340–351, New York, NY, USA, July 2022. ACM.
- [4] Thomas Arts, John Hughes, Joakim Johansson, and Ulf Wiger. Testing telecoms software with Quviq QuickCheck. In *Proceedings of the 2006 ACM SIGPLAN Workshop on Erlang*, pages 2–10, New York, NY, USA, 2006. ACM.
- [5] Hooman Asadian, Paul Fiterău-Broștean, Bengt Jonsson, and Konstantinos Sagonas. Applying symbolic execution to test implementations of a network protocol against its specification. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2022*, pages 70–81. IEEE, April 2022.
- [6] Jinsheng Ba, Marcel Böhme, Zahra Mirzamomen, and Abhik Roychoudhury. Stateful greybox fuzzing. In Kevin R. B. Butler and Kurt Thomas, editors, *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022*, pages 3255–3272. USENIX Association, 2022.
- [7] Benjamin Beurdouche, Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, Alfredo Pironti, Pierre-Yves Strub, and Jean Karim Zinzindohoue. A messy state of the union: Taming the composite state machines of TLS. *Commun. ACM*, 60(2):99–107, February 2017.
- [8] Carsten Bormann and Zach Shelby. Block-Wise Transfers in the Constrained Application Protocol (CoAP). RFC 7959, August 2016.
- [9] S. Bradner. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, March 1997.
- [10] Manfred Broy, Bengt Jonsson, Joost-Pieter Katoen, Martin Leucker, and Alexander Pretschner, editors. *Model-Based Testing of Reactive Systems, Advanced Lectures*, volume 3472 of LNCS. Springer, 2005.
- [11] Cristian Cadar, Daniel Dunbar, and Dawson Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation, OSDI '08*, pages 209–224, Berkeley, CA, USA, 2008. USENIX Association.
- [12] Cristian Cadar and Koushik Sen. Symbolic execution for software testing: three decades later. *Commun. ACM*, 56(2):82–90, 2013.

- [13] Sze Yiu Chau, Omar Chowdhury, Md. Endadul Hoque, Huangyi Ge, Aniket Kate, Cristina Nita-Rotaru, and Ninghui Li. SymCerts: Practical symbolic execution for exposing noncompliance in X.509 certificate validation implementations. In *2017 IEEE Symposium on Security and Privacy*, SP 2017, pages 503–520. IEEE Computer Society, May 2017.
- [14] Sze Yiu Chau, Moosa Yahyazadeh, Omar Chowdhury, Aniket Kate, and Ninghui Li. Analyzing semantic correctness with symbolic execution: A case study on PKCS#1 v1.5 signature verification. In *26th Annual Network and Distributed System Security Symposium*, NDSS 2019. The Internet Society, February 2019.
- [15] Chu Chen, Cong Tian, Zhenhua Duan, and Liang Zhao. RFC-directed differential testing of certificate validation in SSL/TLS implementations. In Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman, editors, *Proceedings of the 40th International Conference on Software Engineering*, ICSE 2018, pages 859–870. ACM, May–June 2018.
- [16] Nanxi Chen, Paul Fiterău-Broștean, and Konstantinos Sagonas. State fuzzing DTLS 1.3 implementations. In *IEEE Conference on Standards for Communications and Networking (CSCN)*, pages 426–432. IEEE, 2025.
- [17] Vitaly Chipounov, Volodymyr Kuznetsov, and George Candea. The S2E platform: Design, implementation, and applications. *ACM Trans. Comput. Syst.*, 30(1):2:1–2:49, 2012.
- [18] Lesly-Ann Daniel, Erik Poll, and Joeri de Ruiter. Inferring OpenVPN state machines using protocol state fuzzing. In *IEEE European Symposium on Security and Privacy (EuroS&P) Workshops*, pages 11–19. IEEE, April 2018.
- [19] Cristian Daniele, Timme Bethe, Marcello Maugeri, Andrea Continella, and Erik Poll. LibAFLstar: Fast and state-aware protocol fuzzing. In *European Symposium on Research in Computer Security*, pages 105–123. Springer, 2025.
- [20] Leonardo De Moura and Nikolaj Bjørner. Z3: an efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, TACAS’08/ETAPS’08, pages 337–340, Berlin, Heidelberg, 2008. Springer-Verlag.
- [21] Joeri de Ruiter and Erik Poll. Protocol state fuzzing of TLS implementations. In *24th USENIX Security Symposium*, pages 193–206. USENIX Association, August 2015.
- [22] Tim Dierks and Eric Rescorla. The transport layer security TLS protocol version 1.2. RFC 5246, August 2008.
- [23] Xavier Manuel van Dommelen, Machiel van der Bijl, and Andy Pimentel. Model-based testing of internet of things protocols. In *International Conference on Formal Methods for Industrial Critical Systems*, pages 172–189. Springer International Publishing, 2022.
- [24] Michael Eddington. Peach fuzzer. <https://peachtech.gitlab.io/peach-fuzzer-community/>, 2004.
- [25] Wesley Eddy. Transmission Control Protocol (TCP). RFC 9293, August 2022.
- [26] Tiago Ferreira, Harrison Brewton, Loris D’Antoni, and Alexandra Silva. Prognosis: Closed-box analysis of network protocol implementations. In *ACM SIGCOMM 2021 Conference*, pages 762–774. ACM, August 2021.

- [27] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. AFL++: Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, August 2020.
- [28] Paul Fiterău-Broștean and Falk Howar. Learning-based testing the sliding window behavior of TCP implementations. In *Critical Systems: Formal Methods and Automated Verification - Joint 22nd International Workshop on Formal Methods for Industrial Critical Systems and 17th International Workshop on Automated Verification of Critical Systems, FMICS-AVoCS 2017*, volume 10471 of *LNCS*, pages 185–200. Springer, September 2017.
- [29] Paul Fiterău-Broștean, Bengt Jonsson, Konstantinos Sagonas, and Fredrik Tåquist. Automata-based automated detection of state machine bugs in protocol implementations. In *30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, February 27 - March 3, 2023*. The Internet Society, 2023.
- [30] Paul Fiterău-Broștean, Ramon Janssen, and Frits W. Vaandrager. Combining model learning and model checking to analyze TCP implementations. In *Computer Aided Verification - 28th International Conference, CAV 2016, Proceedings, Part II*, volume 9780 of *LNCS*, pages 454–471. Springer, 2016.
- [31] Paul Fiterău-Broștean, Bengt Jonsson, Robert Merget, Joeri de Ruiter, Konstantinos Sagonas, and Juraj Somorovsky. Analysis of DTLS implementations using protocol state fuzzing. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2523–2540. USENIX Association, August 2020.
- [32] Paul Fiterău-Broștean, Bengt Jonsson, Konstantinos Sagonas, and Fredrik Tåquist. DTLS-Fuzzer: A DTLS protocol state fuzzer. In *15th IEEE International Conference on Software Testing, Verification and Validation, ICST 2022*. IEEE Computer Society, April 2022.
- [33] Paul Fiterău-Broștean, Toon Lenaerts, Joeri de Ruiter, Erik Poll, Frits W. Vaandrager, and Patrick Verleg. Model learning and model checking of SSH implementations. In *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software*, SPIN 2017, pages 142–151. ACM, 2017.
- [34] Shalini Ghosh, Daniel Elenius, Wenchao Li, Patrick Lincoln, Natarajan Shankar, and Wilfried Steiner. ARSENAL: automatic requirements specification extraction from natural language. In *NASA Formal Methods Symposium*, pages 41–46. Springer, 2016.
- [35] Jiaxing Guo, Chunxiang Gu, Xi Chen, and Fushan Wei. Model learning and model checking of IPSec implementations for Internet of Things. *IEEE Access*, 7:171322–171332, November 2019.
- [36] Janardhan Iyengar and Martin Thomson. QUIC: A UDP-based multiplexed and secure transport. RFC 9000, January 2021.
- [37] James C. King. Symbolic execution and program testing. *Commun. ACM*, 19(7):385–394, July 1976.
- [38] Yishuai Li, Benjamin C. Pierce, and Steve Zdancewic. Model-based testing of networked applications. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2021*, pages 529–539, New York, NY, USA, 2021. Association for Computing Machinery.

- [39] Fredrik Liljedahl. Exploring the possibilities of robustness testing of CoAP implementations using evolutionary fuzzing. Master thesis, KTH, Stockholm, Sweden, 2019.
- [40] Andreas Löschner and Konstantinos Sagonas. Targeted property-based testing. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2017, pages 46–56, New York, NY, USA, 2017. ACM.
- [41] Andreas Löschner and Konstantinos Sagonas. Automating targeted property-based testing. In *11th IEEE International Conference on Software Testing, Verification and Validation*, ICST 2018, pages 70–80. IEEE Computer Society, April 2018.
- [42] Andreas Löschner, Konstantinos Sagonas, and Thiemo Voigt. Property-based testing of sensor networks. In *Sensing, Communication, and Networking, 12th Annual IEEE International Conference on*, SECON '15, pages 100–108. IEEE, June 2015.
- [43] William M McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.
- [44] Chris McMahon Stone, Tom Chothia, and Joeri de Ruiter. Extending automated protocol state learning for the 802.11 4-way handshake. In *Computer Security*, volume 11098 of *LNCS*, pages 325–345, Cham, August 2018. Springer International Publishing.
- [45] Kenneth L. McMillan and Lenore D. Zuck. Formal specification and testing of QUIC. In *Proceedings of the ACM Special Interest Group on Data Communication*, SIGCOMM 2019, pages 227–240. ACM, September 2019.
- [46] Bruno Melo and Paulo Geus. Robustness testing of CoAP server-side implementations through black-box fuzzing techniques. In *Proceedings of the 17th Brazilian Symposium on Information and Computational Systems Security*, pages 533–540. SBC, 2017.
- [47] Ruijie Meng, Zhen Dong, Jialin Li, Ivan Beschastnikh, and Abhik Roychoudhury. Linear-time temporal logic guided greybox fuzzing. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE' 22, pages 1343–1355, New York, NY, USA, May 2022. ACM.
- [48] Bodo Möller, Thai Duong, and Krzysztof Kotowicz. This POODLE bites: exploiting the SSL 3.0 fallback, 2014.
- [49] Roberto Natella. StateAFL: Greybox fuzzing for stateful network servers. *Empirical Software Engineering*, 27(7):191, 2022.
- [50] Luis Pedrosa, Ari Fogel, Nupur Kothari, Ramesh Govindan, Ratul Mahajan, and Todd D. Millstein. Analyzing protocol implementations for interoperability. In *12th USENIX Symposium on Networked Systems Design and Implementation*, NSDI 15, pages 485–498. USENIX Association, May 2015.
- [51] Suzette Person, Matthew B. Dwyer, Sebastian Elbaum, and Corina S. Pasareanu. Differential symbolic execution. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, SIGSOFT '08/FSE-16, pages 226–237, New York, NY, USA, 2008. Association for Computing Machinery.
- [52] Van-Thuam Pham, Marcel Böhme, and Abhik Roychoudhury. AFLNET: A greybox fuzzer for network protocols. In *IEEE 13th International Conference*

- on *Software Testing, Validation and Verification*, ICST 2020, pages 460–465. IEEE, October 2020.
- [53] Erik Poll, Joeri de Ruiter, and Aleksy Schubert. Protocol state machines and session languages: Specification, implementation, and security flaws. In *IEEE Symposium on Security and Privacy Workshops*, SPW 2015, pages 125–133. IEEE Computer Society, May 2015.
 - [54] Clement Poncelet, Konstantinos Sagonas, and Nicolas Tsiftes. So many fuzzers, so little time*: Experience from evaluating fuzzers on the Contiki-NG network (hay)stack. In *37th IEEE/ACM International Conference on Automated Software Engineering*, ASE '22, New York, NY, USA, 2023. ACM.
 - [55] Harald Raffelt, Maik Merten, Bernhard Steffen, and Tiziana Margaria. Dynamic testing via automata learning. *International journal on software tools for technology transfer*, 11(4):307–324, 2009.
 - [56] Abdullah Rasool, Greg Alpár, and Joeri de Ruiter. State machine inference of QUIC. arXiv preprint arXiv:1903.04384, 2019.
 - [57] Felix Rath, Daniel Schemmel, and Klaus Wehrle. Interoperability-guided testing of QUIC implementations using symbolic execution. In *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC*, EPIQ@CoNEXT 2018, pages 15–21. ACM, December 2018.
 - [58] E. Rescorla, M. Ray, S. Dispensa, and N. Oskov. Transport layer security (TLS) renegotiation indication extension. RFC 5746, February 2010.
 - [59] Eric Rescorla and Nagendra Modadugu. Datagram transport layer security version 1.2. RFC 6347, January 2012.
 - [60] Riku, Antti, Matti, and Neel Mehta. Heartbleed, CVE-2014-0160, 2015.
 - [61] Ivan Ristic. Poodle bites TLS, 2014.
 - [62] Konstantinos Sagonas and Thanasis Typaldos. EDHOC-Fuzzer: An EDHOC protocol state fuzzer. In René Just and Gordon Fraser, editors, *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA '23, pages 1495–1498, New York, NY, USA, July 2023. ACM.
 - [63] Raimondas Sasnauskas, Philipp Kaiser, Russ Lucas Jukic, and Klaus Wehrle. Integration testing of protocol implementations using symbolic distributed execution. In *20th IEEE International Conference on Network Protocols*, ICNP 2012, pages 1–6. IEEE Computer Society, October–November 2012.
 - [64] Raimondas Sasnauskas, Olaf Landsiedel, Muhammad Hamad Alizai, Carsten Weise, Stefan Kowalewski, and Klaus Wehrle. KleeNet: Discovering insidious interaction bugs in wireless sensor networks before deployment. In *Proceedings of the 9th International Conference on Information Processing in Sensor Networks*, IPSN 2010, pages 186–196. ACM, April 2010.
 - [65] Raimondas Sasnauskas, Jó Ágila Bitsch Link, Muhammad Hamad Alizai, and Klaus Wehrle. KleeNet: automatic bug hunting in sensor network applications. In *Proceedings of the 6th International Conference on Embedded Networked Sensor Systems*, SenSys 2008, pages 425–426. ACM, November 2008.
 - [66] Zach Shelby, Klaus Hartke, and Carsten Bormann. The Constrained Application Protocol (CoAP). RFC 7252, June 2014.
 - [67] JaeSeung Song, Cristian Cadar, and Peter R. Pietzuch. SYMBEXNET: Testing network protocol implementations with symbolic execution and rule-based specifications. *IEEE Trans. Software Eng.*, 40(7):695–709, 2014.

- [68] W. Richard Stevens. *Unix network programming*. Prentice Hall, 1990.
- [69] Kotaro Tanabe, Yoshinori Tanabe, and Masami Hagiya. Model-based testing for MQTT applications. In *Joint Conference on Knowledge-Based Software Engineering*, pages 47–59. Springer International Publishing, 2020.
- [70] Martin Tappler, Bernhard K. Aichernig, and Roderick Bloem. Model-based testing IoT communication via active automata learning. In *Software Testing, Verification and Validation, (ICST) 2017 IEEE International Conference on*, pages 276–287. IEEE Computer Society, March 2017.
- [71] Sören Tempel, Vladimir Herdt, and Rolf Drechsler. Specification-based symbolic execution for stateful network protocol implementations in the IoT. *IEEE Internet of Things Journal*, 2023.
- [72] Mark Utting and Bruno Legeard. *Practical Model-Based Testing - A Tools Approach*. Morgan Kaufmann, 1st edition, November 2006.
- [73] Frits W. Vaandrager. Model learning. *Communications of the ACM*, 60(2):86–95, 2017.
- [74] Niels van Drueten. Security analysis of DTLS 1.2 implementations. Bachelor thesis, Radboud University, Nijmegen, The Netherlands, 2019.
- [75] Margus Veanes, Colin Campbell, Wolfgang Grieskamp, Wolfram Schulte, Nikolai Tillmann, and Lev Nachmanson. Model-based testing of object-oriented reactive systems with Spec Explorer. In Robert M. Hierons, Jonathan P. Bowen, and Mark Harman, editors, *Formal Methods and Testing, An Outcome of the FORTEST Network, Revised Selected Papers*, volume 4949 of *LNCS*, pages 39–76. Springer, 2008.
- [76] Zhongjie Wang, Shitong Zhu, Yue Cao, Zhiyun Qian, Chengyu Song, Srikanth V Krishnamurthy, Kevin S Chan, and Tracy D Braun. Symtcp: Eluding stateful deep packet inspection with automated discrepancy discovery. In *NDSS*, NDSS 2020, 2020.
- [77] Shameng Wen, Qingkun Meng, Chao Feng, and Chaojing Tang. A model-guided symbolic execution approach for network protocol implementations and vulnerability detection. *PloS one*, 12(11):e0188229, 2017.
- [78] Johannes Wilson and Mikael Asplund. Analysing TLS implementations using full-message symbolic execution. In *Nordic Conference on Secure IT Systems*, pages 283–302. Springer, 2024.
- [79] Aya Zaki-Ismail, Mohamed Osama, Mohamed Abdelrazek, John Grundy, and Amani Ibrahim. RCM-extractor: an automated NLP-based approach for extracting a semi formal representation model from natural language requirements. *Automated Software Engineering*, 29(1):10, 2022.
- [80] Michał Zalewski. American fuzzy lop. <http://lcamtuf.coredump.cx/af1/>, 2013.
- [81] Yingpei Zeng, Mingmin Lin, Shanqing Guo, Yanzhao Shen, Tingting Cui, Ting Wu, Qihua Zheng, and Qihua Wang. MultiFuzz: A coverage-based multiparty-protocol fuzzer for IoT publish/subscribe protocols. *Sensors*, 20(18), 2020.
- [82] Mingwei Zheng, Qingkai Shi, Xuwei Liu, Xiangzhe Xu, Le Yu, Congyu Liu, Guannan Wei, and Xiangyu Zhang. Pardiff: Practical static differential analysis of network protocol parsers. *Proceedings of the ACM on Programming Languages*, 8:1208–1234, 04 2024.

Acta Universitatis Upsaliensis

Digital Comprehensive Summaries of Uppsala Dissertations from the Faculty of Science and Technology 2632

Editor: The Dean of the Faculty of Science and Technology

A doctoral dissertation from the Faculty of Science and Technology, Uppsala University, is usually a summary of a number of papers. A few copies of the complete dissertation are kept at major Swedish research libraries, while the summary alone is distributed internationally through the series Digital Comprehensive Summaries of Uppsala Dissertations from the Faculty of Science and Technology. (Prior to January, 2005, the series was published under the title "Comprehensive Summaries of Uppsala Dissertations from the Faculty of Science and Technology".)



Distribution: publications.uu.se
urn:nbn:se:uu:diva-577919

ACTA UNIVERSITATIS
UPSALIENSIS
2026

Paper I



Applying Symbolic Execution to Test Implementations of a Network Protocol Against its Specification

Hooman Asadian , Paul Fiterău-Broștean , Bengt Jonsson , Konstantinos Sagonas 

Department of Information Technology, Uppsala University, Uppsala, Sweden

{hooman.asadian, paul.fiterau_brostean, bengt, kostis}@it.uu.se

Abstract—Implementations of network protocols must conform to their specifications in order to avoid security vulnerabilities and interoperability issues. We describe our experiences using symbolic execution to thoroughly test several implementations of a network security protocol against its specification. We employ a methodology in which we first extract requirements from the protocol's RFC and turn them into formulas. These formulas are then utilized by symbolically executing the protocol implementation to explore code paths that can be traversed on packet sequences that violate a requirement. When this exploration exposes a bug, corresponding input values are produced and turned into test cases that can validate the bug in the original implementation. Since we let symbolic execution be guided by requirements, it can naturally produce a wide variety of requirement-violating input sequences, which is difficult to achieve with existing techniques for protocol testing. We applied this methodology to test four different implementations of DTLS against the protocol's RFC. We were able to quickly expose a known CVE in an older version of OpenSSL, and to discover numerous previously unknown vulnerabilities and non-conformance issues in DTLS implementations, which have by now been confirmed and fixed by their implementors.

Index Terms—symbolic execution, network security testing

I. INTRODUCTION

Implementations of network protocols, such as TCP, TLS, DTLS, etc., must conform to their specifications in order to avoid security vulnerabilities and interoperability issues. Even seemingly innocent deviations from the standard specification may open implementations up for security attacks. Examples include Heartbleed [8] and the TLS POODLE [3], [23] downgrade vulnerability enabled by insufficient checking of length fields or version numbers in input packets. At the same time, testing of protocol implementations is made difficult by the fact that they are *stateful*. To test the processing of a particular packet, the implementation must first be brought to a specific state by an appropriate packet sequence. Moreover, the requirements concerning that packet may depend on the preceding sequence. Thus, despite the fact that techniques for testing of single-input programs such as fuzzing [39] and symbolic execution [7], [12], [19] have made impressive progress in recent years, their extensions to stateful protocols [25], [28], [29] have not yet achieved the same level of effectiveness as their single-input counterparts: these approaches are not able

to consider a sufficiently large range of adversarial inputs, and may miss bugs that are exposed by specific input sequences.

In this paper, we describe our experiences using symbolic execution to thoroughly test implementations of a network protocol against its specification. We employ a methodology, in which we first form a specification by extracting requirements from the protocol's RFC and formulating them as assertions over the sequence of packets exchanged during a session. The extraction can be done incrementally, considering individual requirements separately. One such requirement could for example concern the sequence numbers in a sequence of packets, another one the version negotiation, etc. We then use symbolic execution to search for code paths and corresponding packet sequences that violate these requirements. In order to test the processing of a sequence of packets, each protocol party is tested separately: we consider the packets it receives as a sequence of inputs and check the party's output when processing this sequence. We leverage symbolic execution to explore the code paths that an implementation may traverse when processing a packet sequence, tailoring it to explore only those code paths and inputs which expose potential requirement violations. Whenever such a violation, crash, or memory error is observed, the symbolic execution engine generates specific values that trigger this error, which are used to construct a complete test case to validate the bug. To achieve scalability for symbolic execution over a packet sequence, only those parts of input packets that are relevant for the tested requirement are made symbolic, using a pre-recorded sequence of packets as a basis.

We applied our methodology to test four implementations of DTLS against the protocol's RFC. The DTLS (Datagram Transport Layer Security) protocol is a variation of TLS over UDP. DTLS is widely used in wireless networks, and is currently one of the primary protocols for securing IoT applications [30]. Two of the DTLS implementations we tested (OpenSSL and MbedTLS) are among the most widely used and well-tested. Using our methodology, we were able to detect numerous, previously unknown, security vulnerabilities and non-conformance issues in them, which have been confirmed and/or fixed by now.

In short, the main contributions of our work are:

- We describe an experience in using symbolic execution to thoroughly test implementations of a network protocol against its specification, in which we tailor symbolic execution to explore code paths that can expose violations

of requirements formulated over the processing of a sequence of packets in a session.

- We describe how our effort was applied to four DTLS implementations against the protocol's RFC, demonstrating the effectiveness of our methodology on DTLS by revealing more than thirty previously unknown vulnerabilities and non-conformance bugs in well-tested DTLS implementations.
- We provide replication material [2] for all our experiments.

Outline: After an overview of our effort in the next section, in §III we show how requirements from DTLS's RFC are encoded as formulas. We describe our implementation in §IV, evaluate its effectiveness in §V, discuss related works in §VI, and end with some final remarks.

II. OVERVIEW OF OUR METHODOLOGY

In this section, we give a high-level overview of the methodology we used. It consists of three steps: i) a step where requirements are extracted from the protocol's RFC or specification and turned into formulas; ii) a step where (negations of) these formulas are represented as assumptions and assertions in the source code of the protocol's implementation, and where symbolic execution is used to explore code paths that expose bugs in the implementation and produce values for (selected) parts of the input(s) that follow these paths; and iii) a step where complete test cases for these bugs are constructed and are used to validate the bugs in the unmodified implementation. We detail these steps below.

A. Extracting Specification Requirements

Protocols encompass specific requirements over sequences of packets exchanged by two or more parties, sequences which are also known as *protocol sessions*. For DTLS, as well as for any other network protocol, we can extract a set of these requirements by scrutinizing its Request for Comments (RFC) [27, p. 13]. This task is facilitated by the fact that it is common for RFCs to use particular keywords (e.g., "MUST", "MUST NOT", "REQUIRED", "SHALL NOT", etc.) to signify the strictness of the protocol's requirements [5]. For example, the keyword "MUST" expresses that the definition is an absolute requirement of the specification, while "SHOULD" indicates a recommendation. Starting from these keywords, one can derive a set of requirements that can be represented by formulas over sequences of inputs and outputs.

A prominent class of requirements, which we will refer to as *input validity requirements*, concerns checking well-formedness of (the sequence of) inputs. For example, the following sentence from the DTLS RFC, specifies a requirement regarding uniqueness of record sequence numbers in sessions:

For each received record, the receiver MUST verify that the record contains a sequence number that does not duplicate the sequence number of any other record received during the life of this session.

For a set of records R received during a DTLS session, we can represent this requirement in predicate logic by the formula:

$$\forall r, r' \in R: r \neq r' \implies r.\text{sequence_number} \neq r'.\text{sequence_number} \quad (1)$$

Another class of requirements concerns output generated in response to (a sequence of) inputs. We refer to these as *input-output requirements*. For example, the following sentence, extracted from the DTLS RFC [27, p. 17] and its errata (ID: 5186), concerns the sequence number in a server's response.

In order to avoid sequence number duplication in case of multiple cookie exchanges, the server MUST use the message_seq in the ClientHello as the message_seq in its initial ServerHello.

Letting $\text{resp}(r, i)$ represent the i -th output generated in response to input record r , we can express this requirement by the formula:

$$\begin{aligned} \forall r \in R: r.\text{msg_type} = \text{client_hello} \implies \\ (\text{resp}(r, 1).\text{msg_type} = \text{server_hello} \implies \\ \text{resp}(r, 1).\text{message_seq} = r.\text{message_seq}) \end{aligned} \quad (2)$$

which uses the fact that the `msg_type` field of a record r contains the type of its enclosed (and perhaps fragmented) message.

B. Symbolic Execution

Given a set of requirements expressed using formulas such as (1) and (2), our goal is to detect bugs (constraint violations, non-conformances, crashes and security vulnerabilities that may be associated with not adhering to the RFC, etc.) in protocol implementations (SUTs). Conceptually, the core idea of this step is simple. For each requirement we insert instrumentation into the SUT's source code as follows: 1) input constraints under which the requirement *can* be violated are inserted as *assumptions* on input to the SUT's source code, and 2) checks whether the SUT actually performs an action that violates the requirement are inserted as *assertions*. We then use *symbolic execution* to explore those executions that satisfy the assumptions on input, looking out for actions that trigger an assertion violation, crash, or memory error, and constructing test cases that are *witnesses* for each of these bugs.

Symbolic execution analyzes programs for which (some of) the input variables are designated as *symbolic*. It explores the code paths that are possible for some values of the symbolic input, and also provides input values that make executions follow specific code paths. In order to apply symbolic execution to a SUT, we represent the sequence of input packets as structured-type variables, which can be named after their message types as `client_hello`, `client_key_exchange`, etc. Making complete packets symbolic would in many cases not scale, therefore only the fields that can be expected to be relevant for the considered requirement are made symbolic. For instance, the payload, length, and several other fields, are unlikely to affect the processing of sequence numbers, and need not be made symbolic when checking requirements on sequence numbers. The non-symbolic fields must then be given concrete default values; we take these from the packet sequence in a pre-recorded session.

Let us illustrate the above with the examples from §II-A. An input validity constraint, such as Formula (1), refers to a typically unrestricted set R of records received during a DTLS session¹. We can specialize R to be a set of three records

¹Occasionally, the RFC might specify related constraints that limit R .

carrying message types `client_hello2`, `client_key_exchange`, and `change_cipher_spec`. In this set, all its elements have a `sequence_number` field. To prepare for symbolic execution, we pre-record a session containing these message types, represent the records as structured-type variables and make their `sequence_number` fields symbolic. We then add to the SUT's code the following assume statement (in C/C++ syntax), assuming the negation of the constraint expressed by that formula (as this is the constraint under which the requirement can be violated):

```
assume( ! // negate the conjunction below
(client_hello2.sequence_number != client_key_exchange.sequence_number &
 client_hello2.sequence_number != change_cipher_spec.sequence_number &
 client_key_exchange.sequence_number != change_cipher_spec.sequence_number) )
```

What we have done here is to manually construct the pair-wise conjunction that the universal quantifier of Formula (1) will produce, by specializing it to the case of three records with the chosen message types. Of course, when the formulas are more complicated or the sets of records that they involve are larger, such assume statements can call appropriate auxiliary functions that we add to the SUT instead of specifying the constraints inline, as we did here. We must also add an assert statement to check that the protocol does not use invalid input in some forbidding way. For this case, the DTLS 1.2 RFC [27, p. 14] specifies that “Invalid records *SHOULD* be silently discarded”, so we may want to check whether an implementation achieves progress (e.g., by changing states) even after reception of invalid records. In implementations that already provide an API to inspect their internal state(s) or when it is easy to add one, the detection of such non-conformances can be fine-grained and very precise. A coarser-grained approach, which is the one we have implemented, is to simply check whether protocol interactions complete successfully (e.g., handshake is achieved) even in the presence of an invalid record in a test; this is done by an `assert(false)` at the point where the SUT is about to complete the interaction. For the DTLS protocol, this clearly indicates the violation of a “SHOULD”-type requirement, and thus non-conformance of the SUT to the standard.

For input-output requirements, such as Formula (2), we augment the SUT with structure-type variables that represent relevant output packets. Specializing $\text{resp}(r, i)$ to the case where r is the packet `client_hello2`, we add the assertion

```
assert( // checks message_seq validity
  (resp(client_hello2,1).msg.type != server_hello) |
  (resp(client_hello2,1).message_seq == client_hello2.message_seq) )
```

Since an input-output requirement can be violated only on valid inputs, we add an assume statement ensuring the validity of the field `client_hello2.message_seq`.

In summary, a protocol requirement expressed by a first-order logic formula is checked by augmenting the SUT as follows: 1) variables are inserted to represent the input packets and (in the case of input-output requirements) relevant output packets; 2) the formula is specialized by instantiating its quantifiers to the possible values in the pre-recorded sequence, producing a quantifier-free expression over the packet-representing variables; 3) for an input validity requirement, its negation is added in an assume statement, and an assert statement is added to

check that the invalid inputs are not handled in a forbidden way; 4) for an input-output requirement, the expression is added in an assert statement checking correctness of the output, together with an assume statement ensuring the validity of inputs.

C. Test Case Construction and Validation

Starting from some pre-recorded valid input sequence S of records as seed, and given some time budget T , a symbolic execution tool such as KLEE [7] will explore a number of code paths of SUT^a (the SUT that has been extended with assume and assert statements and where some fields have been made symbolic). For each code path that satisfies the assume statements and triggers an assertion violation, crash, or memory error, we record and return a tuple of values $\langle v_1, \dots, v_k \rangle$ for the fields $f_1, \dots, f_k$ of records in S which have been made symbolic. For each such code path, we can therefore construct a test case by substituting the value of record field f_i with v_i and keeping all other values in records of seed S unchanged. Each of these test cases executes a unique code path of SUT^a . We can simply run these test cases in the original SUT for validation. For tests that result in crashes (e.g., segmentation faults, memory errors, etc.), the existence of a bug in the SUT is clear. Tests that trigger assertion violations expose non-conformances and other policy violations, given our instrumentation to check protocol requirements.

III. DTLS AND ITS SPECIFICATION REQUIREMENTS

DTLS is a client-server protocol that secures communication over datagram-based transport layer protocols. It is an adaptation of TLS, fulfilling a similar purpose, but doing so for unreliable transport layer protocols such as UDP. DTLS is available in two standardized versions, 1.0 [26] and 1.2 [27], which are respectively based on TLS 1.1 [14] and TLS 1.2 [15].

DTLS secures communication through the establishment and use of session keys. To that end, it is structured in several layers. The Record layer performs encryption and encapsulation of sent data and decryption and decapsulation of received data. The Handshake layer establishes session keys and the cryptographic algorithms to use at the Record layer. Other layers include ChangeCipherSpec that handles key deployment, Application, which provides a carrier for application data, and Alert, which is used to signal unexpected events. Our work concentrates on the Record and Handshake layers. We first explain the operation of these layers, before introducing the requirements defined for each layer.

The Record layer sits at the base, with all other layers operating directly on top of it. It splits received network data into *records*. The payload of these records is decrypted and authenticated using session keys. The decrypted payload is divided into *messages*, which are dispatched to the corresponding upper layer. In the opposite direction, the Record layer encrypts messages received from upper layers, encapsulates them into records, and packs the records inside datagrams for the transport layer. Before deployment of session keys, the Record layer operates without encryption and authentication (e.g., messages are extracted directly from record payloads).

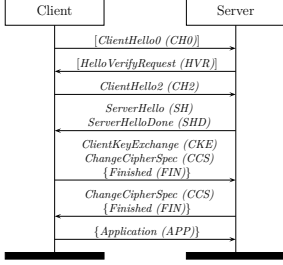


Fig. 1: DTLS handshake when pre-shared key (PSK) is used. Encrypted messages are inside braces. Optional messages are inside square brackets.

The Handshake layer is engaged at the start of each DTLS session in order to establish session keys and the cryptographic algorithms to use at the Record layer. This is achieved by completion of a DTLS *handshake*. Over the course of this handshake, the client and server exchange flights of handshake messages in a specified order; see Fig. 1 showing a handshake using the pre-shared key (PSK) algorithm. The exact form of the handshake varies depending on the key exchange algorithm used to generate session keys. In addition to key establishment, the Handshake layer is also tasked with fragmentation and reassembly of handshake messages. Handshake messages when packed inside a record may be too large to fit inside a datagram. Consequently, these messages are first split by the Handshake layer into *fragments*, before being passed on to the Record layer. Conversely, fragments received from the Record layer are first assembled to form *unfragmented messages*, and only then processed. We shall revisit these terms in §III-B.

A. Record Layer Requirements

A DTLS record is a structured entity whose fields and types are specified in the DTLS RFC [27, p. 6]. Fields which are DTLS-specific (i.e., not part of the corresponding TLS record) are identified with the comment *New field*. As we can see, the epoch number field is specified as a 16-bit unsigned integer, and the record sequence_number as a 48-bit unsigned integer. The type and version fields are defined via *enumerated types*. The value of the type field determines the upper layer the message is destined for. For example, if the value of type is handshake, the fragment should be passed on to the Handshake layer, etc. Finally, the fragment field, denoting the message contained in the record, is defined as a one-dimensional vector.

Given this structure and the TLS and DTLS RFCs, we can formulate constraints over fields of a set of records R used in a DTLS session, as follows.

1) *Record Version*: The DTLS RFC [27, p. 4] specifies only two versions for DTLS. The formula for this field is simple:

$$\forall r \in R: r.version \in \{\text{DTLS1.0}, \text{DTLS1.2}\}$$

2) *Record Length*: The TLS 1.2 RFC [15, p. 20] defines record length as: “The length (in bytes) of the following *TLSPlain-text.fragment*.” Moreover, the DTLS 1.2 RFC [27, p. 8] specifies that the length is “Identical to the length field in a TLS 1.2 record.” Hence, the length field in DTLS represents the number of bytes stored in the DTLS record’s fragment field. We capture these requirements in the following formula, where num_bytes is a function returning the size of its argument (in bytes).

$$\forall r \in R: r.length = \text{num_bytes}(r.fragment)$$

So far, the requirements for record fields were simple and quantifying over the set of records R with variables $r, r', \dots$ sufficed. But there are also RFC requirements that specify relations between consecutive records or even sequences of records that a protocol party receives during a session. We then equip r with a subscript that denotes the record’s position in the sequence. Thus $r_1, r_2, \dots$ is the sequence of records in a session (r_1 is the first record, r_2 the second, etc.). We can also quantify over subscripts. As well, for a record r_i , its previous record is obtained as r_{i-1} and its next as r_{i+1} .

3) *Epoch Number*: The epoch field is a two-byte unsigned integer used to distinguish the session keys a record was encrypted with. The following excerpt from RFC [27, p. 8] explains how this field should be updated.

The epoch should be initialized with zero, and should be incremented with every ChangeCipherSpec message sent.

For n received records in a DTLS session, we capture this requirement with the following formula:

$$\begin{aligned} \forall i \in \mathbb{N}, 2 \leq i \leq n: \\ (r_1.epoch = 0) \wedge \\ (\text{if } r_{i-1}.type = \text{change_cipher_spec} \text{ then } r_i.epoch = r_{i-1}.epoch + 1 \\ \text{else } r_i.epoch = r_{i-1}.epoch) \end{aligned}$$

This formula specifies that the epoch of the first record is zero and that for all subsequent records the value of the epoch field is either one more than the value of the previous record or has the same value as it, depending on whether the value of the type field of the previous record is change_cipher_spec or not.

4) *Record Sequence Number*: The DTLS 1.2 RFC [27] defines the record sequence_number as a 48-bit unsigned integer for the purpose of preventing replay attacks. Section II-A quoted a requirement regarding the uniqueness of the sequence_number and showed a formula for this requirement (Formula 1). In the excerpt below, the RFC [27, p. 13, 14] additionally requires rejection of records whose sequence_number values are smaller than the lowest sequence_number in the current window (i.e., the value at the left edge of the window).

Duplicates are rejected through the use of a sliding receive window. A window size of 64 is preferred and SHOULD be employed as the default. The “right” edge of the window represents the highest validated sequence number value received on this session. Records that contain sequence numbers lower than the “left” edge of the window are rejected. If the received record falls within the window and is new, or if the packet is to the right of the window, then the receiver proceeds to MAC verification.

Assuming a max function that returns the maximum of a set of values, the requirements for sequence number values are given by the following formula:

$$\forall i, j \in \mathbb{N}, 1 \leq i, j \leq n, i \neq j: \\ (r_i.\text{sequence_number} \neq r_j.\text{sequence_number}) \wedge \\ (\max(\{r_k.\text{sequence_number} - 64 \mid 1 \leq k < i\} \cup \{0\}) \leq r_i.\text{sequence_number})$$

The first conjunct is the uniqueness requirement; the second conjunct specifies the “windowing” requirement.

B. Handshake Layer Requirements

The Handshake layer exchanges a sequence of messages to establish session keys and cryptographic algorithms. Larger handshake messages may be split into fragments. An unfragmented message is also regarded as a fragment consisting of an entire message. The Handshake layer passes to the Record layer a sequence of fragments, which are encrypted if session keys have been deployed, and then transmitted as the fragment fields in a sequence of records. Handshake fragments have the structure shown below. The fields msg_type and length indicate the type and length of the (unfragmented) message from which this fragment was derived, whereas message_seq indicates the order, starting from 0, of that message in the handshake interaction. The body field stores a fragment of the original message, whose structure varies depending on msg_type.

```
struct {
  HandshakeType msg_type;
  uint24 length;
  uint16 message_seq;
  uint24 fragment_offset;
  uint24 fragment_length;
  select (HandshakeType) {
    case hello_request: HelloRequest;
    case client_hello: ClientHello;
    // eight more cases here...
    case finished: Finished;
  } body;
} Handshake;
```

The offset and the length of this fragment are given by the fragment_offset and fragment_length fields, respectively.

For the Handshake layer, we define formulas over a set M of received messages, using m to range over individual messages.

1) *Handshake Type*: The RFC [27, p. 27] specifies the valid values for msg_type as belonging to the following enumeration:

```
enum HandshakeType {
  hello_request = 0, client_hello, server_hello, hello_verify_request,
  certificate = 11, server_key_exchange, certificate_request,
  server_hello_done, certificate_verify, client_key_exchange,
  finished = 20 ;
```

We can capture the requirement that the valid values for msg_type are those in the enumeration above with the formula:

$$\forall m \in M: m.\text{msg_type} \in \{\text{hello_request}, \text{client_hello}, \dots, \text{finished}\}$$

We can strengthen this formula if we additionally consider the ordering of messages imposed by the RFC [27, p. 21]. This allows us to capture a requirement enforcing correspondence between the order of a message (given by message_seq) and its specified msg_type. Formulating this requirement is quite demanding: the order of a message for a given type may vary depending on the key exchange the handshake uses, and on the side receiving the messages (whether it is a client or a server). To keep the presentation concise, we show the formula for a handshake that uses PSK (taking the form shown in Fig. 1)

and the side in question is a server. For this particular case, the requirement can be formulated as follows:

$$\forall m \in \text{serverPSK}(M): \\ (m.\text{message_seq} = 0 \implies m.\text{msg_type} = \text{client_hello}) \wedge \\ (m.\text{message_seq} = 1 \implies m.\text{msg_type} = \text{client_hello}) \wedge \\ (m.\text{message_seq} = 2 \implies m.\text{msg_type} = \text{client_key_exchange}) \wedge \\ (m.\text{message_seq} = 3 \implies m.\text{msg_type} = \text{finished})$$

The function serverPSK returns the messages from M which the server receives during PSK handshakes. Similar requirements can be formulated for clients, as well as for handshakes using other key exchange algorithms.

Similarly as for records, there are RFC requirements that specify relations between consecutive messages. We equip m with a subscript that denotes the message’s position in the sequence. Thus $m_1, m_2, \dots$ is the sequence of received messages during a session. We can also quantify over subscripts.

2) *Message Sequence*: The RFC [27, p. 18,19] specifies the following requirement for message_seq:

The first message each side transmits in each handshake always has message_seq = 0. Whenever each new message is generated, the message_seq value is incremented by one. When a message is retransmitted, the same message_seq value is used.

The excerpt suggests that the value for message_seq equals zero in the initial message and is incremented with each following message, except when the same message is retransmitted, in which case message_seq is unchanged. To capture this requirement, we must characterize when a message is “the same” as the previous. We approximate this by equality of their body fields. This leads to the following formula:

$$\forall i \in \mathbb{N}, 2 \leq i \leq n: \\ (m_i.\text{message_seq} = 0) \wedge \\ (\text{if } m_i.\text{body} \neq m_{i-1}.\text{body} \text{ then } m_i.\text{message_seq} = m_{i-1}.\text{message_seq} + 1 \\ \text{else } m_i.\text{message_seq} = m_{i-1}.\text{message_seq})$$

In order to capture requirements related to fragmentation of messages, we continue to assume a sequence $m_1, m_2, \dots$ of messages, and additionally assume that message m_i is fragmented into num_fragments(i) fragments. We use $m_i[j]$ to refer to fragment j of message i . The next four requirements concern fragmentation.

3) *Fragment Reassembly*: For reassembling (un)fragmented handshake messages, the RFC [27, p. 20] mandates that:

When a DTLS implementation receives a handshake message fragment, it MUST buffer it until it has the entire message.

In other words, this RFC excerpt requires that every byte of the original message should exist in one of the fragments. We capture the requirement with the aid of fragment_offset and fragment_length fields in the following formula:

$$\forall i \in \mathbb{N}, 1 \leq i \leq n, \forall b, 0 \leq b < m_i.\text{length}, \exists j, 1 \leq j \leq \text{num_fragments}(i): \\ m_i[j].\text{fragment_offset} \leq b < m_i[j].\text{fragment_offset} + m_i[j].\text{fragment_length}$$

4) *Unfragmented Message Offset / Length*: If a handshake message fits in a datagram, fragmentation will not take place. In this case, the RFC [27, p. 20] specifies the following requirement on fragment_offset and fragment_length:

An unfragmented message is a degenerate case with `fragment_offset = 0` and `fragment_length = length`.

With the help of the `num_fragments` function, this requirement can be formulated as follows:

$$\forall i \in \mathbb{N}, 1 \leq i \leq n: \text{num_fragments}(i) = 1 \implies (m_i.\text{fragment_length} = m_i.\text{length}) \wedge (m_i.\text{fragment_offset} = 0)$$

5) *Message Sequence in Fragments*: In describing fragmentation, the RFC [27, p. 20] specifies the following requirement:

The sender then creates N handshake messages, all with the same message_sequence value as the original handshake message.

We can infer that the `message_sequence` of all fragments should be equal to the `message_sequence` of the unfragmented message. We can capture this requirement using the following formula:

$$\forall i \in \mathbb{N}, 1 \leq i \leq n, \forall j, 1 \leq j \leq \text{num_fragments}(i): m_i[j].\text{message_sequence} = m_i.\text{message_sequence}$$

6) *Message Length in Fragments*: The RFC [27, p. 20] makes a similar remark regarding the `message_length`:

The length field in all messages is the same as the length field of the original message.

Similarly, we can formulate this requirement as the following:

$$\forall i \in \mathbb{N}, 1 \leq i \leq n, \forall j, 1 \leq j \leq \text{num_fragments}(i): m_i[j].\text{length} = m_i.\text{length}$$

7) *Handshake Version*: Similar to how records of the DTLS Record layer contain the DTLS version, there are version fields in the message bodies of *ClientHello*, *HelloVerifyRequest* and *ServerHello* messages. Similar to III-A1, we can formulate a requirement for `client_version` and `server_version`.

8) *Requirements on Length of Fields*: Several handshake messages contain fields that define the length (in bytes) of other fields. Such fields occur in the body of a handshake message (we do not show its structure here). We can capture requirements for such fields in a similar way as for Record Length formula in III-A2. For space reasons, we omit their formulas.

We end this section by mentioning that we have formulated more input validity requirements for DTLS; we chose to present only the ones above because they revealed bugs in the implementations we tested. In our evaluation (§V), we will also check two input-output requirements; that of Formula (2) and another one which is similar. We will refer to these requirements as *CH0/HVR* and *CH2/SH Message Sequence*.

IV. IMPLEMENTATION

In this section, we describe our implementation, including efforts to overcome some obstacles. Our implementation is structured as follows. First, we prepare the SUT's code for symbolic execution (§IV-A). Second, we capture concrete records of a handshake interaction (§IV-B). For a given requirement, we then make symbolic the fields of captured records that participate in it and insert corresponding assertions and assumptions (§IV-C). The functionality for doing so is provided in a shared library, based on which a series of test harnesses were developed, one for each SUT we tested. The SUT is executed symbolically using KLEE [7]. We then

construct test cases (witnesses) from the values of symbolic variables that KLEE generates when requirement-violating behaviors or crashes are detected. These test cases are used to check if the problems can be reproduced on the unmodified SUT (§IV-D). We detail these steps below.

A. SUT Preparation

To prepare the SUT for symbolic execution, we first make modifications to its code to ensure it executes deterministically; this is necessary to make it respond in the same way during record capture as during the subsequent symbolic execution. This involves de-randomization (e.g., in TinyDTLS we modified the function `dtls_fill_random`, used to compute client/server random nonces, to fill a buffer with 1's instead of a random value), and disabling retransmissions. Finding the code places where such changes should be made typically involves a search for the 'random' keyword; the changes themselves are simple.

As mentioned in §II-B, we use completing a DTLS handshake as a means of determining whether input validity requirements are violated. To detect when the handshake is near completion, we insert an `assert(false)` at the point where the SUT is expecting *Finished*. KLEE will terminate the current path and generate a corresponding sequence of input values upon executing such a statement. One may wonder why failing assertions are inserted when *Finished* is expected and not after the handshake is completed. The main reason is to steer symbolic execution away from complicated code used for decryption and authentication. (Recall from Fig. 1 that *Finished* is the first encrypted message that a side receives.) This does mean, however, that in order to validate the bug in the unmodified SUT, we have to provide a valid *Finished* message outside of symbolic execution. We revisit this matter in §IV-D.

B. Record Capture

The next step is to capture the records of a DTLS handshake. We opted for a PSK handshake of the form shown in Fig. 1, which by design omits certain cryptographic operations (e.g., such as those in authentication and key exchange) compared to handshakes using other key exchange algorithms. This minimizes the amount of cryptographic code that is executed symbolically, and also makes symbolic execution faster. To capture records, we designed a DTLS test program that starts by instantiating a client and a server. Upon instantiation, the client generates an initial *ClientHello* record. Records are then passed back and forth between the two parties, with each transmitted record also stored in a separate file. The end result is a folder containing a file for each record in the handshake.

The client/server interaction is performed over the SUT's API. However, DTLS does not have a standardized API. Consequently, the methods for initialization and sending/receiving record data differ between the SUTs. This made coding the initial test program time-consuming (i.e., in the range of a few days) since it required familiarization with each SUT's API. We got inspiration from existing demo programs, in particular from a program used to fuzz the Mbed TLS library [18].

C. Symbolic Execution

The test program that captures records is extended to a *test harness* for symbolic execution with the aid of the *shared library* developed as part of this work. The shared library contains three types of functions:

- 1) helper functions (e.g., for loading the records from file),
- 2) functions to parse DTLS records into pre-defined DTLS data structures with a structure closely resembling that of DTLS records presented in §III-A, and
- 3) functions to make specific fields of records symbolic, and for forming boolean expressions in *assumes* and *asserts*.

By defining these functions in a shared library, we can reuse their functionality to test different DTLS implementations. This is done by linking the shared library to a new implementation and simply calling the functions from the test harness. Minor adaptations are needed to reflect any new extensions a SUT may support (present in the *ClientHello* and *ServerHello* extension headers) or different lengths of the variable-length fields (primarily for the *cipher_suites* field in *ClientHello*).

Given a requirement, the test harness is used to symbolically execute one side (a client or a server). It operates as follows.

Record Loading: The harness first calls the shared library to load the records received by that side during the captured handshake and stores them in separate data structures. For example, function `parse_CH0(uint8_t *buf, CH0 *rec)` initializes a *ClientHello0*-structured record with data from a buffer.

Making Fields Symbolic: Fields that appear in a requirement are made symbolic using `klee_make_symbolic`. Thereafter, the assumptions on input in the formalization are inserted as `klee_assume` statements, and the assertions on output as `klee_assert` statements. To aid this step, our shared library provides methods for constructing the relevant boolean expressions. For example, it includes the method:

```
void epochServer(CH0 *client_hello0, CH2 *client_hello2,  
                CKE *client_key_exchange, CCS *change_cipher_spec);
```

which makes the *epoch* fields in all supplied record entities symbolic, and assumes the negation of the *epoch* number constraint defined in §III-A3. Note that we do not make fields of *Finished* records symbolic for performance reasons similar to those given in §IV-A. This step ends by executing the test harness symbolically using KLEE.

D. Test Case Construction and Validation

During symbolic execution, for each path explored, KLEE generates corresponding values for the symbolic variables. These values are stored in separate files, with sequences leading to errors named differently, making them easily distinguishable. From these values and captured records, we build corresponding test input records which we use to validate that the problem exists (i.e., the handshake is completed) in the unmodified SUT. Validation is done by supplying test input records over UDP to client/server utilities provided with the implementation. For most requirements, validation can be performed using the input records generated via symbolic execution on a de-randomized SUT without assertions and assumptions. This exposes bugs

in the same way as on the unmodified SUT, since the de-randomization does not affect control flow. One exception to when validation can be performed in this way is when it requires some input records (e.g., *Finished*) to be constructed from previous ones using cryptographic functions. For these cases, we use existing DTLS libraries (TLS-Attacker [34] and DTLS-Fuzzer [17]) to generate test cases which expose the requirement violation.

V. EVALUATION

In parallel with extracting requirements for DTLS based on its RFC and its errata, we looked for previously reported bugs for DTLS implementations. We came across CVE-2014-0195, an exploitable security vulnerability of OpenSSL, which is explained in detail in this blog [38]. The bug is in OpenSSL's function for reassembling fragmented messages, and prompted us to implement the Handshake layer requirements regarding fragmentation (§III-B3 to §III-B6). One of the latest versions with this vulnerability is OpenSSL 1.0.1f. So we took its code and tested it. We were able to expose the vulnerability very quickly. When using OpenSSL as a server, the bug, which violates the Message Length requirement (§III-B6), was detected in just 20 seconds, and when using OpenSSL as a client in 38 seconds. In both cases, KLEE version 2.2 explored just eight paths in OpenSSL's code. Out of curiosity, we also checked the code of OpenSSL 1.0.1f for other problems, and discovered, again in less than 20 seconds, two minor non-conformance issues in its code. Both issues were previously unknown and concern not checking the DTLS version requirements (§III-A1 and §III-B7). This warm-up experiment, and in particular exposing a (known but serious) security vulnerability without much effort and very quickly, convinced us of the power and promise of our methodology. So we were eager to find out what we could discover in newer, and hopefully more robust, DTLS implementations. We present these results in the remainder of this section.

SUTs: For our evaluation, we chose a total of four different DTLS implementations. Two of them, OpenSSL and Mbed TLS, are well-known and widely used. For OpenSSL, we used 3.0.0-alpha12, the most recent pre-release of version 3.0.0 at the time of our evaluation. For Mbed TLS, we used version 2.22.0. The other two implementations are variants of TinyDTLS, a lightweight DTLS implementation targeting IoT devices. The first variant, which we will denote as TinyDTLS^E, is hosted and maintained by the Eclipse IoT project. The second variant, which we will denote as TinyDTLS^C, was branched out from Eclipse's IoT project, developed independently ever since, and is used in the Contiki-NG operating system. For both TinyDTLS variants, we used the latest commits of their development branches (7068882 and 53a0d97, respectively) at the time our evaluation was conducted. Because the 'develop' branch of Eclipse's TinyDTLS does not support Handshake layer fragmentation, we used the most recent commit (94205ff) of its 'handshake_fragmentation' branch to test TinyDTLS^E's support for DTLS's handshake fragmentation requirements.

TABLE I: Number and classes of bugs found in the SUTs we used.

	OpenSSL		Mbed TLS	TinyDTLS ^E		TinyDTLS ^C
	1.0.1f	3.0.0-alpha12	2.22.0	7068882	94205ff	53a0d97
Vulnerability	1	1	–	3	+0	2
Other	–	–	–	3	+1	1
Non-conformance	2	2	3	9	+1	10

Summary of Results: Table I shows a breakdown of the total of 36 bugs (not counting the three in OpenSSL 1.0.1f) that we detected across the five SUTs that we consider in our evaluation. In the table, we use the notation $+N$ for the bugs in the handshake fragmentation branch of TinyDTLS^E to refer to the additional bugs that this branch contains compared to the develop branch (and so that we do not count the same bugs twice). All these bugs have been reported to the developers of these DTLS implementations, and most have been confirmed and corrected by now. Also, note that we detected bugs in all the DTLS implementations we considered. In Table I, we classify these bugs into vulnerabilities, non-conformance issues, and use “Other” for bugs that do not fall clearly into the other two classes.

In the next two subsections, we describe the bugs that we detected in OpenSSL and in Eclipse’s TinyDTLS implementation, and provide measurements about our experiments. We omit detailed descriptions of the three non-conformance bugs detected in Mbed TLS, and of the thirteen bugs detected in TinyDTLS^C because they are similar to those in TinyDTLS^E.

A. Bugs in OpenSSL

On OpenSSL 3.0.0-alpha12, we detected a serious vulnerability and a non-conformance bug (cf. Table I).

The vulnerability, which involves accessing an out-of-bounds pointer and crashing the SUT, is detected in less than two minutes both when using OpenSSL as a server and as a client (cf. Tables II and III), with the help of the Handshake Type requirement (§III-B1). The vulnerability has been reported to the OpenSSL developers as issue 14906 and quickly fixed. It occurs as follows: Under normal handshake, the client initiates a handshake by sending a *ClientHello* (*CH0*) message to the server. To trigger the vulnerability, the (malicious) client instead starts the handshake by sending a *CH0* message which is however tagged with `msg_type = finished`. This fools the server, which mistakenly tries to process a *Finished* message and access the non-existing at this point cipher suite elements (which would have been agreed by the client and the server if the handshake had been properly done and the *Finished* message would have been processed when its time had come). This causes the server to crash. In a similar scenario, during a handshake, if the server responds to a *ClientHello* message with a *Finished* message, the client crashes for the same reason. This is what the OpenSSL developers said about this bug: “Good catch! Fortunately this only affects the dev branch and not 1.1.1 (otherwise this would have been a CVE).” We mention in passing that the fix for this issue triggered a general discussion among developers about revisiting how OpenSSL 3.0.0 manages transcript hashes and that its state machine should be redesigned.

TABLE II: Results using the OpenSSL 3.0.0-alpha12 server instance as SUT.

Requirement	Bug	Paths	Time	Participating Records
Record Version	✗	4	1m19s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i> , <i>CCS</i>
Record Length		492	1h56m01s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i> , <i>CCS</i>
Epoch Number		10	2m02s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i> , <i>CCS</i>
Record Sequence Number		74	21m05s	<i>CH2</i> , <i>CKE</i> , <i>CCS</i>
Handshake Type	✗	6	1m10s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i>
Message Sequence		8	1m18s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i>
Fragment Reassembly		51	4m46s	<i>CKE</i>
Unfragmented Message Offset		3	59s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i>
Unfragmented Message Length	?	5579	⊙	<i>CH0</i> , <i>CH2</i> , <i>CKE</i>
Message Sequence in Fragments		16	1m56s	<i>CKE</i>
Handshake Version	✗	8	1h18m36s	<i>CH0</i> , <i>CH2</i>
Fragment Length		272	31m43s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i>
Cookie Length		182	40m13s	<i>CH0</i> , <i>CH2</i>
Session ID Length		646	1h13m20s	<i>CH0</i> , <i>CH2</i>
CH0/HVR Message Sequence		3	1m04s	<i>HVR</i>
CH2/SH Message Sequence		4	1m13s	<i>SH</i>

TABLE III: Results using OpenSSL 3.0.0-alpha12 client instance as SUT.

Requirement	Bug	Paths	Time	Participating Records
Record Version	✗	5	1m47s	<i>HVR</i> , <i>SH</i> , <i>SHD</i> , <i>CCS</i>
Record Length		259	29m18s	<i>HVR</i> , <i>SH</i> , <i>SHD</i> , <i>CCS</i>
Epoch Number		10	2m28s	<i>HVR</i> , <i>SH</i> , <i>SHD</i> , <i>CCS</i>
Record Sequence Number		210	1h23m41s	<i>HVR</i> , <i>SH</i> , <i>SHD</i> , <i>CCS</i>
Handshake Type	✗	18	1m58s	<i>HVR</i> , <i>SH</i> , <i>SHD</i>
Message Sequence		19	1m58s	<i>HVR</i> , <i>SH</i> , <i>SHD</i>
Fragment Reassembly		1076	2h22m21s	<i>SH</i>
Unfragmented Message Offset		3	1m03s	<i>HVR</i> , <i>SH</i> , <i>SHD</i>
Unfragmented Message Length	?	5499	⊙	<i>HVR</i> , <i>SH</i> , <i>SHD</i>
Message Sequence in Fragments		23	2m26s	<i>SH</i>
Handshake Version	✗	6	43m49s	<i>HVR</i> , <i>SH</i>
Fragment Length		112	10m28s	<i>HVR</i> , <i>SH</i> , <i>SHD</i>
Cookie Length		21	6m34s	<i>HVR</i>
Session ID Length		3	1m00s	<i>SH</i>

This provides some evidence that the bugs exposed by our methodology are often quite deep.

Besides this security bug, we also detected two non-conformance bugs. The Record Version and the Handshake Version requirements (§III-A1 and §III-B7) of the RFC revealed that OpenSSL did not properly check for the version fields when the underlined records in Tables II and III were processed.

Let us also describe the information in the tables of this and the next subsection. Each of their rows shows the requirement which is checked, whether it revealed a vulnerability (✗), a non-conformance bug (✗) or was inconclusive (?) due to timeout (⊙), the number of different paths that KLEE explored and the time this required, and the shorthand for the participating records of the DTLS protocol that the requirement involves. Records that expose the bug(s) that were detected are shown in red color (for vulnerabilities) or underlined (for non-conformances). Regarding the time that symbolic execution requires, we notice that most requirements are checked quite fast (in a few minutes when testing OpenSSL 3.0.0-alpha12), but there also exist requirements where KLEE needs to examine a significant number of paths of the SUT and the tests take more than one to two hours to complete or even time out after running for a day. We also note that the time that checking the requirements requires is *not* proportional to the number of execution paths that KLEE explores; for example, notice the Handshake Version rows in Tables II and III and contrast them with other rows in these two tables. The Handshake Version experiment takes

considerably longer despite involving a relatively small number of paths. This is due to the fact that unlike other tested implementations, OpenSSL by default, computes the master secret using a hash over the contents of prior messages. These contents include symbolic fields (`client_version` in *CH0* and *CH2*), causing KLEE to symbolically execute complicated hash code. On a general note, the experiments on OpenSSL 3.0.0-alpha12 were by far the most time-consuming ones among the DTLS implementations we tested.

B. Bugs in TinyDTLS

On the two TinyDTLS^E branches we tested, we detected a total of seventeen bugs. We omit the details of the ten non-conformance bugs, and describe only the three vulnerabilities and the four “Other” errors (cf. Table I).

The vulnerabilities we detected were reported as issues 59, 69, and 74. In short, they are as follows: (1) When a client receives a *ServerHello* (*SH*) where the `length` field has a value smaller than the actual size of the record, the variable that represents the record size wraps around. Later this variable is used to read the extensions contained in the *SH*, causing the client to crash. (2) When a malformed *ClientHello0* with `length = 0` is sent to the server, a size variable used when the server is generating the cookie wraps around. This causes the server to try to compute an HMAC over a large portion of memory using the size variable, which in turn crashes the server. (3) If a *ClientHello2* message has an invalid value for its version field, the server fails to retrieve the cookie from the message. A flaw in the handling of this failure, in turn, causes the server to proceed with the handshake. This makes TinyDTLS servers susceptible to Denial-of-Service attacks.

The four “Other” bugs are also quite serious, but we do not classify them as vulnerabilities because their consequences are unclear; e.g., they do not cause crashes. They have been reported as issues 54, 55, 57 and 70. In short, we have detected: (1) A memory over-read when the client or the server is reassembling a fragmented message, occurring if the `fragment_length` is smaller than the size of the actual fragment. (2) A client accessing an invalid memory past the boundary of the fragment when the server sends a fragmented *ServerHello* to the client where some of the handshake fields are not present in the fragment. (3) An over-shift taking place when a client or a server receives a record with a `sequence_number` greater than 32. (4) Another memory over-read when a *HelloVerifyRequest* contains a 16-byte cookie, but `cookie_length` is greater than 16.

It is interesting to point out that, for some of these bugs, it took up to three pull request attempts before the root of the problem was identified and fixed. Each time, we were able to trigger the bug in another way and show to the developers that their fixes were insufficient. The fact that we had an automatic way to test the version of TinyDTLS^E with the proposed fix was very handy. Finally, note that testing TinyDTLS is very fast for all but one requirement (Record Length); cf. the times in Tables IV and V.

TABLE IV: Results using TinyDTLS^E server instance as SUT.

Requirement	Bug	Paths	Time	Participating Records
Record Version		15	6s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i> , <i>CCS</i>
Record Length		33	19s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i> , <i>CCS</i>
Epoch Number	✗	11	4s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i> , <i>CCS</i>
Record Sequence Number	✗✗	12	3s	<i>CH2</i> , <i>CKE</i> , <i>CCS</i>
Handshake Type		35	14s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i>
Message Sequence	✗	11	11s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i>
Fragment Reassembly	✗	300	1m05s	<i>CKE</i>
Unfragmented Message Offset	✗	1	5s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i>
Unfragmented Message Length	✗✗	7	7s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i>
Message Sequence in Fragments	✗	4	2s	<i>CKE</i>
Handshake Version	✗	2	12s	<i>CH0</i> , <i>CH2</i>
Fragment Length	✗✗	7	4s	<i>CH0</i> , <i>CH2</i> , <i>CKE</i>
Cookie Length	✗	6	8s	<i>CH0</i> , <i>CH2</i>
Session ID Length	✗	32	16s	<i>CH0</i> , <i>CH2</i>
CH0/HVR Message Sequence		2	7s	<i>HVR</i>
CH2/SH Message Sequence	✗	3	9s	<i>SH</i>

TABLE V: Results using TinyDTLS^E client instance as SUT.

Requirement	Bug	Paths	Time	Participating Records
Record Version		65	34s	<i>HVR</i> , <i>SH</i> , <i>SHD</i> , <i>CCS</i>
Record Length	✗	112	2h06m13s	<i>HVR</i> , <i>SH</i> , <i>SHD</i> , <i>CCS</i>
Epoch Number		21	6s	<i>HVR</i> , <i>SH</i> , <i>SHD</i> , <i>CCS</i>
Record Sequence Number	✗✗	146	58s	<i>HVR</i> , <i>SH</i> , <i>SHD</i> , <i>CCS</i>
Handshake Type	✗	256	1m36s	<i>HVR</i> , <i>SH</i> , <i>SHD</i>
Message Sequence	✗	23	15s	<i>HVR</i> , <i>SH</i> , <i>SHD</i>
Fragment Reassembly	✗	726	2m55s	<i>SH</i>
Unfragmented Message Offset	✗	1	2s	<i>HVR</i> , <i>SH</i> , <i>SHD</i>
Unfragmented Message Length	✗	1	2s	<i>HVR</i> , <i>SH</i> , <i>SHD</i>
Message Sequence in Fragments	✗	4	2s	<i>SH</i>
Handshake Version	✗	2	3s	<i>HVR</i> , <i>SH</i>
Fragment Length	✗	1	2s	<i>HVR</i> , <i>SH</i> , <i>SHD</i>
Cookie Length	✗	18	19s	<i>HVR</i>
Session ID Length	✗	4	7s	<i>SH</i>

VI. RELATED WORK

Symbolic execution [19] was introduced already in the 70’s. In the last 15 years, improvements in tool implementations [7], [12] have made symbolic execution a powerful software testing technique. For testing network protocols, several works (e.g., KleeNet [29], SymNet [28], etc.), search for issues arising during the joint execution of several protocol parties combined with a test environment. Symbolic execution is employed to explore as many code paths as possible within some time budget. Since each input is generated by another party, these approaches mainly detect interoperability issues, and have limited power to detect flaws in processing of adversarial (non-valid) inputs.

SYMBEXNET [32] jointly executes the protocol parties symbolically to generate test inputs that explore as many code paths as possible in a given time budget. The generated test inputs are then replayed against the SUT, while monitoring its behavior to check whether any requirement extracted from the specification is violated. In contrast to our approach, SYMBEXNET symbolically executes the SUT without first augmenting it with assumptions or assertions that check for requirement violations. As a consequence, it may miss violations corresponding to paths which are exercised by both violating and non-violating inputs. To understand why, consider the Handshake Version requirement (§III-B7), which constrains the `client_version` field to a value in the set {DTLS1.0, DTLS1.2}. MbedTLS checks this requirement incorrectly;

it checks whether `client_version` is in a range which also includes the invalid version `DTLS1.1`. As a result, the same code path can be exercised by both valid and invalid version fields. For such a path, symbolic execution without inserted assumptions will generate a single test case which may or may not expose the bug (i.e. it may or may not have `client_version` set to `DTLS1.1`). In contrast, the assumptions used in our approach specifically address this, guiding symbolic execution to generate test cases that expose violations whenever possible. Similar examples include the missing checks for Message Offset/Length (§III-B4), Fragment Length, and Epoch (§III-A3) in TinyDTLS. To validate whether symbolic execution unguided by assumptions as used in SYMBEXNET could indeed miss such bugs, we symbolically executed MbedTLS and TinyDTLS without any assumptions or assertions. In all cases, none of the generated test inputs exposed the corresponding requirement violation.

Pedrosa *et al.* [24] use symbolic execution to search for interoperability issues by characterizing messages that during a session can be sent by one party but rejected as non-compliant by the other. Wen *et al.* [37] apply symbolic execution to stateful protocol implementations by first employing model learning (L^*) to extract a finite state machine, whose states can be considered as an additional input in symbolic execution.

Chau *et al.* [9], [10] use symbolic execution for a form of differential testing of libraries that classify certificate chains as valid or invalid. Using symbolic execution, path constraints for the “valid” and “invalid” outcomes are generated. Path constraints from different implementations are compared, and discrepancies are investigated. This approach is suitable for libraries that implement an input-output function for which requirements are difficult to formalize precisely. In our approach, we can formalize and check each requirement individually: violations can be detected and diagnosed directly when they occur, and there is no need to compare potentially complex test data from different implementations.

Chen *et al.* [11] automate the extraction of requirements from the X.509 RFC [4] by extracting sentences with keywords, such as “MUST”, “SHOULD”, etc. From each requirement, “valid” and “invalid” field values are constructed, with manual assistance. These are combined into a compact test suite, which is then used in differential black-box testing. As we have noted in §II, in the case of DTLS, some requirements are clarified and/or corrected in errata, which are documents distinct from the protocol’s RFC. It is unclear how requirement extraction can be automated in such a setting.

There is a rich body of work on protocol conformance testing, with model-based testing (MBT) [6], [35] and property-based testing (PBT) [1], [20], [21] as closely related approaches. An abstract model of the protocol is manually constructed or learned [33], and is used as basis for generating test inputs that are supplied, typically in black-box testing. Many different formalisms for expressing abstract models and specifications have been suggested [22], [36]. PBT is often simpler to perform than MBT, due to the high-level infrastructure and built-in mechanisms for input generation that PBT tools provide. Our

approach is white-box, which gives more power to the search for specific inputs and code paths that expose bugs.

State fuzzing is a black-box technique for detecting flaws in the control logic for handling the order of packets. Such flaws may be exploited, e.g., by tricking an implementation to bypass an authentication step. State fuzzing has discovered several flaws in TLS and DTLS implementations [3], [13], [16], [31]. It can be regarded as complementary to our approach in that it tests different orderings of packet types, whereas we search for requirement violations and bugs in handling packet fields under a specific ordering.

VII. REFLECTIONS AND CONCLUDING REMARKS

In this paper, we have described a test methodology and presented our experiences from using symbolic execution to detect specification violations and security vulnerabilities in implementations of the DTLS network protocol. A central idea in our methodology is to formulate requirements over the processing of packets in a session and let these requirements guide the symbolic execution to search for code paths and corresponding sequences of inputs that expose requirement violations. This allows testing the SUT with a broad range of input sequences, both benign and adversarial.

When testing a code base of significant size and complexity, a challenge for all techniques based on symbolic execution, and to those using KLEE in particular, is scalability. Our experience is that one effective way to achieve scalability is to test for violation of only one requirement at a time, and to make symbolic only the parts of the input that are relevant for the tested requirement. Naturally, this requires attentiveness, and has the drawback that it can miss inputs that can be produced only when multiple requirements are in effect.

If the speed of symbolic execution is slow due to some easily identifiable reason (e.g., the execution of cryptographic functions), one can try to find a way to keep away from such trouble when the requirement to check does not depend on the functionality of the corresponding code. This is what we described in §IV-B: we focused on handshakes with a pre-shared key (PSK) instead of testing the DTLS implementations using handshakes based on more complicated encryption algorithms. Of course, a downside is that testing will not detect bugs in the code which is not executed; it will only check violations of specification requirements.

When testing several implementations of the same protocol, significant effort can be saved by packaging as much reusable functionality as possible into a shared library, as we describe in §IV-C. By making the different implementations take input with a common structure, as we did for DTLS records, many operations on input data, such as making parts of input symbolic and adding assumptions corresponding to requirements, can be implemented in such a shared library.

We have evaluated our methodology by checking four widely-used DTLS implementations against the RFC for DTLS. We were able to quickly expose a known CVE in an older version of OpenSSL, and discover numerous new vulnerabilities, non-conformance issues and subtle errors in all implementations.

REFERENCES

- [1] T. Arts, J. Hughes, J. Johansson, and U. Wiger, "Testing telecoms software with Quiviq QuickCheck," in *Proceedings of the 2006 ACM SIGPLAN Workshop on Erlang*. New York, NY, USA: ACM, 2006, pp. 2–10. [Online]. Available: <http://doi.acm.org/10.1145/1159789.1159792>
- [2] H. Asadian, P. Fiterău-Broștean, B. Jonsson, and K. Sagonas, "Replication material for the ICST 2022 paper: Applying symbolic execution to test implementations of a network protocol against its specification," Apr. 2022. [Online]. Available: <https://doi.org/10.5281/zenodo.5929867>
- [3] B. Beurdouche, K. Bhargavan, A. Delignat-Lavaud, C. Fournet, M. Kohlweiss, A. Pironti, P.-Y. Strub, and J. K. Zinzindohoue, "A messy state of the union: Taming the composite state machines of TLS," *Commun. ACM*, vol. 60, no. 2, pp. 99–107, Feb. 2017. [Online]. Available: <https://doi.org/10.1145/3023357>
- [4] S. Boeyen, S. Santesson, T. Polk, R. Housley, S. Farrell, and D. Cooper, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," RFC 5280, May 2008. [Online]. Available: <https://rfc-editor.org/rfc/rfc5280.txt>
- [5] S. Bradner, "Key words for use in RFCs to Indicate Requirement Levels," RFC 2119, Mar. 1997. [Online]. Available: <https://www.rfc-editor.org/info/rfc2119>
- [6] M. Broy, B. Jonsson, J. Katoen, M. Leucker, and A. Pretschner, Eds., *Model-Based Testing of Reactive Systems, Advanced Lectures*, ser. LNCS, vol. 3472. Springer, 2005. [Online]. Available: <https://doi.org/10.1007/b137241>
- [7] C. Cadar, D. Dunbar, and D. Engler, "KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI '08. Berkeley, CA, USA: USENIX Association, 2008, pp. 209–224. [Online]. Available: <http://dl.acm.org/citation.cfm?id=1855741.1855756>
- [8] M. M. Carvalho, J. DeMott, R. Ford, and D. A. Wheeler, "Heartbleed 101," *IEEE Secur. Priv.*, vol. 12, no. 4, pp. 63–67, 2014. [Online]. Available: <https://doi.org/10.1109/MSP.2014.66>
- [9] S. Y. Chau, O. Chowdhury, M. E. Hoque, H. Ge, A. Kate, C. Nita-Rotaru, and N. Li, "SymCerts: Practical symbolic execution for exposing noncompliance in X.509 certificate validation implementations," in *IEEE Symposium on Security and Privacy*, ser. SP 2017. IEEE Computer Society, May 2017, pp. 503–520. [Online]. Available: <https://doi.org/10.1109/SP.2017.40>
- [10] S. Y. Chau, M. Yahyazadeh, O. Chowdhury, A. Kate, and N. Li, "Analyzing semantic correctness with symbolic execution: A case study on PKCS#1 v1.5 signature verification," in *26th Annual Network and Distributed System Security Symposium*, ser. NDSS 2019. The Internet Society, Feb. 2019. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/analyzing-semantic-correctness-with-symbolic-execution-a-case-study-on-pkcs1-v1-5-signature-verification/>
- [11] C. Chen, C. Tian, Z. Duan, and L. Zhao, "RFC-directed differential testing of certificate validation in SSL/TLS implementations," in *Proceedings of the 40th International Conference on Software Engineering*, ser. ICSE 2018, M. Chaudron, I. Crnkovic, M. Chechik, and M. Harman, Eds. ACM, May–Jun. 2018, pp. 859–870. [Online]. Available: <https://doi.org/10.1145/3180155.3180226>
- [12] V. Chipounov, V. Kuznetsov, and G. Candea, "The S2E platform: Design, implementation, and applications," *ACM Trans. Comput. Syst.*, vol. 30, no. 1, pp. 2:1–2:49, 2012. [Online]. Available: <https://doi.org/10.1145/2110356.2110358>
- [13] J. de Ruiter and E. Poll, "Protocol state fuzzing of TLS implementations," in *24th USENIX Security Symposium*. USENIX Association, Aug. 2015, pp. 193–206. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/de-ruiter>
- [14] T. Dierks and E. Rescorla, "The transport layer security (TLS) protocol version 1.1," RFC 4346, Internet Engineering Task Force, Apr. 2006. [Online]. Available: <http://www.ietf.org/rfc/rfc4346.txt>
- [15] —, "The transport layer security TLS protocol version 1.2," RFC 5246, Aug. 2008. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc5246.txt>
- [16] P. Fiterău-Broștean, B. Jonsson, R. Merget, J. de Ruiter, K. Sagonas, and J. Somorovsky, "Analysis of DTLS implementations using protocol state fuzzing," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 2523–2540. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/fiterau-brosteau>
- [17] P. Fiterău-Broștean, B. Jonsson, K. Sagonas, and F. Tâquist, "DTLS-Fuzzer: A DTLS protocol state fuzzer," in *15th IEEE International Conference on Software Testing, Verification and Validation*, ser. ICST 2022. IEEE Computer Society, Apr. 2022.
- [18] F. Foerg, "Fuzzing the Mbed TLS library," Sep. 2015. [Online]. Available: <https://blog.gdssecurity.com/labs/2015/9/21/fuzzing-the-mbedtls-library.html>
- [19] J. C. King, "Symbolic execution and program testing," *Commun. ACM*, vol. 19, no. 7, pp. 385–394, Jul. 1976. [Online]. Available: <https://doi.org/10.1145/360248.360252>
- [20] A. Löschner and K. Sagonas, "Targeted property-based testing," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2017. New York, NY, USA: ACM, 2017, pp. 46–56. [Online]. Available: <http://doi.acm.org/10.1145/3092703.3092711>
- [21] A. Löschner, K. Sagonas, and T. Voigt, "Property-based testing of sensor networks," in *Sensing, Communication, and Networking, 12th Annual IEEE International Conference on*, ser. SECON '15. IEEE, Jun. 2015, pp. 100–108. [Online]. Available: <https://doi.org/10.1109/SAHCN.2015.7338296>
- [22] K. L. McMillan and L. D. Zuck, "Formal specification and testing of QUIC," in *Proceedings of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM 2019. ACM, Sep. 2019, pp. 227–240. [Online]. Available: <https://doi.org/10.1145/3341302.3342087>
- [23] B. Möller, T. Duong, and K. Kotowicz, "This POODLE bites: exploiting the SSL 3.0 fallback," 2014. [Online]. Available: <https://www.openssl.org/bodo/ssl-poodle.pdf>
- [24] L. Pedrosa, A. Fogel, N. Kothari, R. Govindan, R. Mahajan, and T. D. Millstein, "Analyzing protocol implementations for interoperability," in *12th USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI 15. USENIX Association, May 2015, pp. 485–498. [Online]. Available: <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/pedrosa>
- [25] V.-T. Pham, M. Böhme, and A. Roychoudhury, "AFLNET: A greybox fuzzer for network protocols," in *IEEE 13th International Conference on Software Testing, Validation and Verification*, ser. ICST 2020. IEEE, Oct. 2020, pp. 460–465. [Online]. Available: <https://ieeexplore.ieee.org/document/9159093>
- [26] E. Rescorla and N. Modadugu, "Datagram transport layer security," RFC 4347, Internet Engineering Task Force, Apr. 2006. [Online]. Available: <http://www.ietf.org/rfc/rfc4347.txt>
- [27] —, "Datagram transport layer security version 1.2," RFC 6347, pp. 1–32, Jan. 2012. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc6347.txt>
- [28] R. Sasnauskas, P. Kaiser, R. L. Jukic, and K. Wehrle, "Integration testing of protocol implementations using symbolic distributed execution," in *20th IEEE International Conference on Network Protocols*, ser. ICNP 2012. IEEE Computer Society, Oct.–Nov. 2012, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/ICNP.2012.6459940>
- [29] R. Sasnauskas, O. Landsiedel, M. H. Alizai, C. Weise, S. Kowalewski, and K. Wehrle, "KleeNet: discovering insidious interaction bugs in wireless sensor networks before deployment," in *Proceedings of the 9th International Conference on Information Processing in Sensor Networks*, ser. IPSN 2010, T. F. Abdelzaher, T. Voigt, and A. Wolisz, Eds. ACM, Apr. 2010, pp. 186–196. [Online]. Available: <https://doi.org/10.1145/1791212.1791235>
- [30] Z. Shelby, K. Hartke, and C. Bormann, "The constrained application protocol (CoAP)," RFC 7252, Jun. 2014. [Online]. Available: <https://rfc-editor.org/rfc/rfc7252.txt>
- [31] J. Somorovsky, "Systematic fuzzing and testing of TLS libraries," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '16. New York, NY, USA: ACM, 2016, pp. 1492–1504. [Online]. Available: <http://doi.acm.org/10.1145/2976749.2978411>
- [32] J. Song, C. Cadar, and P. R. Pietzuch, "SYMBEXNET: Testing network protocol implementations with symbolic execution and rule-based specifications," *IEEE Trans. Software Eng.*, vol. 40, no. 7, pp. 695–709, 2014. [Online]. Available: <https://doi.org/10.1109/TSE.2014.2323977>
- [33] M. Tappler, B. K. Aichernig, and R. Bloem, "Model-based testing IoT communication via active automata learning," in *Software Testing, Verification and Validation, (ICST) 2017 IEEE International Conference on*. IEEE Computer Society, Mar. 2017, pp. 276–287. [Online]. Available: <https://doi.org/10.1109/ICST.2017.32>

- [34] "TLS-Attacker," <https://github.com/tls-attacker/TLS-Attacker>, Online; accessed 12-December-2021.
- [35] M. Utting and B. Legeard, *Practical Model-Based Testing - A Tools Approach*, 1st ed. Morgan Kaufmann, Nov. 2006. [Online]. Available: <http://www.elsevierdirect.com/product.jsp?isbn=9780123725011>
- [36] M. Veanes, C. Campbell, W. Grieskamp, W. Schulte, N. Tillmann, and L. Nachmanson, "Model-based testing of object-oriented reactive systems with Spec Explorer," in *Formal Methods and Testing, An Outcome of the FORTEST Network, Revised Selected Papers*, ser. LNCS, R. M. Hierons, J. P. Bowen, and M. Harman, Eds., vol. 4949. Springer, 2008, pp. 39–76. [Online]. Available: https://doi.org/10.1007/978-3-540-78917-8_2
- [37] S. Wen, Q. Meng, C. Feng, and C. Tang, "A model-guided symbolic execution approach for network protocol implementations and vulnerability detection," *PloS one*, vol. 12, no. 11, p. e0188229, 2017. [Online]. Available: <https://doi.org/10.1371/journal.pone.0188229>
- [38] M. Yason, "CVE-2014-0195: Adventures in OpenSSL's DTLS fragmented land," Dec. 2014. [Online]. Available: <https://securityintelligence.com/cve-2014-0195-adventures-in-openssls-dtls-fragmented-land/>
- [39] M. Zalewski, "American fuzzy lop," <http://lcamtuf.coredump.cx/afl/>, 2013.

Paper II



Monitor-based Testing of Network Protocol Implementations Using Symbolic Execution

Hooman Asadian
hooman.asadian@it.uu.se
Uppsala University
Uppsala, Sweden

Bengt Jonsson
bengt@it.uu.se
Uppsala University
Uppsala, Sweden

Paul Fiterău-Broștean
paul.fiterau_brostean@it.uu.se
Uppsala University
Uppsala, Sweden

Konstantinos Sagonas
kostis@it.uu.se
Uppsala University
Uppsala, Sweden
National Technical University of Athens
Athens, Greece

ABSTRACT

Implementations of network protocols must conform to their specifications in order to avoid security vulnerabilities and interoperability issues. To detect errors, testing must investigate an implementation's response to a wide range of inputs, including those that could be supplied by an attacker. This can be achieved by symbolic execution, but its application in testing network protocol implementations has so far been limited. One difficulty when testing such implementations is that the inputs and requirements for processing a packet depend on the sequence of previous packets. We present a novel technique to encode protocol requirements by *monitors*, and then employ symbolic execution to detect violations of these requirements in protocol implementations. A monitor is a component external to the SUT, that observes a sequence of packets exchanged between protocol parties, maintains information about the state of the interaction, and can thereby detect requirement violations. Using monitors, requirements for stateful network protocols can be tested with a wide variety of inputs, without intrusive modifications in the source code of the SUT. We have applied our technique on the most recent versions of several widely-used DTLS and QUIC protocol implementations, and have been able to detect twenty two previously unknown bugs in them, twenty one of which have already been fixed and the remaining one has been confirmed.

CCS CONCEPTS

• **Security and privacy** → **Software security engineering**; • **Software and its engineering** → **Software defect analysis**.

KEYWORDS

Software security, security testing, network security, network protocols, symbolic execution, monitors, DTLS, QUIC

ACM Reference Format:

Hooman Asadian, Paul Fiterău-Broștean, Bengt Jonsson, and Konstantinos Sagonas. 2024. Monitor-based Testing of Network Protocol Implementations Using Symbolic Execution. In *The 19th International Conference on Availability, Reliability and Security (ARES 2024)*, July 30-August 2, 2024, Vienna, Austria. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3664476.3664521>

1 INTRODUCTION

Implementations of network protocols, such as TCP, TLS, DTLS, QUIC, etc., must conform to their specifications in order to avoid security vulnerabilities and interoperability issues. Even seemingly innocent deviations from the standard may open implementations up for security attacks. Examples include Heartbleed [10] and the TLS POODLE downgrade vulnerability [7, 22], enabled by insufficient checking of length fields or version numbers in input packets. In order to detect such errors, protocol testing techniques must be able to investigate an implementation's response to a wide range of inputs, not only inputs that are expected from protocol peers, but also inputs that could be supplied by malicious attackers.

A technique that explores an implementation's response to the full range of possible inputs is *symbolic execution (SE)* [19]. It analyzes programs for which some inputs are designated as *symbolic*, and explores the code paths that are possible for some values of these symbolic inputs, thereby implicitly considering all possible values of these inputs. In recent years, SE tools such as KLEE [8] and S²E [15] have demonstrated impressive results for testing programs, such as systems libraries, device drivers, and computer vision code, being able to find safety violations that have been difficult to expose using other techniques [9].

For implementations of network protocols and other programs that repeatedly interact with their environment, SE faces several difficulties. One of them is that most network protocols are *stateful*: to test the processing of a particular packet, the implementation must first be brought to a specific state by an appropriate packet sequence. Moreover, the requirements concerning that packet may depend on the contents of packets in the previously exchanged packet sequence. To illustrate, consider a requirement stating that if the sequence number field in any packet has the same value as the sequence number field of any previously received packet,

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

ARES 2024, July 30-August 2, 2024, Vienna, Austria

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1718-5/24/07

<https://doi.org/10.1145/3664476.3664521>

then an error packet must be output. An idea for an SE-based approach to check that a protocol implementation satisfies such a requirement for any sequence of input packets could be to devise techniques for designating the sequence number field in each input packet as symbolic (thereby implicitly considering any value in these fields), and for checking the uniqueness requirement on these fields by inserting instrumentation and assertions into the code of the SUT. Such techniques could be applied to a given SUT by appropriate modifications in its code, but such modifications need be re-applied when testing different implementations of a given protocol or different versions of an implementation.

Challenge: To thoroughly test several implementations of a protocol, we need a testing technique that can check requirements over sequences of exchanged packets, with the properties that it

- ① covers a wide range of inputs, including all potentially requirement-violating values of fields in sequences of packets,
- ② can automatically be applied to any implementation of a protocol, without requiring modification in a SUT's code.

Current state of the art: Unfortunately, existing SE techniques for testing protocol implementations do not achieve both ① and ②. Several combinations of SE and requirement checking (RC) have been presented, but they apply SE and RC separately. For instance, SymbexNet [32] tests protocol implementations by first employing SE to generate test inputs that exercise a wide range of code paths; these inputs are thereafter applied to the SUT to generate concrete executions that are subject to RC. This approach does not satisfy property ①, since its use of SE only explores code paths that can be exercised in a non-adversary environment, and furthermore considers only one test input for each such path. Consequently, the approach may miss requirement violations that are exposed by specific, possibly adversarial, inputs. Tempel et al. [34] show that SE adapted for protocols increases coverage, but they do not check requirements. Rath et al. [26] use SE to detect interoperability issues by provoking different failure scenarios, and Pedrosa et al. [24] use SE to check for mismatches in message communications. A combination of SE and RC which satisfies property ① has been presented by our group [3], however at the cost of subjecting each SUT to significant manual source code modifications (including to make it read input from a file, and injecting assertions) which are SUT-specific and not easily portable to its next version(s). Thus, we are still lacking a technique for testing stateful protocol implementations which achieves properties ① and ②.

This paper: We present a technique, based on symbolic execution, for testing network protocol implementations, which achieves both properties. The technique operates in a test environment which includes the parties of the protocol, and applies symbolic execution to check requirements against a protocol's implementation. Rather than relying on intrusive code modification in the SUT for checking requirements, our technique is based on concentrating the logic for checking each requirement in a software component which is external to the SUT; for this, we have developed a novel adaptation of the concept of *monitors* (e.g., [31]) for symbolic execution. During an interaction, a monitor observes the packets exchanged between the SUT and its peers to maintain relevant up-to-date information about the current state of the interaction. Using this information, it can check whether packets sent by the SUT conform to protocol

requirements. To cover all potentially requirement-violating values in input fields, the monitor designates relevant fields in input packets as symbolic. A symbolic execution engine will then explore all code paths that can be reached for any values (including those that may induce requirement violations) of these fields. In addition, the SE engine will also detect memory errors, crashes, and other runtime problems. By being external to the SUT, monitors provide a uniform mechanism for checking requirements and detecting runtime errors, which must be implemented *only once* for each protocol, and can be automatically applied to any implementation of the protocol, *without* requiring modification in a SUT's code. In some cases, the effectiveness of our technique can be increased by small modifications in the SUT's code, e.g., to disable encryption (which is problematic for SE to handle).

We have implemented our technique by creating a generic infrastructure for SE-based testing of network protocol implementations which exchange packets over sockets. To evaluate the effectiveness of our technique and directly compare it with the state-of-the-art technique of our previous work, which is also based on requirement-driven symbolic execution [3], we have first applied it on four widely-used DTLS implementations. The technique we describe in this paper has reproduced all bugs detected by the other technique, but with less implementation effort and considerably faster. It has also detected several, previously unknown bugs, in newer versions of two of these DTLS implementations. We have also applied the technique of this paper to three implementations of the QUIC protocol, and have detected previously unknown bugs in all of them. We have reported these bugs to the developers, and the vast majority of them have already been fixed.

Contributions: In short, the contributions of this paper are:

- A new technique to test protocol implementations against their requirements using symbolic execution and monitors which are external to the SUTs and, as such, can be directly reused by different versions and implementations of a particular protocol.
- An evaluation showing the technique's bug finding effectiveness, time performance and implementation effort when compared with another state-of-the-art SE technique.
- A publicly available infrastructure [5] with test harnesses and sets of monitors for two widely used protocols (DTLS and QUIC) to repeat our experiments or extend our work with more monitors for these or other network protocols.

Overview: The next section introduces the concept of monitors and their adaptation for SE, followed by §3 that presents the steps of our technique. Section 4 overviews the two protocols that we have applied our technique on, and presents representative monitors for their requirements. We then describe our implementation (§5), and evaluate its effectiveness (§6). The paper ends with discussing threats to validity (§7), related work (§8) and some final remarks.

2 MONITORS BY EXAMPLE

In this section, we present our adaptation of the concept of *monitors* [31] for symbolic execution of protocol implementations. We first explain monitors and, thereafter, our adaptation. of monitors for symbolic execution.

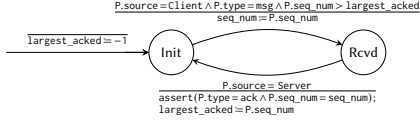


Figure 1: A monitor for Req1.

A monitor is a software component which is external to the parties of a protocol, and executes concurrently with them. It observes the sequence of packets exchanged between protocol parties and maintains information about relevant aspects of the current state of the protocol interaction; this information can be updated upon observing each new packet. Using this information, the monitor can detect whenever a packet sent by the SUT violates a protocol requirement, in which case it signals this violation.

As an illustration, consider a protocol for exchanging packets between a client and server, in which there are two types of packets: messages and acknowledgments, both of which contain a sequence number as a field. Each party strictly alternates between sending and receiving, i.e., packets are ping-ponged between the client and server. Suppose that a requirement for the protocol is that:

(Req1) When the server receives a message with a new sequence number (i.e., greater than the largest acknowledged sequence number so far), it MUST respond with an acknowledgment containing the same sequence number as the received message.

A monitor for this requirement is shown in Fig. 1. For monitors, we find it convenient to use the format of state machines, in which their local state consists of a *control state*, which ranges over one of the shown circles, and additional local variables. The monitor in Fig. 1 maintains two local variables:

- `largest_acked` contains the largest sequence number acknowledged by the server;
- `seq_num` contains the sequence number of the most recent message with a new sequence number, sent by the client; it is used to determine whether the acknowledgment by the server has the required sequence number.

The monitor observes packets sent by the server or the client. It can perform a transition whenever a packet satisfies the condition above the bar in the label of an outgoing transition. We let P denote the observed packet, and let $P.source$ denote the origin of P . When performing the transition, the monitor executes the statement below the bar. Thus, starting in state `Init`, if the monitor in Fig. 1 observes a packet from the client, which is of type `msg` and whose sequence number is greater than `largest_acked`, it stores the packet's sequence number in variable `seq_num`, and transitions to state `Rcvd`. In state `Rcvd`, when the monitor observes a packet from the server, it checks using an `assert` statement that it is of type `acknowledgment` and that its sequence number is equal to that of the last message from the client (which was stored in `seq_num`). If that is the case, the monitor updates `largest_acked` and transitions back to `Init`. If the `assert` statement fails, then the execution will abort with an error.

The monitor in Fig. 1 can be used to check the requirement during testing. Alas, our monitor checks the server's response only to messages that are actually sent by the client during a testing

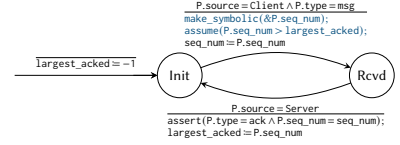


Figure 2: Monitor for Req1, adapted for symbolic execution.

session. It would be desirable to check the server's response to any sequence number for which the requirement is relevant, but generating an input message for each such sequence number is infeasible. However, we can leverage the power of symbolic execution to consider the range of all relevant sequence numbers. This is done by making selected input data *symbolic*. A symbolic execution engine will then explore all code paths that are reachable for some value of the symbolic input. In our example, we can test the server's response to any relevant value of the `seq_num` field by modifying the monitor in two ways:

- (1) Making the `seq_num` field in the message from the client symbolic.
- (2) Adding an assumption that the value in this `seq_num` field is larger than the previously largest acknowledged sequence number. This makes symbolic execution explore only code paths that are reachable on a new value of this field. In general, such assumptions can be used to restrict the considered values of symbolic fields to those for which a requirement violation is possible.

The bottom transition, where the server's response is checked, need not be modified. The result is shown in Fig. 2, with the additions marked in blue. A symbolic execution engine will then explore all code paths that can be reached for a value of the `seq_num` field, which is larger than the current value of `largest_acked`. If there is such a code path at which the `assert` statement on the bottom transition can be triggered, then the symbolic execution engine will stop executing this code path, and return a concrete value for the `seq_num` field which induces the assertion failure.

The monitor in Fig. 2 checks that packets from the server satisfy a constraint that depends only on the preceding packet. However, such constraints may also depend on several preceding packets. As an illustration, consider the following requirement.

(Req2) For each received packet of type msg, the server MUST verify that its sequence number does not duplicate the sequence number of any other packet of type msg received during the life of this session. The server MUST discard packets with duplicate sequence numbers and may generate an Alert packet.

A monitor for this requirement is shown in Fig. 3. The monitor maintains the local variable `S`, which contains the sequence numbers of all packets sent by the client so far. Starting in state `Init` with `S` initialized to the empty set, when the monitor observes an incoming packet of type `msg` from the client, it can either (i) take the self-loop transition and add its sequence number to the set `S`, or (ii) take the transition to `Rcvd` and add the assumption that its sequence number is a duplicate (i.e., included in `S`), and thereafter check by an `assert` statement that the next packet from the server is of type `Alert`. If there is a code path for which the server responds

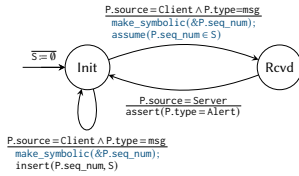


Figure 3: Monitor for Req2, adapted for symbolic execution.

with a packet other than an Alert, the assertion on the bottom transition is triggered. In this monitor, the choice between the two outgoing transitions from state `Init` is nondeterministic. This allows the monitor to make the transition to `Rcvd` on any packet from the client, assume it to have a duplicate sequence number, and check the server's response. The overall effect is to consider, given a sequence of packets from the client, all sequence number fields that exhibit a duplicate sequence number.

3 METHODOLOGY

Having introduced the concept of monitors, in this section we present a general methodology for using a set of such monitors to detect requirement violations in protocol implementations. Its steps are:

- creating a test harness to execute the SUT;
- encoding monitors as programs;
- invoking monitors at appropriate moments during SE;
- (optional) developing a protocol-specific parser/serializer;
- (optional) applying small modifications to the SUT.

We note that some of these steps are somewhat constrained by the SE engine which is employed. In §5, we will describe our implementation and infrastructure we have built for the two protocols to which we have applied our methodology.

3.1 Creating a Test Harness

The first step is to construct a test harness in which the protocol parties execute, and which invokes a monitor whenever a packet is received by one of the parties. Such a test harness will then be executed symbolically. To minimize the effort spent, our methodology advocates creating a single uniform harness that can be equipped with an arbitrary monitor which tests one or several protocol requirements regardless of which protocol party they refer to. Depending on the monitor, the protocol interaction that the test harness performs ranges from a simple message exchange (in stateless protocols) to the parties completing a handshake and then exchanging some application data. When using KLEE as SE engine, the harness must consolidate all involved protocol parties in a single-threaded (Unix) process, and needs to coordinate interactions between them. Our experience is that this is possible for all protocol implementations we have tried so far, since protocol implementations (i.e., our SUTs) often come in the form of libraries that offer an API. Harnesses can utilize such an API to coordinate an interaction between protocol parties (e.g., between a client and a server) by letting parties take turns; in each turn a party processes the packets that have been sent to it so far, thereby producing packets for the other parties.

3.2 Encoding Monitors as Programs

Once specified, perhaps using a graphical notation similar to that of §2, the set of monitors needs to be translated into code. When using KLEE as SE engine, one is constrained to use a programming language that can be compiled to LLVM bytecode. Our implementation uses C. The translation of a monitor's state machine is roughly as follows:

- Control states are encoded using an enumeration (`enum`).
- The state of the state machine is kept in a data structure that records its current state and any associated local variables.
- State machine transitions are implemented using conditionals (`if-else` statements). In these `if-elses`, the conditions above a bar specify the condition to check, while the statements below the bar are the actions to perform before moving on to the target state of the transition.
- Whenever a choice between outgoing transitions is nondeterministic, we introduce a symbolic variable whose value ranges over the available alternatives, and determines which one is chosen. On encountering such a variable, the SE engine forks, effectively covering all possible outcomes of nondeterministic choices in the monitor.

3.3 Invoking Monitors

During testing of a protocol requirement, the SE engine must be able to invoke the monitor at appropriate points during the interaction that the harness involves. Since the monitor observes and possibly manipulates the packets exchanged during this interaction, it should be invoked when a packet is received and before it is processed by a protocol party. This can be realized by implementing the monitor as a callback function, which is called just before delivering the packet to the protocol party, with the data in the received packet passed as a parameter.

3.4 Developing Protocol-specific Infrastructure

In many protocol implementations, packets are exchanged as arrays of bytes. When a monitor is invoked immediately after a packet is received by a protocol entity, it perceives the packet as an array of bytes. Encoding monitors as programs becomes significantly easier if monitors receive packets as parsed objects instead. This can be done by implementing a parser for the protocol that the SUT implements (or reusing such a parser if it exists). This parser can be invoked just prior to calling the monitor. In this setup, instead of directly invoking the callback for the monitor, the callback triggers a protocol-specific infrastructure that performs the following operations:

- The received data is parsed into a structured data object.
- The monitor is called with this object as its parameter.
- After the monitor has completed its modifications to this object, a serializer converts the object back into a byte array format and passes it for processing to the harness.

This optional step streamlines the development of monitors.

3.5 Applying Small Modifications to the SUT

For certain SUTs, it can be advantageous to make small modifications to their code in order to streamline the testing process

using symbolic execution. Specifically, when a protocol involves the transmission of packets between parties in encrypted form, one may want to disable encryption. This not only avoids the encryption/decryption overhead, but also accelerates symbolic execution by removing the need to handle cryptographic functions. Another optional adjustment involves timing-based behavior. Some protocols initiate re-transmission or prematurely terminate an interaction if no response is received within a specified time frame. In most cases, this modification can be achieved by setting the timeout value to its maximum allowable value. Another option is to completely disable timeouts.

4 MONITORS FOR DTLS AND QUIC REQUIREMENTS

We have applied our methodology to test implementations of DTLS and QUIC, two widely-used complex security protocols. This section briefly describes these protocols and, for each of them, introduces two representative monitors for their requirements.

4.1 Datagram Transport Layer Security

DTLS is a client-server protocol based on TLS that is used to secure communication over datagram transport layer protocols such as UDP. Its lower overhead and reduced latency compared to TLS makes it suitable for resource-constrained environments. Our work is concerned with DTLS version 1.2 [27]. The DTLS protocol is structured in layers. The Record Layer lies at the base. It encrypts messages received from upper layers, encapsulates them into *records*, and sends them over the network. Dually, it receives records from the network, decapsulates and decrypts them, producing messages which it delivers to the appropriate upper layer. The Handshake Layer is engaged at the start of each DTLS session, and establishes encryption keys for the Record Layer, by performing a *DTLS handshake*. A handshake involves a complex exchange of messages between the client and server. As part of this exchange, the two parties agree on a key exchange algorithm, and use this algorithm to establish session keys, which are then deployed. The Handshake Layer delivers handshake messages to the Record Layer as fragments that can fit in a datagram. Conversely, the Handshake Layer assembles handshake messages from fragments received from the Record Layer.

Let us present two requirements for the DTLS protocol and their encoding using appropriate monitors.

4.1.1 Matching Content Type. According to the TLS RFC [16, p. 19], the content type in the Record Layer “determines the higher-level protocol used to process the enclosed fragment.” The same RFC [16, p. 19] defines *ContentType* as a one-byte entity representing a set with four values, one of which is *alert*. Based on this, we can check whether the type of the content matches the enclosed fragment. In order to construct a monitor for checking that the server correctly responds to records with invalid *content_type* fields, we consult the DTLS RFC [27, p. 14], which prescribes a general rule for dealing with invalid records (e.g., records with invalid *content_type* values):

In general, invalid records SHOULD be silently discarded ...

Implementations which choose to generate an alert instead, MUST generate fatal level alerts.

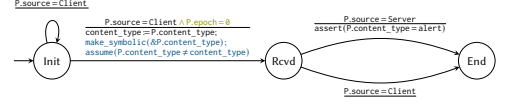


Figure 4: Monitor for the Matching Content Type requirement of DTLS.

We can now construct the monitor in Fig. 4. Let us analyze it, initially ignoring the optional conjunct colored olive. The monitor checks a server’s reaction to records whose content type mismatches their enclosed fragment. When in state *Init* and observing a client-generated record, the monitor can nondeterministically choose to either (i) take the self-loop or (ii) take the transition to state *Rcvd*, where the *content_type* field in the record is made symbolic and assumed to be different from its original concrete value. The transition to *Rcvd* in effect transforms the record sent by the client, so that its *content_type* field is invalid, i.e., inconsistent with its payload. According to the RFC, the server must either discard an invalid record and continue the interaction or respond with an *Alert* message and terminate the connection. Thus, when in state *Rcvd*, if the monitor receives a packet from the server, it checks that the packet is of type *alert* and transitions to state *End*. If, instead, the monitor gets a packet from the client, it goes directly to state *End*, because in this case the server may have silently discarded the invalid record previously sent by the client.

Now let us consider the case where we enable the optional conjunct. Doing so makes the monitor invalidate only client-generated records whose epoch field is zero. In DTLS, such records enclose unencrypted fragments, whereas records with nonzero epoch fields enclose encrypted fragments. Although the *content_type* field is not encrypted, it is used in subsequent cryptographic functions by the server whenever epoch is nonzero. If we do not want to expose symbolic execution to cryptographic functions, we can choose to check this requirement only for records whose epoch field is zero.

4.1.2 Invalid Epoch. Records are equipped with the integer-typed field *epoch* which uniquely identifies the keys used to encrypt the record fragment. The DTLS RFC [27, p. 9] imposes the following requirement on epoch:

The epoch number is initially zero and is incremented each time a ChangeCipherSpec message is sent.

Based on this requirement and the rule for dealing with invalid records, Fig. 5 shows a monitor which checks that the server appropriately responds to records with invalid epoch values, by either discarding them or by generating an *Alert* to report an error. This monitor uses the variable *counter* to keep track of the number of *ChangeCipherSpec* messages received from the client. In the initial

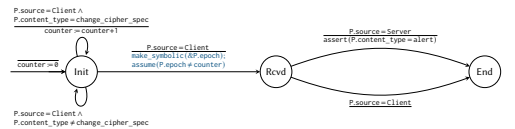


Figure 5: Monitor for the Invalid Epoch requirement of DTLS.

state, upon observing a client record, the monitor can either (i) take the self-loop transition enabled by the record type (indicated by $P_content_type$) and update counter accordingly, or (ii) take the transition to state $Rcvd$, making the epoch field symbolic and assuming its value to be invalid. Once in state $Rcvd$, the monitor checks that if the next record is received from the server, its content type is alert.

4.2 QUIC

QUIC [18] is a client-server transport layer protocol that provides secure and reliable stream-based communication. Operating over UDP, QUIC supports many of the services also provided by TCP, TLS and HTTP, but does so in a single protocol that is optimized to reduce latency. QUIC parties communicate by exchanging *QUIC packets*, with each packet containing a sequence of *frames*. Like TCP, QUIC is connection-based. To establish a connection, the parties perform a handshake in which they agree on encryption keys and the application protocol to invoke once the handshake is done.

4.2.1 Unacceptable Version. QUIC packets include an integer field that indicates the protocol’s version. Through a version negotiation process, a server notifies a client that the version selected by the client is not supported. Regarding this process, the QUIC RFC [18, §6.1 and §17.2.1] requires that:

If the version selected by the client is not acceptable to the server, the server responds with a Version Negotiation (VN) packet.
A Version Negotiation packet is inherently not version specific. Upon receipt by a client, it will be identified as a VN packet based on the Version field having a value of 0.

This requirement is captured as a monitor in Fig. 6. We assume that the acceptable versions of QUIC are captured in the predicate $is_acceptable$. When in state $Init$ and observing a client-generated

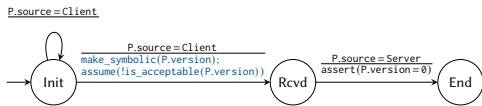


Figure 6: Monitor for the Unacceptable Version requirement of QUIC.

packet, the monitor can nondeterministically choose either to take the self-loop and do nothing, or take the transition to state $Rcvd$, during which the version field in the packet is made symbolic and assumed to be not acceptable. At this point, the only valid response from the server is a version (re)negotiation packet. While in the $Rcvd$ state, the monitor reviews if the server responded with such a packet by checking that the version field in the next packet sent by the server equals to zero. If this is not the case, the symbolic execution engine will abort this code path and generate a concrete value for the version field in the packet sent by the client.

4.2.2 Invalid New Connection ID Length. With every QUIC connection is associated a set of connection IDs that ensure that packets for QUIC connections are not delivered to the wrong endpoint due to changes in addressing at lower protocol layers. Each connection

possesses multiple connection IDs to prevent potentially malicious observers from correlating activities on multiple paths. Endpoints provide alternative connection IDs to their peers by sending frames of type $NEW_CONNECTION_ID$. Each such frame contains a length field, giving the length of the transmitted connection ID. For this field, the QUIC RFC [18, §19.15] states that:

$NEW_CONNECTION_ID$ frames contain the following fields:

Length: An 8-bit unsigned integer containing the length of the connection ID. Values less than 1 and greater than 20 are invalid and MUST be treated as a connection error of type $FRAME_ENCODING_ERROR$.

This requirement is captured as a monitor in Fig. 7. Since each packet may contain an arbitrary number of frames, and since several of these may be of type $NEW_CONNECTION_ID$, the notation $P.f$ frame for accessing a frame within a packet is not well-defined. We rather want to refer to *some* frame of type $NEW_CONNECTION_ID$ within a packet. For this purpose, transitions can be *parameterized* by a local variable, which ranges over a finite set. For this requirement,

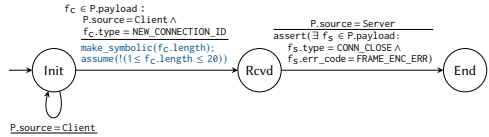


Figure 7: Monitor for the Invalid New Connection ID Length QUIC requirement.

the monitor contains a parameterized transition from state $Init$ to state $Rcvd$, parameterized by the variable f_c , which ranges over frames in the observed packet. Such a parameterized transition represents a transition for each value of the parameter. Thus, the monitor in Fig. 7 can perform a transition from $Init$ to $Rcvd$ for each frame in the payload of the current packet, which is of type $NEW_CONNECTION_ID$. When performing the transition, the monitor will make the length field of this frame symbolic and assume that its value is invalid.

The check that an invalid length field is treated as a connection error is performed by the transition from $Rcvd$. The monitor of Fig. 7 checks that the next packet from the server contains a frame of type $CONNECTION_CLOSE$ with error code $FRAME_ENCODING_ERROR$. Also here, we must take into consideration that a packet may contain several frames, and check that one of them reports the error. Such a check must be performed by a single transition, which checks all frames in the next packet from the server. This transition is labeled by an `assert` statement with an existential quantification over the frames in the payload of the observed packet.

5 IMPLEMENTATION

In this section, we describe our implementation. At its core one finds a generic infrastructure (§5.1) which suffices for all network protocols exchanging packets over POSIX sockets [33]. It mediates communication between protocol entities, and is responsible for “executing” the monitor on the packets exchanged between them. Besides the test harness, it needs to be supplied with the set of

monitors for the protocol's requirements and, optionally, a parser and a serializer (§5.2). With these components supplied and optional small changes to the SUT (§5.3), a protocol implementation can be tested using a symbolic execution engine (§5.4).

5.1 Generic Infrastructure

Many protocols exchange packets over POSIX sockets using an API provided by an OS library [33]. Our generic infrastructure is used as a proxy to this library and allows it to mediate, by redirecting API calls, the interaction between the SUT and its peers. For instance, library functions that send a packet to a socket (e.g., `sendto` in POSIX sockets) are replaced by functions that put the packet in a shared queue. Receive library functions (e.g., `recvfrom` in POSIX sockets) are replaced by functions that obtain the first packet in the shared queue, and use it to "execute" the monitor for a requirement of interest by performing the following sequence of steps:

- (1) The first packet in the queue is parsed, using a protocol-specific parser (§5.2), and copied to a data structure P .
- (2) The monitor is invoked with P given as an argument.
- (3) Once (the function implementing) the monitor returns, P is serialized back and returned to the calling entity. Note that the returned P may have some fields in the original P manipulated or made symbolic.

A separate packet queue is used for each direction of the communication (e.g., from client to server, and from server to client). The `bind` function is redirected to a function which initializes packet queues. All other functions are left unchanged.

5.2 Protocol-Specific Infrastructure

Our implementation also comprises the following protocol-specific components.

Sets of Monitors. Monitors are encoded as C functions using the translation described in §3.2. These functions take a pointer to the data structure as a parameter and access packet fields by selecting the corresponding fields in this structure. Monitors have persistent states implemented by static variables. For example, the monitor in Fig. 5 maintains two such variables, one for storing the current control state of the monitor, and the other for the counter variable.

Figure 8 contains the C code implementing the monitor for the Matching Content Type DTLS requirement for servers (Fig. 4). The code first declares an enumeration for the monitor's states and a static variable for storing the current state of the monitor (lines 1 and 3). The function implementing the monitor takes as parameter a pointer to the data structure storing the currently observed packet. Its body is quite straightforward and closely follows the pre-conditions and actions of Fig. 4. The `kleener_make_symbolic()` function is our wrapper around KLEE's `klee_make_symbolic()` that also allows for treating only *part* of a buffer as symbolic. Its first argument is a pointer to the relevant packet field, the second is its size, and the third is a name. The only aspect of the monitor of Fig. 4 which is not included in the code of Fig. 8 is the self-loop in its `init` state. This is handled by infrastructure that supports nondeterminism between two transitions, one of which is a self-loop in the initial control state. It does so by assigning to each packet P an index (i.e.,

```

1 enum STATE { INIT, RCVD, END };
2
3 static STATE curr_state = INIT;
4
5 void matching_content_type_server(RECORD *P) {
6     if (curr_state == INIT && P->source == CLIENT &&
7         byte_to_int(P->epoch, sizeof(P->epoch)) == 0) {
8         uint8_t content_type = P->content_type;
9
10        kleener_make_symbolic(&P->content_type,
11                             sizeof(P->content_type), "content-type");
12        klee_assume(P->content_type != content_type);
13        curr_state = RCVD;
14    }
15    else if (curr_state == RCVD && P->source == SERVER) {
16        if (P->content_type != Alert_REC)
17            assert(0 && "Mismatching Content Type");
18        curr_state = END;
19    }
20    else if (curr_state == RCVD && P->source == CLIENT)
21        curr_state = END;
22    else if (curr_state == END) return;
23 }

```

Figure 8: The C implementation of the monitor of Fig. 4.

the i th packet in the interaction has index i), and declaring a symbolic variable `trigger_index` ranging over possible values of this index. The infrastructure then allows the non-self-loop transition to be taken if its guard is true *and* the index of the current packet is equal to `trigger_index`, otherwise it allows the self-loop to be taken. Since `trigger_index` is symbolic, the SE engine will explore all feasible code paths in which the monitor performs an arbitrary number of self-loops followed by a non-self-loop transition; each such path is feasible for some specific value of `trigger_index`. Transitions parameterized on frames, as in the monitor of Fig. 7 are handled in analogous manner: our infrastructure implements a loop which iterates over frames in a packet, and exits when the index of the current frame equals the value of a symbolic variable.

Parser & Serializer. These two components are optional, but we have implemented them nonetheless for the two protocols we have tested. Based on the protocols' RFC, we have constructed a parser which parses protocol packets into data structures; this allows monitors to access specific packet fields (e.g., `P->epoch` in Fig. 8). Conversely, the serializer serializes into protocol packets the data structures, whose fields may have been modified or made symbolic by the monitors. Without these two components, monitors would need to directly manipulate the bytes at the appropriate field offsets within the byte array that stores the protocol packet.

5.3 Optional SUT-Specific Preparations

As mentioned in §3, for some network protocols, it may be beneficial to make small modifications to their implementations that may speed up SE. For example, if the protocol transmits packets between entities in encrypted form, we can replace the concerned cryptographic functions (e.g., encryption, decryption, hashing, etc.) by simplified ones (e.g., identity). Another optional modification is to disable timing-based behavior.

For example, in DTLS, if no response to a flight of messages is received within a certain amount of time, the flight may be retransmitted or the entire interaction may be aborted. While such

SUT modifications are optional, disabling timeouts or increasing their values simplifies monitor construction.

5.4 Testing Using Symbolic Execution

Testing is performed by symbolically executing the constructed test harness, in which the protocol parties and monitors execute (cf. §3.1). We have used KLEE [8] as symbolic execution engine. For each explored path, KLEE produces concrete values for the data that has been made symbolic by a specific monitor. Some of these values cause crashes in the SUT or assertion failures in the monitor’s code. We have constructed test cases that exhibit the bugs we have detected by substituting these values in the corresponding exchanged messages, and reported these bugs to the SUT’s developers.

6 EVALUATION

In this section, we evaluate our technique on a variety of widely-used DTLS and QUIC implementations. With this evaluation, we intend to answer the following questions:

- Q.1 Is our technique, which is based on a SUT-agnostic set of monitors, as *effective* as another state-of-the-art SE-based testing technique that requires intrusive modifications (e.g., declaring packet fields symbolic, adding assertions, etc.) to each SUT for testing the same protocol requirements?
- Q.2 If it is equally effective, does it require *less or more effort*? And is it *faster or slower*?
- Q.3 Can it detect *previously unknown bugs* in the most recent versions of the two network protocols we consider?
- Q.4 Can it do this *without any* modifications to the SUTs?

6.1 Comparison with a Related Technique

To answer Q.1 and Q.2, we compared the approach described in this paper with another technique that our group has proposed [3], which is also based on requirement-driven symbolic execution. The corresponding paper of our prior work comes with a replication package [4], which we used for the comparison of the two techniques. We implemented as monitors, in a set of new files, all the DTLS protocol requirements that were checked in that work, and applied our new technique on the same versions of the four DTLS implementations that were tested, starting from the original code base of these SUTs (i.e., *without* any modifications). The technique we describe in this paper was able to reproduce *all* the vulnerabilities and non-conformance bugs that were previously identified. Moreover, the new technique did this:

With less effort. Both approaches require having a test harness and, optionally, making modifications to the code of the SUT (e.g., to de-randomize it and disable encryption). However, in the old approach [3], the code that checks the requirements needs to be added as calls to a shared library directly in the code of the SUT, which requires understanding and modifying a complex code base. In contrast, in the approach of this paper, the requirements are encoded as monitors in a set of new files (i.e., outside the SUTs), which can be shared when testing various versions of the same protocol implementation (or even different ones), and require no code interventions. Even disregarding this particular advantage, and concentrating on the effort required to test just one SUT and only one

of its versions, the coding effort is overall smaller in our approach. For example, for OpenSSL 3.0.0-alpha12, whose code base consists of $\approx 289\,160$ lines of code (LOC), the test harness required $\approx 1\,250$ LOC for the old approach, but only ≈ 670 for the one of this paper. In addition, the code for the 16 server and 14 client DTLS protocol requirements (i.e., 30 requirements in total) comprises $\approx 1\,700$ LOC in their approach, while the same requirements need $\approx 1\,200$ LOC to be implemented as a set of monitors. (Thus, monitors require about 40 lines on average.) Observe that the total amount of code that both approaches require is relatively small compared to the code size of the (quite complex) SUT which is tested. An additional drawback of the old approach [3] is that checking different protocol configurations relies on capturing a new set of packets, whereas in the new approach this is just a matter of adjusting the desired configuration through the SUT’s API within the test harness.

Considerably faster. The old approach [3] relies on completing a protocol interaction. (For example, in the case of DTLS, establishing a handshake or definitely failing to do so.) This means that symbolic execution may need to explore many paths unnecessarily. In contrast, in the approach that this paper advocates, a monitor that detects a requirement violation can signal the error and stop symbolic execution long before it reaches code “deep” in the protocol interaction. As concrete evidence for what this means in practice, the old approach detects the violation of the “Handshake Version” requirement for the OpenSSL 3.0.0-alpha12 DTLS server after exploring eight paths in 1h18m36s (cf. [3, Table II]), while the technique of this paper detects the violation after exploring six valid plus one erroneous (i.e., a total of seven) paths in just 1m39s. The same requirement for OpenSSL 3.0.0-alpha12 DTLS clients explores 6 paths in 43m49s [3, Table III], while the approach of this paper explores $4 + 1 = 5$ paths in 1m48s. Similar differences exist for many other requirements and SUTs. For example, the violation of the “Record Length” requirement for Eclipse’s TinyDTLS (commit 7068882) requires exploring 112 paths in 2h06m13s [3, Table IV], while the approach of this paper explores just 17 paths in only 5 seconds.

6.2 Effectiveness on DTLS Implementations

To answer Q.3 for the DTLS protocol, we represented a total of 21 requirements as monitors, and checked them in a scenario where the test harness completes an interaction between the client and server and then exchanges some application data. As SUTs we used four widely-used DTLS implementations (MbedTLS 3.1.0, OpenSSL 3.0.2, Eclipse’s TinyDTLS at commit e4d547ce, and WolfSSL 5.3.0). For all SUTs, the versions or commit points we used were the latest at the time when our experiments were conducted. Our technique did not detect any bugs in these versions of MbedTLS and OpenSSL. However, it detected a total of thirteen previously unknown bugs in TinyDTLS and WolfSSL, which we describe below. We mention that all the bugs we report in this paper for DTLS implementations have been communicated to their developers and have *all been fixed by now*; more information about them and what changes their fixes required can be found by following the hyperlinks in the last column of Table 1. We analyze them below.

In Eclipse’s TinyDTLS implementation, the monitor-based technique of this paper detected five previously unknown bugs: (1) If a

Table 1: The discovered, previously unknown, bugs in the most recent versions of two DTLS and three QUIC implementations. The entries in the “Status” column contain links to GitHub issues that were used for reporting and discussing the bugs.

SUT	Role(s)	Short Description of the Bug(s)	Status
TinyDTLS	Client	Out-of-bounds pointers when a client receives a <i>CertificateRequest</i> with an invalid value for the <code>certificate_types_count</code> field.	Fixed
	Client	Out-of-bounds pointer when a client receives a <i>CertificateRequest</i> with an invalid value for the <code>signature_hash_algorithms_length</code> field.	Fixed
	Client&Server	Out-of-bounds pointers caused by a handshake message other than <i>ClientHello</i> whose <code>msg_type</code> field has been set to <i>ClientHello</i> .	Fixed
	Client	Assertion failure when a client expects a <i>ServerHello</i> message but instead receives a <i>ServerKeyExchange</i> .	Fixed
	Client&Server	Out-of-bounds pointers caused by a <i>ChangeCipherSpec</i> message whose <code>content_type</code> field has been set to <i>Alert</i> .	Fixed
WolfSSL	Client&Server	Four (4) non-conformance bugs due to insufficient checking of length-related fields.	All Fixed
	Server	Non-conformance bug when the message sequence numbers (SNs) in the initial <i>ClientHello</i> and <i>HelloVerifyRequest</i> differ.	Fixed
	Server	Non-conformance bug when the message sequence numbers (SNs) in the second <i>ClientHello</i> and <i>ServerHello</i> differ.	Fixed
	Server	Non-conformance bug when a server proceeds with the interaction despite receiving a <i>ClientHello</i> message with a non-existing version.	Fixed
	Client	Non-conformance bug when a client fails to update the highest validated record SN; thus, accepting messages with outdated SNs.	Fixed
LSQUIC	Client&Server	Non-conformance bug when a client (or server) fails to discard a packet with the fixed bit assigned to zero.	Confirmed
	Client&Server	Three (3) non-conformance bugs occur where the incorrect error code is returned by a client (or server) in the presence of invalid input.	All Fixed
Quicly	Server	Non-conformance bug when a server fails to discard a <i>NEW_TOKEN</i> frame generated by a client and proceeds with the interaction.	Fixed
Picoquic	Server	Non-conformance bug when a server fails to discard a <i>NEW_TOKEN</i> frame generated by a client and proceeds with the interaction.	Fixed
	Client&Server	Three (3) non-conformance bugs occur where the incorrect error code is returned by a client (or server) in the presence of invalid input.	All Fixed

TinyDTLS client receives a *CertificateRequest* with more certificate types than the number specified in the `certificate_types_count` field, two out-of-bounds pointers are accessed. (2) Similarly, an out-of-bounds pointer is dereferenced in some cases when the `signature_hash_algorithms_length` field is inconsistent with the actual size of the `signature_hash_algorithms` field. (3) During the DTLS handshake, if a TinyDTLS client (or server) is fed a message for which the handshake type is advertised as *ClientHello* but its size is smaller than the minimum size for a valid *ClientHello* message, the receiving party initiates the cookie generation process. In this case, the cookie generation function tries to dereference pointers past the boundary of the message in two different locations. (4) Similarly, when a TinyDTLS client expects a *ServerHello* message, if the handshake type is advertised as *ServerKeyExchange*, an incorrect function for processing the message is invoked which, later, causes an assertion to fail. (5) If a TinyDTLS server (or client) expects a *ChangeCipherSpec*, but is sent a record for which the content type is advertised as *Alert*, the server mistakenly invokes the wrong function for processing that record as an *Alert*, which causes accessing an out-of-bounds pointer of the record. This bug was caught with the monitor of Fig. 4 whose implementation was shown in Fig. 8.

In addition, we detected a total of eight previously unknown bugs in WolfSSL’s implementation. Four of them are non-conformance bugs due to insufficient checking of length-related fields. The remaining four are as follows: (1) A non-conformance bug where the implementation fails to ensure that the message sequence in a *ClientHello* message sent to the server should be reused by the server in a *HelloVerifyRequest* message. (2) A similar non-conformance occurs for the message sequence in the second *ClientHello* and the *ServerHello* message. These two non-conformances make a WolfSSL DTLS server susceptible to message sequence duplication in the case of multiple *HelloVerifyRequests* and multiple cookie exchanges, respectively. (3) A non-conformance bug occurs if the server is fooled to proceed with the handshake interaction despite receiving a *ClientHello* message with a non-existing protocol version. (4) An insufficient check in the anti-replay mechanism in WolfSSL’s implementation creates a situation where the client fails to update the highest validated record sequence number. The failure makes the client susceptible to replay attacks where one party is fed outdated messages.

To answer Q.4, we mention that all these bugs were detected in *completely unmodified* SUTs. However, to also see whether *some* modifications in the SUTs would somehow improve the effectiveness of our technique, we also performed an experiment where we disabled the checks of the hash values in the *Finished* messages. We did not detect any more bugs in that setting.

6.3 Effectiveness on QUIC Implementations

To answer Q.3 for the QUIC protocol, we represented a total of 16 requirements from its RFC [18] as monitors, and checked them in a scenario where in addition to completing the underlying TLS handshake, two parties of the protocol (client and server) exchange various transport parameters. As SUTs we used the most recent versions or commits of three C-based open-source QUIC implementations: LiteSpeed QUIC 3.2.0, quicly 482610bf, and picoquic 7f2fbdfb. The LiteSpeed QUIC (LSQUIC) library is a commercial implementation of QUIC and HTTP/3 functionality for servers and clients. Quicly is a modular implementation of the QUIC protocol designed to be used within the H2O HTTP server. Picoquic is a minimal implementation of QUIC aligned with the IETF specifications.

In contrast to the DTLS experiments, and because QUIC is a network protocol whose interaction is almost fully encrypted, for QUIC implementations we made two small modifications to the SUTs: we disabled encryption and timeouts. These changes were quite small though; less than 100 LOC for each of these three implementations.

Our technique detected a total of nine bugs in them, with all but one of them fixed by now. Six of these bugs (second lines for LSQUIC and picoquic in Table 1) are non-conformance bugs that manifest as incorrect error codes returned by a client (or server) in the presence of invalid input. We mention that one of these six bugs was caught with the monitor of Fig. 7. The remaining three bugs, one per implementation, are: (1) A non-conformance bug in LSQUIC when a client (or server) fails to discard a packet with the fixed bit assigned to zero. The LSQUIC developers confirmed the issue as a non-conformance to the QUIC specification but noted that, for performance and simplicity, their implementation currently does not come with logic to check the fixed bit. (2&3) The QUIC protocol mandates that only servers are allowed to generate *NEW_TOKEN* frames. Servers receiving *NEW_TOKEN* frames are expected to discard them while generating an appropriate error. In both the quicly and picoquic implementations, the server failed to generate an error

upon receiving a NEW_TOKEN frame. Both implementations have by now been fixed.

7 THREATS TO VALIDITY

In this section, we briefly discuss limitations of our technique. The main limitation is related to the scope of requirements to be tested. As our monitors can only observe and manipulate the (fields of) messages exchanged during a protocol interaction, they are limited to testing requirements directly observable from these interactions. Consequently, our technique cannot effectively capture protocol requirements that involve the SUT's internal state and do not result in change in externally observable behaviour. Luckily, for both DTLS and QUIC, there are only few such requirements.

Unsurprisingly, our technique is sensitive to the definition and accuracy of the monitors used. A monitor that is flawed or that inadequately captures the intended requirements can result in erroneous test outcomes, manifesting as false positives or false negatives. During our experiments, we did not encounter any false positives; this is corroborated by the fact that all issues we discovered were confirmed (and most fixed) by the protocol implementors.

On a related point, the extraction of protocol requirements from natural language documents (the RFC and all its errata, which are in most cases separate documents) and the definition of monitors is currently not automated. This is an orthogonal issue, which is outside the scope of this paper. Work along these lines exists in the literature (e.g., by Chen et al. [14]), but how to take advantage of such work for protocols such as DTLS whose errata are separate documents is a challenge left for future work.

8 RELATED WORK

Symbolic execution was introduced already in the 70's [19]. In the last fifteen years, it has developed into a powerful software testing technique, being able to find safety violations that have been difficult to expose using other techniques [9].

Applications of symbolic execution (SE) for testing stateful software such as network protocol implementations are not so numerous. Several approaches, including [24, 26, 29, 30, 32, 34], allow a test setup to designate fields in incoming packets as symbolic, in order to explore code paths reachable for different values of such fields. Tempel et al. [34] show that the test coverage of SE can be increased if statefulness is taken into account by specifying a protocol's state machine. Wen et al. [35] use model learning to generate a protocol's state machine, which is then supplied as an additional input in SE and the approach is applied to the FTP protocol. These approaches use SE to detect generic run-time errors (e.g., in memory accesses), but are not applied to check protocol requirements.

KleeNet [30] supports checking of correctness properties after manually inserting assertions into the SUT's source code. An extension of KleeNet with a simulated network model [29], supports SE of a distributed application executing on several nodes. Occurrences of packet loss and node failures can be represented by symbolic variables, prompting SE to explore different error scenarios. Assertions can be inserted into the SUT's code to check consistency of the distributed state. The use of symbolic variables to represent non-deterministically occurring errors is analogous to our use of symbolic variables for nondeterminism in monitors.

Ideas of KleeNet are used in SymNet [28], which employs the S²E platform [15], to perform integration testing of different synchronization scenarios and apply it to test the Host Identity Protocol [23]. These lines of work aim to uncover interoperability issues under different synchronization and failure scenarios. They do not test that packet data is manipulated according to protocol requirements; in particular they have limited abilities to detect flaws and requirement violations in processing of adversarial (i.e., non-valid) inputs. Interoperability is also addressed by Rath et al. [26] who present a SE-based methodology applied to QUIC implementations, and by Pedrosa et al. [24] whose work uses symbolic execution to characterize messages that during a session can be sent by one party but rejected as non-compliant by the other, and constraint solving to find interoperability violations.

Checking correctness of packet data processing is addressed by SymbexNet [32]. However, SymbexNet's technique applies symbolic execution and requirement checking separately. It first symbolically executes the protocol parties jointly on partly symbolic input, in order to generate test inputs that explore a variety of code paths. Only code paths that can be exercised in a non-adversary environment are explored. The generated test inputs are thereafter supplied to the party under test, while monitoring the resulting execution to check against requirements which have been encoded as rules over packet sequences. SymbexNet's approach exercises a wide variety of code paths in a protocol party, but may still miss specific requirement-violating inputs. This can happen e.g., when the same code path can be exercised by both violating and non-violating inputs, and the symbolic execution engine happens to generate a non-violating input for this particular path. This problem was addressed by our prior work [3] for testing DTLS implementations against the requirements using symbolic execution. That work addresses statefulness by presenting the sequence of all input packets sent to the SUT during an interaction as input before SE starts, which requires pre-recording such as sequence of packets. Since SE will replay the pre-recorded session, randomization and other sources of nondeterminism must first be manually removed from the SUT's source code. Each protocol party is tested separately. Moreover, for testing different protocol configurations, additional sequences of packets must be pre-recorded. In contrast, our use of protocol parties (e.g., client/server) to perform the interaction allows to reuse monitors for checking the same requirement in different configurations simply by adjusting the desired configuration through the SUT's API within the test harness. In the technique of this paper, all protocol parties can be tested using the same test harness. Furthermore, our prior work [3] represented stateful protocol requirements by manually inserting assumptions and assertions into the source code of the SUT, whereas our use of monitors provides a structured way of representing requirements for processing packet data and does not require modification to the SUT's code. Thus, monitors can be reused out-of-the-box for different implementations of the same protocol.

To uncover deep bugs in stateful protocol implementations, Chau et al. [12, 13] use symbolic execution for a form of differential testing of non-stateful libraries that classify certificate chains as valid or invalid. Using symbolic execution, path constraints for the "valid"

and “invalid” outcomes are generated. Path constraints from different implementations are compared, and discrepancies are investigated. This approach can be suitable for non-stateful components, if requirements are difficult to formalize precisely. In our approach, we can check individual protocol requirements separately, without needing to access other implementations than the one under test.

Greybox fuzzing is a highly successful testing technique for detecting crashes and vulnerabilities in non-stateful programs [36]. Adaptations to testing stateful programs include LTL-Fuzzer [21], which employs greybox fuzzing to detect violations of requirements expressed in temporal logic. LTL-Fuzzer requires manually instrumenting the SUT’s code with requirement-specific checks. AFLNET [25], further developed in SnapFuzz [1], aims to increase the effectiveness of greybox testing on stateful programs, such as network protocol implementations. Like many other applications of greybox fuzzing, AFLNET mainly finds run-time errors (e.g., crashes); it does not check protocol requirements.

The well-known concept of monitors [31] is commonly used to check requirements. Monitors have been used to check a rich range of properties, including properties expressed in fragments of first-order temporal logic (e.g., [6, 17]). Run-time verification (RV) [11] checks properties of concrete individual executions, whereas our adaption of monitors for SE allows to check such properties for all possible values of symbolic inputs. Previous combinations of RV and SE (e.g., the work of Artho et al. [2]) exhibit the same separation between symbolic execution and requirements checking as in SymbexNet.

For *black-box* testing, McMillan and Zuck [20] propose a monitor-based approach, which is applied to QUIC implementations. In their approach, a monitor-like machinery which can detect requirement violations is constructed and applied during interoperability testing. Unlike the approach of this paper, their approach solely evaluates how implementations respond to specification-conforming inputs, and therefore achieves limited coverage of adversarial inputs. Their paper acknowledges this as something limiting the coverage achieved, and leaves combination of their technique with symbolic execution as future work.

9 CONCLUSIONS

In this paper, we presented a technique that employs symbolic execution for testing stateful network protocol implementations. The technique is based on the use of monitors to concentrate the logic for testing each requirement into a software component external to the SUT. Monitors provide a uniform mechanism for checking requirements and obviate the need for modifying the SUT. To evaluate the effectiveness of our technique, we applied it to four widely-used DTLS implementations and three implementations of the QUIC protocol. Our technique was able to reproduce all bugs detected by another recently proposed related technique, but with less implementation effort and considerably faster. It was also able to detect several previously unknown bugs in newer versions of these implementations. With a single exception, due to a specific design choice for performance reasons in the LSQUIC implementation, all the requirement violations and bugs that our technique detected have since then been fixed in the corresponding implementations.

We argue that our monitor-based technique is a suitable basis for building environments that perform testing of protocol implementations based on symbolic execution. Such test environments can automatically check requirements in different implementations of the protocol. Moreover, they can easily be extended to check additional requirements by constructing corresponding monitors. In this way, common test infrastructure can be maintained and used by the community of developers for a specific network protocol.

REPLICATION MATERIAL

We provide replication material [5] for all our experiments.

ACKNOWLEDGMENTS

This research was partially funded by the Swedish Foundation for Strategic Research (SSF) through project aSSIsT and the Swedish Research Council (Vetenskapsrådet).

REFERENCES

- [1] Anastasios Andronidis and Cristian Cadar. 2022. SnapFuzz: High-throughput Fuzzing of Network Applications. In *31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA ’22)*. ACM, New York, NY, USA, 340–351. <https://doi.org/10.1145/3533767.3534376>
- [2] Cyrille Artho, Howard Barringer, Allen Goldberg, Klaus Havelund, Sarfraz Khurshid, Michael R. Lowry, Corina S. Pasareanu, Grigore Rosu, Koushik Sen, Willem Visser, and Richard Washington. 2005. Combining test case generation and runtime verification. *Theor. Comput. Sci.* 336, 2-3 (2005), 209–234. <https://doi.org/10.1016/j.tcs.2004.11.007>
- [3] Hooman Asadian, Paul Fiterău-Broştean, Bengt Jonsson, and Konstantinos Sagonas. 2022. Applying Symbolic Execution to Test Implementations of a Network Protocol Against its Specification. In *IEEE Conference on Software Testing, Verification and Validation (ICST 2022)*. IEEE, 70–81. <https://doi.org/10.1109/ICST53961.2022.00019>
- [4] Hooman Asadian, Paul Fiterău-Broştean, Bengt Jonsson, and Konstantinos Sagonas. 2022. Replication Material for the ICST 2022 Paper: Applying Symbolic Execution to Test Implementations of a Network Protocol Against its Specification. <https://doi.org/10.5281/zenodo.5929867>
- [5] Hooman Asadian, Paul Fiterău-Broştean, Bengt Jonsson, and Konstantinos Sagonas. 2024. Replication material for the ARES 2024 paper: Monitor-based Testing of Network Protocol Implementations Using Symbolic Execution. <https://doi.org/10.5281/zenodo.11945614>
- [6] David A. Basin, Felix Klaedtke, Samuel Müller, and Eugen Zalinescu. 2015. Monitoring Metric First-Order Temporal Properties. *J. ACM* 62, 2 (2015), 15:1–15:45. <https://doi.org/10.1145/2699444>
- [7] Benjamin Beurdouche, Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, Alfredo Pirotoni, Pierre-Yves Strub, and Jean Karim Zinzindohoue. 2017. A Messy State of the Union: Taming the Composite State Machines of TLS. *Commun. ACM* 60, 2 (Feb. 2017), 99–107. <https://doi.org/10.1145/3023357>
- [8] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (San Diego, California) (OSDI ’08)*. USENIX Association, Berkeley, CA, USA, 209–224. <http://dl.acm.org/citation.cfm?id=1855741.1855756>
- [9] Cristian Cadar and Koushik Sen. 2013. Symbolic execution for software testing: three decades later. *Commun. ACM* 56, 2 (2013), 82–90. <https://doi.org/10.1145/2408776.2408795>
- [10] Marco M. Carvalho, Jared DeMott, Richard Ford, and David A. Wheeler. 2014. Heartbleed 101. *IEEE Secur. Priv.* 12, 4 (2014), 63–67. <https://doi.org/10.1109/MSP.2014.66>
- [11] Ian Cassar, Adrian Francalanza, Luca Aceto, and Anna Ingólfssdóttir. 2017. A Survey of Runtime Monitoring Instrumentation Techniques. In *Proceedings Second International Workshop on Pre- and Post-Deployment Verification Techniques, PrePost@IFM 2017 (Torino, Italy) (EPTCS, Vol. 254)*. Adrian Francalanza and Gordon J. Pace (Eds.), 15–28. <https://doi.org/10.4204/EPTCS.254.2>
- [12] Sze Yiu Chau, Omar Chowdhury, Md. Endadul Hoque, Huangyi Ge, Aniket Kate, Cristina Nita-Rotaru, and Ninghui Li. 2017. SymCerts: Practical Symbolic Execution for Exposing Noncompliance in X.509 Certificate Validation Implementations. In *2017 IEEE Symposium on Security and Privacy (San Jose, CA, USA) (SP 2017)*. IEEE Computer Society, 503–520. <https://doi.org/10.1109/SP.2017.40>

- [13] Sze Yiu Chau, Moosa Yahyazadeh, Omar Chowdhury, Aniket Kate, and Ninghui Li. 2019. Analyzing Semantic Correctness with Symbolic Execution: A Case Study on PKCS#1 v1.5 Signature Verification. In *26th Annual Network and Distributed System Security Symposium* (San Diego, CA, USA) (NDSS 2019). The Internet Society. <https://doi.org/10.14722/ndss.2019.23430>
- [14] Chu Chen, Cong Tian, Zhenhua Duan, and Liang Zhao. 2018. RFC-directed differential testing of certificate validation in SSL/TLS implementations. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) (ICSE 2018). Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman (Eds.). ACM, 859–870. <https://doi.org/10.1145/3180155.3180226>
- [15] Vitaly Chipounov, Volodymyr Kuznetsov, and George Candea. 2012. The S2E Platform: Design, Implementation, and Applications. *ACM Trans. Comput. Syst.* 30, 1 (2012), 2:1–2:49. <https://doi.org/10.1145/2110356.2110358>
- [16] Tim Dierks and Eric Rescorla. 2008. The Transport Layer Security TLS Protocol Version 1.2. RFC 5246. <http://www.rfc-editor.org/rfc/rfc5246.txt>
- [17] Klaus Havelund and Doron Peled. 2018. Efficient Runtime Verification of First-Order Temporal Properties. In *Model Checking Software - 25th International Symposium, SPIN 2018 (LNCS, Vol. 10869)*, Maria-del-Mar Gallardo and Pedro Merino (Eds.). Springer, 26–47. https://doi.org/10.1007/978-3-319-94111-0_2
- [18] Janardhan Iyengar and Martin Thomson. 2021. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000, 151 pages. <https://www.rfc-editor.org/rfc/rfc9000.txt>
- [19] James C. King. 1976. Symbolic Execution and Program Testing. *Commun. ACM* 19, 7 (July 1976), 385–394. <https://doi.org/10.1145/360248.360252>
- [20] Kenneth L. McMillan and Lenore D. Zuck. 2019. Formal Specification and Testing of QUIC. In *Proceedings of the ACM Special Interest Group on Data Communication* (Beijing, China) (SIGCOMM 2019). ACM, 227–240. <https://doi.org/10.1145/3341302.3342087>
- [21] Ruijie Meng, Zhen Dong, Jialin Li, Ivan Beschastnikh, and Abhik Roychoudhury. 2022. Linear-time Temporal Logic guided Greybox Fuzzing. In *Proceedings of the 44th International Conference on Software Engineering (ICSE’22)*. ACM, New York, NY, USA, 1343–1355. <https://doi.org/10.1145/3510003.3510082>
- [22] Bodo Möller, Thai Duong, and Krzysztof Kotowicz. 2014. This POODLE bites: exploiting the SSL 3.0 fallback. <https://www.openssl.org/~bodo/ssl-poodle.pdf>
- [23] R. Moskowitz, P. Nikander, P. Jokela, and T. Henderson. 2008. Host Identity Protocol. RFC 5201. <https://www.ietf.org/rfc/rfc5201.txt> updated by RFC 6253.
- [24] Luis Pedrosa, Ari Fogel, Nupur Kothari, Ramesh Govindan, Ratul Mahajan, and Todd D. Millstein. 2015. Analyzing Protocol Implementations for Interoperability. In *12th USENIX Symposium on Networked Systems Design and Implementation* (Oakland, CA, USA) (NSDI 15). USENIX Association, 485–498. <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/pedrosa>
- [25] Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. 2020. AFLNET: A Greybox Fuzzer for Network Protocols. In *IEEE 13th International Conference on Software Testing, Validation and Verification (ICST 2020)*. IEEE, 460–465. <https://doi.org/10.1109/ICST46399.2020.00062>
- [26] Felix Rath, Daniel Schemmel, and Klaus Wehrle. 2018. Interoperability-Guided Testing of QUIC Implementations using Symbolic Execution. In *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC* (Heraklion, Greece) (EPIQ@CoNEXT 2018). ACM, 15–21. <https://doi.org/10.1145/3284850.3284853>
- [27] Eric Rescorla and Nagendra Modadugu. 2012. Datagram Transport Layer Security Version 1.2. RFC 6347, 32 pages. <http://www.rfc-editor.org/rfc/rfc6347.txt>
- [28] Raimondas Sasnauskas, Philipp Kaiser, Russ Lucas Jukic, and Klaus Wehrle. 2012. Integration testing of protocol implementations using symbolic distributed execution. In *20th IEEE International Conference on Network Protocols* (Austin, TX, USA) (ICNP 2012). IEEE Computer Society, 1–6. <https://doi.org/10.1109/ICNP.2012.6459940>
- [29] Raimondas Sasnauskas, Olaf Landsiedel, Muhammad Hamad Alizai, Carsten Weise, Stefan Kowalewski, and Klaus Wehrle. 2010. KleeNet: Discovering Insidious Interaction Bugs in Wireless Sensor Networks Before Deployment. In *Proceedings of the 9th International Conference on Information Processing in Sensor Networks* (Stockholm, Sweden) (IPSN 2010). ACM, 186–196. <https://doi.org/10.1145/1791212.1791235>
- [30] Raimondas Sasnauskas, Jő Ágila Bitsch Link, Muhammad Hamad Alizai, and Klaus Wehrle. 2008. KleeNet: automatic bug hunting in sensor network applications. In *Proceedings of the 6th International Conference on Embedded Networked Sensor Systems* (Raleigh, NC, USA) (SenSys 2008). ACM, 425–426. <https://doi.org/10.1145/1460412.1460485>
- [31] Fred B. Schneider. 2000. Enforceable security policies. *ACM Trans. Inf. Syst. Secur.* 3, 1 (2000), 30–50. <https://doi.org/10.1145/353323.353382>
- [32] JaeSeung Song, Cristian Cadar, and Peter R. Pietzuch. 2014. SYMBEXNET: Testing Network Protocol Implementations with Symbolic Execution and Rule-Based Specifications. *IEEE Trans. Software Eng.* 40, 7 (2014), 695–709. <https://doi.org/10.1109/TSE.2014.2323977>
- [33] W. Richard Stevens. 1990. *Unix network programming*. Prentice Hall.
- [34] Sören Tempel, Vladimir Herdt, and Rolf Drechsler. 2023. Specification-based Symbolic Execution for Stateful Network Protocol Implementations in the IoT. *IEEE Internet of Things Journal* (2023). <https://doi.org/10.1109/JIOT.2023.3236694>
- [35] Shameng Wen, Qingkun Meng, Chao Feng, and Chaojing Tang. 2017. A model-guided symbolic execution approach for network protocol implementations and vulnerability detection. *PLoS one* 12, 11 (2017), e0188229. <https://doi.org/10.1371/journal.pone.0188229>
- [36] Michał Zalewski. 2013. American Fuzzy Lop. <http://lcamtuf.coredump.cx/afl/>.

Paper III



Uncovering Flaws in Network Protocol Implementations Using Differential Symbolic Execution

Hooman Asadian
hooman.asadian@it.uu.se
Uppsala University
Uppsala, Sweden

Bengt Jonsson
bengt@it.uu.se
Uppsala University
Uppsala, Sweden

Paul Fiterău-Broștean
paul.fiterau_brostean@it.uu.se
Uppsala University
Uppsala, Sweden

Konstantinos Sagonas
kostis@it.uu.se
Uppsala University
Uppsala, Sweden
National Technical University of Athens
Athens, Greece

ABSTRACT

Implementations of network protocols must closely follow their specifications to avoid security flaws and interoperability issues. Ensuring compliance is difficult given the complex and often stateful nature of modern protocols. Oracle-driven, specification-based testing helps but checks only requirements that have been explicitly encoded, which leaves gaps for complex, stateful protocols such as DTLS. We present a differential symbolic execution approach that removes reliance on an explicitly specified oracle by comparing implementations on the same interaction while exploring classes of inputs. Our approach symbolically executes a common harness for each system under test, observes the packet exchange, tracks interaction state, makes selected input fields symbolic, and records compact, semantically meaningful outputs for every explored path. For pairs of tested implementations, we use SMT solving to identify pairs of paths which can be exercised by the same input, but result in different outputs. We then check the implementations' divergent behaviors for compliance to the specification. We applied our technique to the most recent versions of several widely used DTLS implementations and detected nine distinct bugs across versions, including five that were previously unknown and were later fixed or confirmed.

1 INTRODUCTION

Testing network protocol implementations against their specifications is essential for ensuring secure, reliable, and interoperable communication. Even minor deviations from the protocol specification can cause interoperability failures or introduce subtle security vulnerabilities that attackers can exploit. For example, in the case of the TLS protocol, seemingly trivial errors in handling input fields have led to serious attacks such as ROBOT [4] and the POODLE downgrade attack [3, 23].

Several established techniques exist for testing protocol implementations. A prominent approach is specification-based testing [18, 19, 22], where tests are derived directly from the official protocol specification (for example, an RFC). The idea is to execute the implementation with selected inputs and check the corresponding outputs against the requirements expressed in the specification. In practice, this often involves creating an oracle that encodes the

requirements, e.g., in the form of assertions, to detect deviations from the specification.

The detection power in specification-based testing can be significantly enhanced by using Symbolic Execution [20] (SE), which explores an implementation's response to the full range of possible inputs, rather than just a few selected ones. Symbolic execution is a white-box technique that analyzes programs in which some inputs or variables are designated as *symbolic*, and explores the code paths that are feasible for some assignment to these symbolic inputs, thereby reasoning over the entire space of their possible input values. In the context of protocol testing, SE can be used to explore a wide range of behaviors exhibited by an implementation. These behaviors are then checked, which has traditionally relied on an oracle incorporating checks for selected requirements [1, 2, 29–31]. We call such an approach *specification-based testing*.

While specification-based testing is effective for cases where the requirements can be accurately captured in an oracle, it is inherently incomplete: any requirements not explicitly encoded in the oracle will not be checked. For complex protocols such as TLS and DTLS, whose specifications are long, intricate, and sometimes ambiguous, manually encoding all requirements and corner cases is impractical, leaving coverage gaps where bugs can hide. Even if enhanced by symbolic execution, ensuring compliance still depends on the oracle; if a particular requirement or edge case is missing from the oracle, violations in that area will remain undetected.

Differential testing removes the need for specification-based oracles by comparing the behaviors of multiple implementations. The idea, originating from McKeeman's work [21], is to compare multiple independent implementations of the same protocol on identical inputs and flag any differences as potential flaws. This method has proven highly effective in uncovering bugs, as demonstrated by e.g., the “Frankencerts” study on TLS certificate validation [5] and the differential fuzzing of C compilers [35]. Differential testing can be further enhanced by integrating SE, a technique known as *differential symbolic execution*, which explores a broader range of behaviors to detect subtle differences. This approach has been successfully applied to detect inconsistencies in protocol message parsing logic [36] and to identify noncompliance in X.509 certificate validation implementations [7]. However, it has so far only

been employed to test *stateless* components of network protocol implementations.

In this work, we employ differential symbolic execution to test the *stateful* behavior of network protocol implementation. Specifically, we exercise multiple implementations on the same concrete protocol interaction while selectively making chosen input fields symbolic. SE then explores paths exercised on each implementation using different values for the symbolic fields. For each path, we record the path condition and a compact set of semantically-meaningful outputs. We then check whether there exists an input that satisfies two recorded path conditions for different implementations while the corresponding outputs differ. When such an input exists, we consult the RFC to determine whether the difference reflects non-compliance or a security-relevant flaw. By generating differences automatically, our approach can expose non-conforming behaviors even with respect to corner-case requirements that are easily missed by specification-based testing.

Our approach proved highly effective during this study: we analyzed four popular DTLS implementations and automatically identified numerous discrepancies resulting in the discovery of nine bugs, of which five were previously unknown. Furthermore, we demonstrate that our approach can be applied in a cross-version setting (differentially testing an implementation against a prior version of itself) to catch regressions.

In summary, our work:

- Presents a differential symbolic execution method for testing stateful behavior of network protocol implementations without requiring an explicitly specified oracle to guide the testing.
- Evaluates our approach on four real-world DTLS server implementations (Mbed TLS, OpenSSL, TinyDTLS, and wolfSSL). Our analysis identified multiple discrepancies in how these implementations handle edge cases leading to the discovery of nine distinct bugs, five of which were previously unknown and have since been fixed or confirmed.
- Compares our approach against a state-of-the-art approach in specification-based testing using symbolic execution. Our approach is able to find all the bugs reported by the other approach, as well as new bugs.
- Provides a publicly available infrastructure to enable the replication of our experiments or the extension of our work.

We next present our methodology for differential symbolic execution of stateful protocol implementations, including the common test harness, lightweight monitors, and discrepancy detection procedure (§2). We then describe our implementation (§3), evaluate the approach on four DTLS servers with a comparison to a specification-based SE baseline and a study of cross-version regressions (§4), review related work (§5), and conclude (§6).

2 METHODOLOGY

In this section, we present our methodology for identifying differences in the responses different protocol implementations produce for the same input. Our approach involves, for each System Under Test (SUT), first creating a common test harness (§2.1), then symbolically executing it to record what outputs the SUT produces for different values of selected fields in input packets (§2.2). We use

the result of SE to detect differences in the outputs different SUTs generate for the same input. These differences are then investigated with respect to protocol specification (§2.3). An important step in our approach involves selecting the fields (1) to make symbolic in input packets, and (2) to consider when comparing the responses of different SUTs to the same input. This step is addressed separately in §2.4. To aid understanding, this section uses a running example to show our approach can be employed to test real implementations of DTLS and find an actual bug.

2.1 Creating a Test Harness

The initial step involves constructing for each SUT a test harness which orchestrates a predefined interaction between the SUT and another protocol party (e.g., client/server). This interaction might involve completing a handshake, or more generally, exchanging a set of packets between the protocol parties. The test harness connects two components: the SUT, which is the target of the analysis, and another protocol party, called the *facilitator*, which provides the necessary inputs to drive the interaction forward. For example, in a client/server protocol such as DTLS, if the SUTs are server implementations, the facilitator will be a client implementation. Since the goal is to examine how different SUTs respond to the same inputs, for a SUT protocol role (e.g., server) we use the same facilitator (e.g., client implementation of a particular library) for all the tested SUTs (e.g., server implementations).

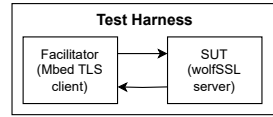


Figure 1: Architecture of the test harness used to test wolfSSL’s server implementation.

Suppose we want to find differences between two DTLS server implementations: Mbed TLS and wolfSSL. Applying our approach requires developing two separate test harnesses, one for each SUT. These test harnesses share the same facilitator. The facilitator needs to be based on a DTLS client implementation, e.g., Mbed TLS,¹ as is the case in our evaluation. Figure 1 gives a high-level view of the test harness we use to test wolfSSL.

2.2 Running Symbolic Execution

Assuming there are multiple protocol implementations available, we can already perform *basic differential testing*. Concretely, we could execute the SUT on the predefined interaction using the corresponding test harness, observe selected response packets, and flag pairwise differences. However, this evaluates only the responses to the concrete input packets in a single run, so many corner cases might remain untested. To go beyond the limits of basic differential testing, we leverage the power of symbolic execution to check the SUT’s responses to classes of input packets, thereby performing *differential SE*. This is done by making selected fields in facilitator-supplied input packets symbolic, and executing the entire harness

¹The facilitator can very well be a different library from those tested, e.g., OpenSSL.

(including the SUT and the facilitator) under an SE engine. The SE engine will then explore the code paths that are feasible for some values of these symbolic fields, and for each code path record the outputs of the SUT, including how these outputs depend on the symbolic input fields. Technically, this is achieved by recording the outputs as *symbolic expressions*. These per-path artifacts are later compared across implementations to detect discrepancies (see §2.3).

For each pair of SUTs, we run multiple differential SE campaigns. Each campaign analyzes a specific interaction and makes specific fields symbolic. In order to manage these campaigns in an orderly way, we concentrate the logic for each campaign in a lightweight *monitor*. A monitor is a software component that observes and may influence the interaction between the facilitator and the SUT (e.g., by making fields of concrete packets symbolic). It encapsulates the testing logic for a single differential SE campaign: it declares which fields to make symbolic, at which point in an interaction to make the field symbolic, and what fields to record from the SUT's response. We adapt the monitor concept from Asadian et al. [2]. In our setting the monitor does not check requirements (as in [2]); instead, it (i) makes selected fields in input packets symbolic at chosen points in the interaction between facilitator and SUT, and (ii) records the SUT's immediate response.²

To explain how monitors are defined and employed in our approach, we return to our running example. Let us assume we want to find discrepancies between the Mbed TLS and wolfSSL servers by considering their handling of the DTLS field `sequence_number`. This 48-bit field is present in every DTLS record (packet), uniquely identifying it. The value of `sequence_number` is incremented in each record sent, typically starting from 0. This allows the recipient to process records in the right order³, and to detect duplicated records (which have the same `sequence_number`). Testing that SUTs properly handle this field is difficult owing to its large domain, and the many requirements specified over it by the DTLS specification. An oracle used for specification-based testing can easily miss some of its corner-case requirements. As we will see, our approach can check even such requirements and find bugs doing so.

To analyze the SUT's handling of `sequence_number`, we use a monitor which marks `sequence_number` as symbolic for each input packet where this field appears, and records the SUT's response to that packet. We succinctly express the behavior of such monitors using a state machine formalism, as shown in Fig. 2. A monitor is represented as a state machine whose local state consists of a *control state* and possibly additional local variables. The monitor in Fig. 2 has three control states, represented by the three circles. It observes packets sent by the client or the server and may perform a transition whenever an observed packet satisfies the condition above the bar in the label of an outgoing transition. We let P be the observed packet, and let $P.source$ denote its origin. When performing the transition, the monitor executes the statement below the bar.

Starting from the initial state `Init`, the monitor in Fig. 2 can (non-deterministically) take one of two actions upon receiving a client packet. Either it takes the self-loop transition and remains in `Init`, or it transitions to `Rcvd`, making the `sequence_number` in the observed packet symbolic. Once in `Rcvd`, if the monitor observes a server

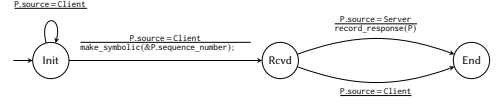


Figure 2: Monitor recording a DTLS server's responses for different values of `sequence_number`.

packet (which the monitor sees as a response to the previous client packet), it records it and transitions to `End`. In case, however, it sees another client packet, it assumes that the server has generated no response and transitions to `End`. Arrival to `End` terminates the execution, prompting SE to record the condition of the terminated path. The self-loop in `Init` allows the monitor to transition to `Rcvd` on any packet from the client during an interaction, make the packet's `sequence_number` symbolic and record the server's response (if any).

To identify the packet containing the symbolic field, we also record the packet's position in the interaction, which we refer to as the *index* (e.g., the first packet has index 0; if two packets precede it, the index is 2). We then only compare responses to packets with the same index, so the SUTs are in the same interaction state. The index is not tracked inside the monitor. Instead, our infrastructure assigns every packet a concrete counter value and introduces a *symbolic* variable that selects which packet the monitor will act on; both this selected index and the symbolic field become part of the path condition. Further details appear in §3.

Running symbolic execution on wolfSSL and Mbed TLS with the monitor in Fig. 2 produces, for each implementation, a set of explored paths. For every path, we record the path condition and the SUT's observed response at the selected interaction point. In §2.3 we describe how these per-path records are compared across implementations to reveal discrepancies.

Finally, to limit the number of discrepancies and reduce the manual effort required to analyze them, we use a single monitor per SE campaign. The monitors we use make only one instance of a field symbolic, while all other instances of the same field (e.g., those in other packets) remain fixed to the concrete values assigned during the interaction.

2.3 Finding and Analyzing Discrepancies

Given two SUTs, our objective is to identify common inputs that prompt differing outputs (responses) between the SUTs. We do this via the three-step procedure described below.

Deriving Path-Output Sets. For each SUT, SE generates a set of Path-Output pairs, one for each path explored. A pair takes the form $\langle pc, o \rangle$ where pc represents the path condition and o represents the output generated (recorded response) when exploring the path. Both pc and o are parametric on the field made symbolic and the packet index. Let S_I denote the set of Path-Output pairs generated by the SE campaign for implementation I .

Deriving Discrepancies Between Implementations. Given the sets S_I for several implementations, we compare for every pair of implementations I and I' , the sets S_I and $S_{I'}$, to determine inputs for which I and I' produce different outputs while in the same state

²We use the term *recording responses* to mean recording selected fields of responses.

³Since DTLS runs over UDP, records can be received out-of-order.

of the interaction. If some concrete input can cause I and I' to produce different outputs, there must be a path of I characterized by a Path-Output pair $\langle pc, o \rangle$ and a path of I' characterized by a Path-Output pair $\langle pc', o' \rangle$ such that: (1) pc and pc' are satisfied by this common value of the input; and (2) this value results in different values of o and o' . We can express this by the conjunction:

$$(pc \wedge pc') \wedge (o \neq o') \quad (1)$$

Conjoining $(o \neq o')$ ensures that this common input value will prompt different outputs on I and I' . For each pair of Path-Output pairs $\langle pc, o \rangle$ and $\langle pc', o' \rangle$, we check the satisfiability of Formula (1) using an SMT solver. In case of satisfiability, we ask the solver to produce a valuation for the symbolic input field and the containing packet index, that satisfies Formula (1). We include each valuation in a (discrepancy) witness along with the differing outputs o and o' produced by the two SUTs. Our approach generates a witness for every pair of Path-Output pairs satisfying Formula (1).

Analyzing Discrepancies for Bugs. For every discrepancy, the protocol's RFC is consulted to determine whether the compared SUTs are compliant. In case one of the SUTs is not, we validate the non-conformance by executing the test harness using an unmodified SUT (the SUT can be optionally subjected to minor modifications; see §3). We recall that a discrepancy includes a value for the field that is made symbolic and the index of the packet containing the field such that, assigning this value to the field will prompt different outputs for two SUTs. During execution, we modify the packet by assigning the value to the field, and check whether the issue can be reproduced. If it can, we have identified a non-conformance, and we report it to developers.

We end this section by concluding our running example. For Mbed TLS, 35 Path-Output pairs were generated, while for wolfSSL, 28 Path-Output pairs were generated by following the steps we have described. We instantiated Formula (1) for these pairs, and found two which satisfy Formula (1), forming a discrepancy witness for each. A witness contains the value for `sequence_number` and for the index of the packet containing it, and for each SUT, its response to the input packet when its `sequence_number` is assigned the included value. One of the witnesses suggests that in the initial state, when processing the first packet, upon receiving a first *ClientHello* packet with `sequence_number` that is slightly over 2^{47} , Mbed TLS responds with *HelloVerifyRequest*, indicating that it has accepted the packet, while wolfSSL does not produce any response, appearing to silently discard the packet. Consulting the RFC, we determine that Mbed TLS has handled this packet correctly. In contrast, wolfSSL has wrongly rejected a packet with a large but not invalid `sequence_number`. This is bug in wolfSSL, which can cause interoperability issues. The second witness reflects a difference in strictness regarding reuse of a record sequence number across handshake records; although the implementations behave differently, both behaviors are permitted by the RFC. Section 4 describes this and some other bugs we identified by analyzing discrepancies.

2.4 Selecting Fields

Choosing which fields to make symbolic and which to record from response packets can greatly influence the effectiveness of our approach. In this section, we give some general guidelines.

2.4.1 Selecting Fields to Make Symbolic in Input Packets. In theory, we can test implementations on all fields, particularly those like `sequence_number`, whose incorrect handling can impact security and/or functionality. In practice, for the sake of performance, we may avoid fields fed to cryptographic functions, which are difficult to handle by symbolic execution. Accessing these fields in monitors might also require additional machinery (e.g., for decryption). One option is to consider only unencrypted fields, as they can be accessed without decrypting packets. Such is the case for `sequence_number`, which is unencrypted in DTLS records, and thus can be accessed without decryption. Another option is to disable encryption, which can be done via settings supported by the SUT, or by minor SUT modifications. For DTLS, we did the former (cf. §3.1).

2.4.2 Selecting Fields to Record in Response Packets. Similarly to input fields, we can record the entire packet to maximize the number of discrepancies we can identify. Finding many discrepancies, however, this comes with the cost of increased time for analyzing them. Hence, we want to reduce the number of fields to make it more manageable for analysis. To that end, our approach excludes from analysis three classes of fields which are very likely to produce uninteresting (e.g., unlikely to expose bugs) discrepancies. The first class comprises fields assigned nonces (i.e., random values). Recording nonces in response packets will lead to trivial discrepancies due to nonces being different across the responses of the SUTs. For example, we did not consider `random` in DTLS's *ServerHello* message, as this field stores a nonce. The second class comprises fields whose different values likely reflect differences in the features/settings supported by the SUTs which cannot be easily eliminated (e.g., by enabling only common features/settings across SUTs, or by making minor SUT code changes). Recording such fields would lead to trivial discrepancies due to varying support across SUTs. For example, we did not consider the field `cipher_suites` in DTLS's *ServerHello* message, whose value depends on the algorithms (e.g., for encryption) supported by the SUT, which vary greatly among SUTs. The final class comprises fields whose values depend on the values of fields in the first and second classes. As an example, DTLS's *Finished* message contains `verify_data`, which we did not consider as it is a hash over values of fields including `cipher_suites` and `random`.

3 IMPLEMENTATION

This section details how we implemented steps in §2 for testing DTLS server implementations. We start by describing the development of the test harness (§3.1), designed to coordinate interactions between the facilitator and the SUT. We then outline the augmentations made to the SE engine (§3.2) so that Path-Output pairs are generated for each SUT tested. Pairs generated for different SUTs are then compared to find and analyze discrepancies, which is our final step (§3.3). The section ends with an overview of the adaptations made to the SUT to optimize symbolic execution (§3.4).

3.1 Creating a Test Harness

A test harness is a single executable containing two modules: the SUT and a facilitator that acts as the peer (a client when the SUT is a server). Since KLEE [6], the symbolic execution engine we use, supports only single-threaded processes, both modules are consolidated into one single-threaded Unix process. Within this

process, the facilitator and the SUT exchange packets via an in-memory queue by overriding POSIX socket APIs; implementation details are given in §3.2.1.

Protocol implementations are typically shipped as libraries that include a set of APIs, allowing users to develop custom programs tailored to their needs. These libraries often include sample programs to demonstrate their usage and functionality. We constructed the harness by adapting these samples. In the DTLS setting, we started from the sample client and server provided by each library, and refactored them into the facilitator and SUT modules. To ensure all the SUTs are tested on similar inputs, we fixed the facilitator to a single library implementation (e.g., an Mbed TLS DTLS client) and varied only the SUTs (e.g., Mbed TLS and wolfSSL DTLS servers). Although a new harness is required for each library, our experience is that a harness can usually be reused with little or no change across different versions of the same library (see §4).

For each DTLS library, the harness performs a handshake and then exchanges a small unit of *application data* between the facilitator and the SUT. The two roles execute in a loop that terminates when the handshake has completed on both sides and exactly one record containing *application data* has been exchanged in each direction. The loop also exits on any fatal alert, or when an iteration produces no new bytes. We limit execution to at most N iterations.

3.2 Augmentations to the SE Engine

We execute the test harness with KLEE, using the engine to mark selected fields as symbolic in specific packets exchanged during the interaction and to record the indices of those packets along with the SUT's responses. To this end, we augment KLEE with components adapted from the monitor-based framework of Asadian et al. [2].

3.2.1 Common Infrastructure. Many network protocols rely on POSIX sockets for packet exchange, leveraging APIs provided by the operating system's library [32]. We employ as-is the generic infrastructure of Asadian et al. as proxy to this library, in order to mediate, by redirecting API calls, communication between the SUT and facilitator. We replace the standard socket APIs with custom versions linked with the harness; when the harness runs under KLEE, it calls our versions instead of the OS ones. This proxy works for any protocol that exchanges packets through POSIX sockets.

Concretely, calls that send bytes to a socket (e.g., `sendto`) are intercepted and their payloads are enqueued in an in-memory queue. Calls that receive bytes (e.g., `recvfrom`) dequeue from this queue instead. Upon receiving a packet, the proxy runs a monitor in three steps:

- Parsing:** The first packet in the queue is parsed by a protocol-specific parser (§3.2.2) and copied into a data structure P .
- Execution:** The monitor runs with P as input and makes selected fields in P symbolic.
- Return:** After the execution of the monitor completes, P is serialized and returned to the original caller.

Each direction of the data flow (e.g., client to server and server to client) has its own packet queue. We override `bind` to initialize these queues; other socket functions remain unchanged.

3.2.2 Protocol-Specific Infrastructure. Our implementation also includes the following components for each protocol considered:

- a set of monitors, one for every input field considered for symbolic execution (as per §2.4.1);
- an output-capture function, meant to record selected response fields (as per §2.4.1);
- a serializer and parser which are optional, but facilitate constructing monitors and the output-capture function.

While the first and the third components are adapted from Asadian et al., the output-capture function is specific to our work.

Set of Monitors. We define monitors to plug in to the common infrastructure in the same way as Asadian et al., albeit our monitors are much simpler as their purpose is simply to make fields symbolic rather than to check requirements. Monitors are implemented as C functions that accept a pointer to a data structure (P) as a parameter and access packet fields by referencing the corresponding fields within this structure.

```

1 enum STATE { INIT, RCVD, END };
2
3 static STATE curr_state = INIT;
4
5 void sequence_number_diff_testing_server(RECORD *P) {
6     if (curr_state == INIT && P->source == CLIENT) {
7         kleener_make_symbolic(&P->sequence_number,
8                               sizeof(P->sequence_number), "sequence_number");
9         curr_state = RCVD;
10    }
11    else if (curr_state == RCVD && P->source == SERVER) {
12        capture_dtls_output(P);
13        curr_state = END;
14    }
15    else if (curr_state == RCVD && P->source == CLIENT)
16        curr_state = END;
17 }

```

Figure 3: The C implementation of the monitor of Fig. 2.

Figure 3 presents the C code for implementing the monitor for `sequence_number` in Fig. 2. The code begins by declaring an enumeration for the monitor's states and a static variable to store the monitor's current state (lines 1 and 3). The function implementing the monitor accepts a pointer to the data structure containing the currently observed packet. Its logic is straightforward and closely follows the preconditions and actions specified in Fig. 2. The `kleener_make_symbolic()` function is a wrapper around KLEE's `klee_make_symbolic()` and enables treating only part of a buffer as symbolic. It takes three arguments: a pointer to the relevant packet field, its size, and a descriptive name.

Our methodology needs the indices of the input packets whose fields have been made symbolic. Our code in Fig. 3 does not provide them, nor does it define the non-deterministic self-loop in the `Init` state. Both are implemented in the infrastructure within which monitors are run. This infrastructure currently supports non-determinism in the initial state between two transitions, one of which is a self-loop. It assigns each packet P an index (where the i -th packet in the interaction has index i), and a symbolic variable `trigger_index` declared to range over all possible packet indices. The infrastructure ensures that the non-self-loop transition is taken if its guard evaluates to true and the index of the current packet equals `trigger_index`. Otherwise, the self-loop transition is selected. Since `trigger_index` is symbolic, the SE engine

will explore all feasible execution paths in which the monitor performs an arbitrary number of self-loops followed by a non-self-loop transition. Each explored path corresponds to a specific value of `trigger_index`. Making `trigger_index` and the input fields symbolic makes KLEE include them in the path condition. This allows to get values for them by solving Formula (1). In each campaign, we run one monitor and mark as symbolic only one instance of the chosen field; other instances remain concrete, thereby limiting the number of distinct discrepancies produced downstream (§3.3).

Output-capture function. Output recording is implemented as a function invoked by the monitor. For DTLS, we implement an output-capture function (`capture_dtls_output(P)`) that reads the parsed response packet `P`, selects the response fields specified in §2.4.2, and makes KLEE dump their valuation in a separate file, as part of the output the output generated for the corresponding path.

When a recorded output field depends directly on symbolic inputs, the function records the corresponding symbolic expression as the value of that field. See the invocation in Fig. 3.

Parser & Serializer. These optional components simplify and make the definition of monitors more intuitive. The parser converts protocol packets into structured data, enabling monitors to access specific packet fields (e.g., `P->sequence_number` in Fig. 3). Conversely, the serializer reconstructs protocol packets from the data structures, whose fields may have been made symbolic by the monitors. Without these components, monitors would need to directly manipulate the bytes at specific offsets within the byte array representing the protocol packet, significantly increasing complexity. The parser also facilitates construction of the output-capture function in a similar way. For our application to DTLS, we reused the parser and serializer from Asadian et al. [2].

3.3 Finding and Analyzing Discrepancies

Executing the test harness with the symbolic execution engine produces a set of Path-Output pairs for each tested SUT, as described in §2.3. These pairs are then analyzed pairwise to identify discrepancies between the two respective SUTs. Concretely, for each field considered, we run our augmented KLEE on each tested SUT. This produces, for each SUT, a folder which includes for each path, a file specifying the path condition, and a file containing the response recorded on that path by our output-capture function. Both the path condition and the output are expressed in SMT-LIBv2 format. The folders generated by KLEE for every pair of SUTs are fed to a Python script, which logs discrepancy witnesses that we later analyze. For every pair of explored paths, one for each SUT compared, the script uses the aforementioned files generated for these paths to instantiate and check the satisfiability of Formula (1) using Z3. If Formula (1) is not satisfiable, our script discards the pair. Otherwise, it uses Z3 to obtain satisfying values for the symbolic variables, and build a discrepancy witness (comprising the values of symbolic input field, packet indices and differing response fields), logging it for later analysis. Since discrepancy witnesses are analyzed manually, our script can deduplicate the witnesses before printing them. Deduplication keeps only one witness for each distinct combination of `trigger_index` and the response fields. For instance, in our running example, our script identifies ten discrepancies; deduplication

reduces this number to two representative witnesses. Since each SE campaign marks as symbolic only a single field instance and we retain only deduplicated witnesses, the number of discrepancies we must inspect remains modest.

3.4 Optional SUT Adaptations

Implementations of certain network protocols, such as DTLS, may retransmit packets or prematurely terminate interactions if a response is not received within a predefined timeout. This can lead to timing-based differences, which are beyond this work’s scope as they are hard to reproduce. We therefore suppressed the time-based behavior of implementations, either by appropriate settings via the SUT’s library API or, when not possible, by making minor changes to the SUT’s code to disable timeouts.

In security protocols, common cryptographic operations (e.g., encryption, hashing) are time-consuming for SE. To speed up SE, these operations may be disabled/simplified/avoided (e.g., by appropriate SUT settings or minor modifications in the SUT’s code). In DTLS, after the client and server agree on a shared key, subsequent exchanges are encrypted. To avoid this encryption, we configured the parties to not use encryption when communicating.

4 EVALUATION

In this section, we evaluate our technique across several widely used DTLS implementations. Through this evaluation, we aim to answer the following questions:

- Q.1 How effective is our technique at finding bugs compared to the state-of-the-art in SE-based protocol testing when considering DTLS server implementations?
- Q.2 How does the required testing effort of our technique compare to the state-of-the-art, and how can it be optimized?
- Q.3 Can our technique identify previously unknown bugs in the latest versions of DTLS implementations?
- Q.4 Can our technique detect regressions?

Section 4.1 describes the evaluation setup. Section 4.2 answers Q.1 and Q.2, while Sections 4.3 and 4.4 answer Q.3 and Q.4, respectively.

4.1 Evaluation Setup

We compare our approach to that of Asadian et al. [2], which is recent, it also employs symbolic execution for protocol testing, and is similar also in the sense that it targets DTLS version 1.2. We target DTLS servers, for which we have developed test harnesses. We tested the server implementations of Mbed TLS, OpenSSL, wolfSSL and TinyDTLS, which were also analyzed by Asadian et al. [2]. We consider the original versions/commits analyzed by Asadian et al. [2] to answer Q.1 and Q.2, and the latest commits at the time of writing to answer Q.3 and Q.4. Concretely, we tested servers of Mbed TLS versions 3.1.0 and 3.5.0, OpenSSL versions 3.0.2 and 3.2.0, wolfSSL versions 5.3.0 and 5.6.3, and TinyDTLS commits e4d547ce (old) and fd47d5 (latest). We were able to use the exact same test harness for Mbed TLS, OpenSSL and wolfSSL across the two versions tested for each, and had to perform only minor changes to the test harness between the two commits considered for TinyDTLS. This suggests that once a test harness has been developed for a library, it can be reused across multiple versions of the library with only minor or no adaptations.

Table 1: Discovered bugs in various versions of DTLS server implementations. Each row lists the implementation version, a brief description of the bug and its resolution status. Clicking on the status leads to the corresponding GitHub issue used for reporting the bug. Descriptions highlighted with a cyan background and blue text indicate new bugs, while those marked in black text indicate rediscoveries of bugs previously reported by Asadian et al. [2]. Footnotes denote bugs that affect multiple versions. New SUTs tested in this work are shown in blue, while old SUTs tested also by Asadian et al. are in black.

SUT	Short Description of the Bug(s)	Status
TinyDTLS e4d547ce	Out-of-bounds pointers caused by a handshake message other than <i>ClientHello</i> whose message type field has been set to <i>ClientHello</i>	Fixed
	Out-of-bounds pointers caused by a <i>ChangeCipherSpec</i> message whose content type field has been set to <i>Alert</i>	Fixed
	Record sequence number wrapping in a response triggered by receiving a record with the maximum possible record sequence number [†]	Confirmed
TinyDTLS f1d47d5	Susceptibility to Denial-of-Service due to generating an alert instead of silently discarding a record of invalid content type	Fixed
wolfSSL 5.3.0	Non-conformance bug when the message sequence numbers in the initial <i>ClientHello</i> and <i>HelloVerifyRequest</i> differ	Fixed
	Non-conformance bug when the message sequence numbers in the second <i>ClientHello</i> and <i>ServerHello</i> differ	Fixed
	Non-conformance bug when a server incorrectly discards <i>ClientHello</i> messages with large record sequence numbers [‡]	Confirmed
wolfSSL 5.6.3	Non-conformance bug where the server fails to discard a <i>ClientKeyExchange</i> message containing a duplicate record sequence number	Fixed
	Non-conformance bug where a server proceeds with the interaction despite receiving <i>ClientHello</i> messages with a non-existing version	Fixed

[†] This bug also affects TinyDTLS f1d47d5.

[‡] This bug also affects wolfSSL 5.6.3.

4.2 Comparison with a Related Technique on DTLS Servers

To address Q.1 and Q.2, we compare our technique with the approach proposed by Asadian et al. [2], which uses symbolic execution to check the conformance of protocol implementations against specification-derived requirements. For a fair comparison, we apply both approaches to the same DTLS server implementations and versions evaluated by Asadian et al. [2].

Concretely, for each requirement tested by Asadian et al., we constructed a corresponding server-side lightweight monitor that makes symbolic only the field relevant to that requirement. We then performed SE using these monitors on the same DTLS servers, and collected the resulting Path-Output pairs. Instead of checking these executions against specification assertions, as done by Asadian et al., we performed pairwise comparisons of the Path-Output pairs produced by different implementations and analyzed the resulting discrepancies.

Using this setup, our approach successfully identified *all* bugs reported by Asadian et al. [2]. Furthermore, our approach uncovered two additional bugs that were missed by their specification-based technique. These results indicate that our approach is more effective at exposing deviations in implementation behavior.

All identified bugs are summarized in Table 1. We next explain the two additional bugs and discuss why they were not detected by the approach of Asadian et al.. We conclude this section by answering Q.2.

Record Sequence Number Wrap. TinyDTLS mishandles the second *ClientHello* received during a DTLS handshake if the (record) sequence number in this message is the maximum possible value. In this case, TinyDTLS responds with a *ServerHello* with the same sequence number, followed by *ServerHelloDone* with sequence number zero, indicating a wrap-around⁴. By doing this, TinyDTLS violates the DTLS 1.2 RFC [27, p. 9], according to which, DTLS implementations MUST either abandon an association or initiate a re-handshake before the sequence number wraps. TinyDTLS does neither.

This bug was missed by the technique of Asadian et al. [2] because it only tested if DTLS server implementations correctly check uniqueness of sequence numbers, and not how they handle (rare) wrapping situations.

Handling Large Record Sequence Numbers. wolfSSL incorrectly responds to the first or second *ClientHello* of a DTLS handshake by discarding these messages if their encapsulating records have sequence numbers which exceed a certain large threshold. According to the DTLS 1.2 RFC [27, p. 9], records should only be discarded if their sequence numbers are large enough to potentially cause sequence number wrapping, which is not the case here. Therefore, wolfSSL’s behavior does not align with the RFC and can lead to interoperability problems with other DTLS implementations which accept large sequence numbers.

This bug was missed by the technique of Asadian et al. [2] because it stems from a corner case for which no test (monitor) was implemented to appropriately check the handling of large sequence numbers.

Regarding Q.2, this approach has the advantage that it does not require tests to be constructed for the entire specification, which is impractical for complex protocols such as DTLS, whose behavior is described across multiple RFC documents. Instead, discrepancies are identified automatically using lightweight monitors that merely designate selected protocol fields as symbolic.

In our approach, however, identified discrepancies must be inspected against the RFC to determine whether they correspond to actual bugs. By contrast, the specification-based approach of Asadian et al. [2] detects bugs automatically once a test harness and a (complete) set of specification-derived tests have been constructed. It is important to note that not all discrepancies indicate bugs, as the RFC may explicitly permit multiple compliant behaviors.

A limitation of our approach is that it may miss bugs that are exhibited in the same way by all tested implementations, although this can be mitigated by testing a larger and more diverse set of implementations. Both approaches also require the manual construction of a test harness of comparable complexity for each SUT.

⁴The server is supposed to increment the sequence number in each record it sends.

In practice, the two approaches are complementary. The monitor-based infrastructure is shared across both techniques, making their combination natural. The specification-based approach of Asadian et al. [2] can automatically detect bugs using tests derived directly from the specification, while our approach can uncover new bugs and extend the existing test suite so that these bugs can subsequently be detected automatically in other implementations.

4.3 Effectiveness on Recent DTLS Implementations

To answer Q3, we applied our technique to the latest versions of Mbed TLS 3.5.0, OpenSSL 3.2.0, TinyDTLS commit f1d47d5, and wolfSSL 5.6.3. Our technique identified bugs in two of these systems (TinyDTLS and wolfSSL), indicating that the approach is effective at uncovering previously unknown issues in modern DTLS implementations. Concretely, it discovered nine bugs, five of which were previously unknown, highlighted with blue text/cyan background in Table 1. All previously unknown bugs were reported to the corresponding developers and have since been fixed or confirmed. We proceed to analyze these bugs in detail.

Handling Invalid Content Type. TinyDTLS responds with a warning level alert to a *ClientKeyExchange* message encased in a record with the content type set to *alert*, instead of its expected value *handshake*. This goes against the RFC [27, p. 14], which specifies that in this case an implementation should simply discard the record, or must alternatively issue a fatal level alert. Subsequently, the RFC recommends against issuing fatal alerts, as it can make an implementation susceptible to DoS attacks. Consequently, TinyDTLS’s behavior is non-compliant and possibly susceptible to DoS attacks.

Handling Duplicate Record Sequence Numbers. wolfSSL accepts a *ClientKeyExchange* message even if its (record) sequence number duplicates that in the initial *ClientHello*. According to the DTLS 1.2 RFC [27, p. 13], implementations should check that records received have unique sequence numbers and should reject those that do not. wolfSSL fails to enforce this RFC requirement.

Handling Invalid Record Version. wolfSSL fails to validate the record version for records containing *ClientHello* messages. According to the DTLS 1.2 RFC [27, p. 8], the version field indicates the version of the protocol being used. The TLS 1.2 RFC [11, p. 88], on which DTLS 1.2 is based, emphasizes the importance of validating the first byte of the version field. These observations indicate that wolfSSL does not fully comply with the specification.

4.4 Effectiveness in Detecting Regressions

To evaluate the effectiveness of our technique in detecting regressions, we conducted a controlled experiment using TinyDTLS. Specifically, we focused on a previously identified vulnerability documented in Issue #74 of the TinyDTLS repository. This vulnerability entails bypassing the server’s cookie check on the second *ClientHello* message in a handshake, by using an invalid version in the record encapsulating this message. TinyDTLS will then proceed to the next phase of the handshake, becoming susceptible to DoS

attacks.⁵ This vulnerability has since been fixed and does not affect the latest version.

Our investigation indicates that the vulnerability was present since the first commit. We thus artificially reintroduced the vulnerability onto the latest version of TinyDTLS, which we used as baseline since it properly managed *ClientHello* messages with invalid version numbers. We then applied our technique to compare the behavior of the original (unmodified) version against that of the modified (buggy) version. In this experiment, the version field of the initial *ClientHello* message was made symbolic. Symbolic execution explored seven execution paths for each version of the implementation. In both cases, SE completed in under ten seconds. The comparison of the resulting Path-Output pairs revealed a single discrepancy, which was detected within one second. Inspection of this discrepancy directly exposed the reintroduced vulnerability in the modified version.

These results demonstrate that the proposed technique can efficiently detect regressions with minimal exploration effort and low analysis overhead. This experiment therefore provides an affirmative answer to Q4.

5 RELATED WORK

Differential testing has been widely applied to uncover semantic bugs in network protocol implementations by comparing multiple implementations under the same inputs. Among these approaches, differential symbolic execution, introduced by Person et al. [24], extended SE to systematically identify and analyze behavioral differences across versions. However, this technique has been applied mostly to stateless systems.

ParDiff [36] introduces a static differential analysis framework that extracts finite-state machines (FSMs) from the message parsing code of different protocol implementations and applies bisimulation with SMT solving to identify semantic inconsistencies between the FSMs. Chau et al. [7, 8] employ differential symbolic execution to classify certificate chains as valid or invalid, generating path constraints for each outcome. These constraints are then compared across implementations to identify and analyze discrepancies. In contrast to these approaches, which focus on isolated components of an implementation, our work performs dynamic differential SE of the entire protocol implementation, enabling the detection of specification violations during stateful protocol interactions.

Other approaches combine symbolic execution with differential testing by synthesizing concrete test cases and running them to detect discrepancies. RFCcert [9] extracts rule constraints from the relevant RFCs, uses SE to construct certificates that satisfy or violate each rule, and then differentially tests multiple X.509 validators to pinpoint specification violations. SymTCP [34] applies selective SE to an end host’s TCP stack to synthesize packets that reach key accept or drop paths, then sends them through black-box DPI middleboxes and the end host; any disagreement reveals insertion or evasion opportunities that desynchronize DPI state. Both methods use symbolic execution to generate concrete test cases,

⁵The cookie field in the second *ClientHello* should duplicate the cookie in the server’s *HelloVerifyRequest* response to the first *ClientHello*. This mechanism prevents DoS attacks as it ensures an attacker is unable to exhaust the resources of server by flooding it with many *ClientHello* messages which prompt the server to start as many computationally-expensive handshakes.

which are run on the SUTs to find discrepancies. By contrast, we use symbolic execution to identify discrepancies by systematically cross-examining path conditions of the SUTs.

Specification-based testing checks an implementation against an oracle derived from the specification. Several approaches [1, 2, 29–31] propose incorporating SE to improve the detection power of specification-based testing. KleeNet [30] introduces a technique that integrates KLEE into the TinyOS platform for testing sensor network applications. Using KleeNet, correctness properties can be tested by manually adding assertions to the SUT’s codebase. A follow-up work [29] extended KleeNet with a simulated network model, enabling SE of distributed applications across multiple nodes. Packet loss and node failures can be represented as symbolic variables, allowing SE to explore different error scenarios.

SymbexNet [31] focuses on checking specification-derived requirements using symbolic execution. It performs SE on both single- and multi-packet exchanges to generate high-coverage concrete test packets. These packets are then replayed on the unmodified implementation, and the resulting input-output streams are checked against requirements derived from the protocol specification. SE can overlook inputs that violate requirements when those inputs follow the same code path in the SUT as compliant ones. Asadian et al. [1] address this issue by embedding assertions and assumptions directly into the SUT’s code, guiding the symbolic execution engine toward code paths exercised by packet sequences that may violate protocol requirements. Subsequent work [2] moves the requirements into monitors that reside outside the SUT. These monitors provide a uniform mechanism for validating protocol requirements and reduce the need for extensive SUT changes.

Although these specification-based techniques are effective when the requirements are accurately captured in an oracle, they are inherently incomplete. Any requirement not explicitly encoded in the oracle will remain unchecked, as observed in §4.

In addition to white-box approaches, black-box techniques have been used to evaluate protocol implementations against their specifications. Protocol state fuzzing, for example, applies model learning [25, 33] to infer state machine models of implementations, often in the form of Mealy machines. This technique has been extensively applied to network and IoT protocols, including TLS [15], TCP [13, 14], SSH [16], OpenVPN [10], IPsec [17], QUIC [12, 26], and EDHOC [28].

6 CONCLUSIONS

In this paper, we presented a differential symbolic execution approach for testing the stateful behavior of network protocol implementations without relying on an explicitly specified oracle. Our approach performs SE of the SUTs, treating as symbolic selected fields in input packets exchanged during an interaction, and recording the SUT’s responses to these packets. It automatically processes the results of SE to identify inputs that satisfy path conditions of both SUTs while prompting different responses. These discrepancies are analyzed for conformance.

We evaluated our approach on DTLS server implementations of four widely-used libraries, and demonstrated its effectiveness. In short, our approach rediscovered all four bugs previously identified by the state-of-the-art specification-based testing approaches,

while also identifying two new bugs that those techniques missed. Our approach also identified four new bugs in the latest versions of the libraries tested. In total, we identified nine distinct bugs across versions, five of which were previously unknown and later confirmed or fixed by maintainers. We also showed that our approach can be effective to detect regressions in a cross-version setting. While clearly outperforming specification-based testing in our evaluation, it is worth stressing that our approach is a complement to specification-based testing, rather than a replacement. Unlike specification-based testing, our approach’s effectiveness does not hinge on the availability of an oracle encoding all the specification’s requirements. Yet our approach is also not fully automatic, requiring manual inspection of discrepancies. Future work can explore combining our approach with specification-based testing, such as Asadian et al., to increase automation.

REFERENCES

- [1] Hooman Asadian, Paul Fiterău-Broștean, Bengt Jonsson, and Konstantinos Sagonas. 2022. Applying Symbolic Execution to Test Implementations of a Network Protocol Against its Specification. In *IEEE Conference on Software Testing, Verification and Validation (ICST 2022)*. IEEE, 70–81. <https://doi.org/10.1109/ICST53961.2022.00019>
- [2] Hooman Asadian, Paul Fiterău-Broștean, Bengt Jonsson, and Konstantinos Sagonas. 2024. Monitor-based Testing of Network Protocol Implementations Using Symbolic Execution. In *Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES ’24)*. ACM, 1–12. <https://doi.org/10.1145/3664476.3664521>
- [3] Benjamin Beurdouche, Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, Alfredo Pironti, Pierre-Yves Strub, and Jean Karim Zinzindohoue. 2015. A Messy State of the Union: taming the Composite State Machines of TLS. In *IEEE Symposium on Security and Privacy*. IEEE.
- [4] Hanno Böck, Juraj Somorovsky, and Craig Young. 2018. Return Of Bleichenbacher’s Oracle Threat (ROBOT). In *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, Baltimore, MD, 817–849. <https://www.usenix.org/conference/usenixsecurity18/presentation/bock>
- [5] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. 2014. Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations. In *2014 IEEE Symposium on Security and Privacy*. 114–129. <https://doi.org/10.1109/SP.2014.15>
- [6] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (San Diego, California) (OSDI ’08)*. USENIX Association, Berkeley, CA, USA, 209–224. <http://dl.acm.org/citation.cfm?id=1855741.1855756>
- [7] Sze Yiu Chau, Omar Chowdhury, Md. Endadul Hoque, Huangyi Ge, Aniket Kate, Cristina Nita-Rotaru, and Ninghui Li. 2017. SymCerts: Practical Symbolic Execution for Exposing Noncompliance in X.509 Certificate Validation Implementations. In *2017 IEEE Symposium on Security and Privacy (San Jose, CA, USA) (SP 2017)*. IEEE Computer Society, 503–520. <https://doi.org/10.1109/SP.2017.40>
- [8] Sze Yiu Chau, Moosa Yahyazadeh, Omar Chowdhury, Aniket Kate, and Ninghui Li. 2019. Analyzing Semantic Correctness with Symbolic Execution: A Case Study on PKCS#1 v1.5 Signature Verification. In *26th Annual Network and Distributed System Security Symposium (San Diego, CA, USA) (NDSS 2019)*. The Internet Society. <https://doi.org/10.14722/ndss.2019.23430>
- [9] Chu Chen, Cong Tian, Zhenhua Duan, and Liang Zhao. 2018. RFC-directed differential testing of certificate validation in SSL/TLS implementations. In *Proceedings of the 40th International Conference on Software Engineering (Gothenburg, Sweden) (ICSE 2018)*. ACM, 859–870. <https://doi.org/10.1145/3180155.3180226>
- [10] Lesly-Ann Daniel, Erik Poll, and Joeri de Ruiter. 2018. Inferring OpenVPN State Machines Using Protocol State Fuzzing. In *IEEE European Symposium on Security and Privacy (EuroS&P) Workshops (London, U.K.)*. IEEE, 11–19. <https://doi.org/10.1109/EuroSPW.2018.00009>
- [11] Tim Dierks and Eric Rescorla. 2008. The Transport Layer Security TLS Protocol Version 1.2. RFC 5246. <http://www.rfc-editor.org/rfc/rfc5246.txt>
- [12] Tiago Ferreira, Harrison Brewton, Loris D’Antoni, and Alexandra Silva. 2021. Prognosis: Closed-box Analysis of Network Protocol Implementations. In *ACM SIGCOMM 2021 Conference*. ACM, 762–774. <https://doi.org/10.1145/3452296.3472938>
- [13] Paul Fiterău-Broștean and Falk Howar. 2017. Learning-Based Testing the Sliding Window Behavior of TCP Implementations. In *Critical Systems: Formal Methods*

- and Automated Verification - Joint 22nd International Workshop on Formal Methods for Industrial Critical Systems and 17th International Workshop on Automated Verification of Critical Systems, FMICS-AVoCS 2017. LNCS, Vol. 10471. Springer, 185–200. https://doi.org/10.1007/978-3-319-67113-0_12
- [14] Paul Fiterău-Broștean, Ramon Janssen, and Frits W. Vaandrager. 2016. Combining Model Learning and Model Checking to Analyze TCP Implementations. In *Computer Aided Verification - 28th International Conference, CAV 2016, Proceedings, Part II (LNCS, Vol. 9780)*. Springer, 454–471. https://doi.org/10.1007/978-3-319-41540-6_25
- [15] Paul Fiterău-Broștean, Bengt Jonsson, Konstantinos Sagonas, and Fredrik Täquist. 2022. DTLS-Fuzzer: A DTLS Protocol State Fuzzer. In *15th IEEE International Conference on Software Testing, Verification and Validation (ICST 2022)*. IEEE Computer Society.
- [16] Paul Fiterău-Broștean, Toon Lenaerts, Joeri de Ruiter, Erik Poll, Frits W. Vaandrager, and Patrick Verleg. 2017. Model Learning and Model Checking of SSH Implementations. In *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software (Santa Barbara, CA, USA) (SPIN 2017)*. ACM, 142–151. <https://doi.org/10.1145/3092282.3092289>
- [17] Jiaxing Guo, Chunxiang Gu, Xi Chen, and Fushan Wei. 2019. Model Learning and Model Checking of IPsec Implementations for Internet of Things. *IEEE Access* 7 (Nov. 2019), 171322–171332. <https://doi.org/10.1109/ACCESS.2019.2956062>
- [18] Samuel Jero, Maria Leonor Pacheco, Dan Goldwasser, and Cristina Nita-Rotaru. 2019. Leveraging textual specifications for grammar-based fuzzing of network protocols. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 9478–9483. <https://doi.org/10.1609/aaai.v33i01.33019478>
- [19] Siva Kesava Reddy Kakarla, Ryan Beckett, Todd Millstein, and George Varghese. 2022. SCALE: Automatically Finding RFC Compliance Bugs in DNS Nameservers. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 307–323. <https://www.usenix.org/conference/nsdi22/presentation/kakarla>
- [20] James C. King. 1976. Symbolic Execution and Program Testing. *Commun. ACM* 19, 7 (July 1976), 385–394. <https://doi.org/10.1145/360248.360252>
- [21] William M McKeeman. 1998. Differential testing for software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [22] Kenneth L. McMillan and Lenore D. Zuck. 2019. Formal Specification and Testing of QUIC. In *Proceedings of the ACM Special Interest Group on Data Communication (Beijing, China) (SIGCOMM 2019)*. ACM, 227–240. <https://doi.org/10.1145/3341302.3342087>
- [23] Bodo Möller, Thai Duong, and Krzysztof Kotowicz. 2014. This POODLE bites: exploiting the SSL 3.0 fallback. <https://www.openssl.org/~bodo/ssl-poodle.pdf>
- [24] Suzette Person, Matthew B. Dwyer, Sebastian Elbaum, and Corina S. Pasareanu. 2008. Differential symbolic execution. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering (Atlanta, Georgia) (SIGSOFT ’08/FSE-16)*. Association for Computing Machinery, New York, NY, USA, 226–237. <https://doi.org/10.1145/1453101.1453131>
- [25] Harald Raffelt, Maik Merten, Bernhard Steffen, and Tiziana Margaria. 2009. Dynamic testing via automata learning. *Int. J. Softw. Tools Technol. Transf.* 11, 4 (2009), 307–324. <https://doi.org/10.1007/s10009-009-0120-7>
- [26] Abdullah Rasool, Greg Alpár, and Joeri de Ruiter. 2019. State machine inference of QUIC. (2019). arXiv preprint arXiv:1903.04384.
- [27] Eric Rescorla and Nagendra Modadugu. 2012. Datagram Transport Layer Security Version 1.2. RFC 6347, 32 pages. <http://www.rfc-editor.org/rfc/rfc6347.txt>
- [28] Konstantinos Sagonas and Thanasis Typaldos. 2023. EDHOC-Fuzzer: An EDHOC protocol state fuzzer. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (Seattle, WA, USA) (ISSTA ’23)*, René Just and Gordon Fraser (Eds.). ACM, New York, NY, USA, 1495–1498. <https://doi.org/10.1145/3597926.3604922>
- [29] Raimondas Sasnauskas, Olaf Landsiedel, Muhammad Hamad Alizai, Carsten Weise, Stefan Kowalewski, and Klaus Wehrle. 2010. KleeNet: Discovering Insidious Interaction Bugs in Wireless Sensor Networks Before Deployment. In *Proceedings of the 9th International Conference on Information Processing in Sensor Networks (Stockholm, Sweden) (IPSN 2010)*. ACM, 186–196. <https://doi.org/10.1145/1791212.1791235>
- [30] Raimondas Sasnauskas, Jó Ágila Bitsch Link, Muhammad Hamad Alizai, and Klaus Wehrle. 2008. KleeNet: automatic bug hunting in sensor network applications. In *Proceedings of the 6th International Conference on Embedded Networked Sensor Systems (Raleigh, NC, USA) (SenSys 2008)*. ACM, 425–426. <https://doi.org/10.1145/1460412.1460485>
- [31] JaeSeung Song, Cristian Cadar, and Peter R. Pietzuch. 2014. SYMBEXNET: Testing Network Protocol Implementations with Symbolic Execution and Rule-Based Specifications. *IEEE Trans. Software Eng.* 40, 7 (2014), 695–709. <https://doi.org/10.1109/TSE.2014.2323977>
- [32] W. Richard Stevens. 1990. *Unix network programming*. Prentice Hall.
- [33] Frits W. Vaandrager. 2017. Model learning. *Commun. ACM* 60, 2 (2017), 86–95. <https://doi.org/10.1145/2967606>
- [34] Zhongjie Wang, Shilong Zhu, Yue Cao, Zhiyun Qian, Chengyu Song, Srikanth V Krishnamurthy, Kevin S Chan, and Tracy D Braun. 2020. SymTCP: Eluding Stateful Deep Packet Inspection with Automated Discrepancy Discovery. In *NDSS (NDSS 2020)*. <https://doi.org/10.14722/ndss.2020.24083>
- [35] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (San Jose, California, USA) (PLDI ’11)*. Association for Computing Machinery, New York, NY, USA, 283–294. <https://doi.org/10.1145/1993498.1993532>
- [36] Mingwei Zheng, Qingkai Shi, Xuwei Liu, Xiangzhe Xu, Le Yu, Congyu Liu, Guannan Wei, and Xiangyu Zhang. 2024. ParDiff: Practical Static Differential Analysis of Network Protocol Parsers. *Proceedings of the ACM on Programming Languages* 8 (04 2024), 1208–1234. <https://doi.org/10.1145/3649854>

Paper IV



Testing IoT Protocol Requirements Using Fuzzing and Symbolic Execution: Application to CoAP

Hooman Asadian*, Paul Fiterău-Broștean*, Bengt Jonsson*, and Konstantinos Sagonas*[†]

*Department of Information Technology, Uppsala University, Uppsala, Sweden

[†]School of Electrical and Computer Engineering, National Technical University of Athens, Athens, Greece

Abstract—The rapid expansion of the Internet of Things (IoT) has introduced the need for thorough testing of its protocol implementations to ensure conformance to their specifications and increase their security. This paper investigates the application of two of the major testing techniques, *fuzz testing* and *symbolic execution*, to test the implementations of the Constrained Application Protocol (CoAP), a key protocol in the IoT ecosystem. We explore the efficacy of these techniques in discovering requirement violations in CoAP implementations. Focusing on two widely-used CoAP implementations, *libcoap* and *FreeCoAP*, we systematically apply both fuzzing and symbolic execution to test conformance with key requirements derived from CoAP's specifications. Our findings demonstrate the strengths and limitations of both approaches, and highlight nine non-conformances in these implementations, most of which have been fixed. Finally, we provide insights into how fuzzing and symbolic execution can be effectively utilized for protocol testing.

I. INTRODUCTION

Network protocols enable reliable communication between different software and hardware components in most infrastructures of our society. Ensuring that the implementations of these protocols conform to their specifications is critical. Non-conformance can lead to security breaches, such as the Heartbleed [8] and the TLS POODLE downgrade [16] vulnerabilities, both of which were caused by improper handling of protocol-specific requirements.

With the rapid expansion of the Internet of Things (IoT), similar attention must be given to quality assurance of its underlying infrastructure; this includes testing of protocol implementations developed specifically for IoT systems. Several communication protocols, including MQTT, CoAP, and EDHOC, have been designed specifically for IoT environments, such as low-power devices operating over lossy networks. Implementations of such protocols merit particular testing effort, since sometimes they are deployed on devices that are not easy to access or update upon the discovery of errors. In this paper, we focus on testing implementations of the CoAP protocol [23], which plays a crucial role in the IoT ecosystem. CoAP follows a client/server interaction model similar to that of HTTP, but it is specifically optimized for machine-to-machine interactions.

Techniques for testing software and communication protocols have advanced dramatically in the last two decades. One major

class of software testing focuses on detecting runtime errors, such as crashes and memory access errors, using techniques such as *fuzz testing* (fuzzing) and *symbolic execution* (SE). These testing techniques have proven very effective at identifying critical bugs, e.g., in systems libraries and device drivers, but have not been so much used to guarantee that protocol implementations conform to their specifications. That is the purpose of *conformance testing*, which examines whether a protocol implementation faithfully follows defined standards; this is essential to prevent subtle but potentially dangerous deviations that could compromise reliability and security. Traditional conformance testing techniques are less effective in uncovering bugs than fuzzing and symbolic execution; for this purpose some researchers are developing adaptations of fuzz testing [15] and symbolic execution [22], [24], [2], [3] that directly check whether a protocol implementation satisfies (parts of) its specification, e.g., as formulated in its RFC.

This paper investigates the application of fuzzing and symbolic execution to the challenge of testing CoAP implementations against its main RFCs. Both techniques aim to check how a system under test (SUT) responds to a wide range of different inputs. Fuzzing achieves this by generating a large number of (random) test inputs, constructed by mutating previously applied inputs. Using cleverly designed mutation strategies, it can often—but not always—quickly penetrate into the many corners of a SUT's code. Symbolic execution achieves a similar goal by systematically exploring a range of code paths followed by the program. It designates part of the test input as *symbolic*, and explores the code paths that are possible for different values of these symbolic inputs. Technically, this relies on instrumenting the SUT's code to enable symbolic manipulation of executed program statements. Against the above background, this paper:

- investigates the effectiveness of (adaptations of) fuzzing and symbolic execution in discovering bugs and requirement violations in IoT protocol implementations;
- compares fuzzing and SE in terms of their effectiveness at exposing bugs, as well as in time that this requires;
- (as a by-product) improves the quality of two widely used CoAP implementations by exposing bugs in them which have been fixed and/or reported to their developers; and
- offers some advice on how to apply these two techniques to IoT protocol implementations.

In our investigation, we have tested two well-known CoAP im-

plementations written in C: libcoap [12] and FreeCoAP [9]. To assess their conformance, we employ twelve core requirements from the RFC documents that define the CoAP standard. In order to produce an unbiased comparison of fuzzing and SE, we have created a common test harness for both these techniques, which has a common mechanism for checking a requirement, and which can be re-targeted to either technique by minimal modifications. The results indicate that both fuzzing and SE are very effective in locating protocol requirement violations fast; they have managed to uncover a total of nine bugs in these two implementations, the majority of which have been fixed by them and the rest have been reported to their developers.

The rest of this paper is organized as follows: After some background on fuzzing and SE, §III overviews CoAP, §IV details our methodology for applying fuzzing and SE to CoAP implementations, and §V presents our evaluation and findings. Finally, §VI concludes with a brief discussion.

II. FUZZING AND SYMBOLIC EXECUTION

In this section, we overview fuzzing and symbolic execution and recent works applying them to protocol implementations.

Fuzz Testing (Fuzzing) has established itself as a powerful technique for uncovering software bugs by mutating and executing numerous inputs to trigger and observe unintended behaviors, particularly in the context of memory errors and crashes. In the last decade, this area has been led by coverage-guided greybox fuzzing tools such as American Fuzzy Lop (AFL) [27] and its successor AFL++ [10], which offer advanced mutation strategies and enhanced performance, and have been used across a wide range of applications. Fuzzing tools also have been applied to test IoT protocol implementations of IPv6 network stacks (e.g., [20]). Other studies [13], [14], [28] are specifically aimed at fuzz testing of CoAP implementations. Several recent works propose techniques that tailor greybox fuzzing to stateful systems such as communication protocols [19], [17], [4]. A common characteristic of all these works is their primary emphasis on uncovering runtime errors (e.g., crashes) and robustness issues, rather than detecting non-conformance to the protocol's specification.

Symbolic Execution (SE), first introduced in the 1970s [11], has evolved into a powerful technique for exhaustive software testing [7], enabling systematic exploration of program paths by representing (selected) inputs symbolically rather than as concrete values. As a result, SE can uncover bugs, security vulnerabilities, and potential failures that might be missed by traditional testing methods. Applications of symbolic execution for testing network protocol implementations have been limited. Several works [22], [24], [18], [21], [2], [25], [3] target network protocol implementations by enabling test setups to designate specific fields in incoming packets as symbolic, allowing for the exploration of code paths that are reachable with different values for these fields. To enhance test coverage, some works [26], [25] incorporate the protocol's state machine into symbolic execution to account for the protocol's statefulness. However, the application of these approaches is limited to detecting runtime errors, and they are not utilized

for checking protocol requirements. KleeNet [22] enables the checking of correctness properties by manually adding assertions to the SUT's source code. Symbolic variables are employed to simulate occurrences of packet loss and node failures, prompting SE to explore various error scenarios. The inserted assertions validate the consistency of the distributed state. SymbexNet [24] tests protocol implementations by first employing SE to generate test inputs that explore a wide range of code paths. These inputs are then replayed on the SUT, observing potential violations of rules derived from the protocol specification. Symbolic execution might miss requirement-violating input, when this input exercises the same code path on the SUT as non-violating input. Recently, we have proposed a technique that overcomes this issue by manually embedding assertions and assumptions into the SUT's source code [2]. The net effect is to guide the SE engine to investigate specifically code paths exercised by packet sequences that can violate protocol requirements. Follow-up work [3] concentrates the requirement testing logic into monitors, which are implemented in a component that is external to the SUT. Monitors provide a uniform mechanism for checking protocol requirements and alleviate the need for extensive SUT modifications.

III. CONSTRAINED APPLICATION PROTOCOL

The Constrained Application Protocol (CoAP), as standardized in RFC 7252 [23], is designed specifically for constrained nodes and networks, playing a crucial role in the IoT ecosystem. Although the protocol draws inspiration from HTTP by adopting the REST architectural style, CoAP uniquely operates over UDP, distinguishing itself through asynchronous request and response management via CoAP messages. This design results in a simpler protocol, free from historical complexities. CoAP follows a client/server interaction model similar to that of HTTP, but it is specifically optimized for machine-to-machine communications that dominate its use. In these contexts, CoAP devices frequently act as both client and server. A CoAP client begins an interaction by sending a request to a server, using methods such as GET or POST, to act upon a resource identified by a URI on the server. The server responds with either a representation of the resource or a status code indicating the outcome of the action. CoAP accommodates both reliable and unreliable modes of communication. For reliable communication, a CoAP sender transmits a request as a Confirmable (CON) message, and the recipient is required to acknowledge receipt by responding with an Acknowledgment (ACK) message before a preset timeout period elapses. In cases where the recipient cannot process the CON message, it sends back a Reset (RST) message as a form of reply. On the other hand, when reliability is not a priority, CoAP allows for the sending of Non-confirmable (NON) messages. These messages are sent without the expectation of an acknowledgment.

Figure 1 depicts a typical interaction between two CoAP entities employing a combination of Confirmable and Non-confirmable messages. The interaction begins with Entity A sending a CON message to Entity B, requesting the resource located at the "/resource". Entity B responds with an ACK message

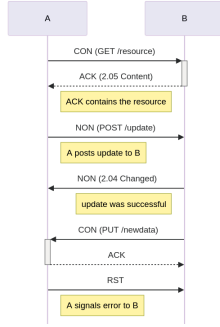


Fig. 1. Sample CoAP Interaction

containing the requested resource, signifying a successful GET operation. Subsequently, A issues a NON message to perform a POST request, targeting the "/update" URI to modify a resource on B. In reply, entity B sends a NON message with status code "2.04 Changed", confirming that the resource has been updated. Entity B then sends a CON message with a PUT request to either create or update the resource at the "/newdata". Entity A acknowledges receipt of the PUT request with an ACK message. This ACK merely indicates the message's receipt and not the success of the PUT operation. The sequence concludes with Entity A sending a RST message to Entity B. This message indicates an error encountered during the processing of the PUT request or the inability to fulfill the request, resulting in the termination of the interaction for this specific operation.

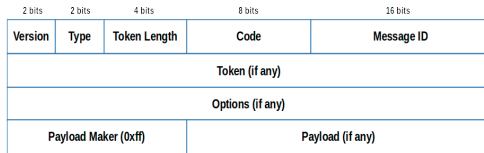


Fig. 2. CoAP Message Format

CoAP employs a concise message format (shown in Fig. 2) for both requests and responses. The format comprises a 4-byte binary header, which may be followed by a series of compact binary options and an optional payload. We will now go over the key fields in a CoAP message.

- Version** The header of a CoAP message begins with a version field, represented as a 2-bit unsigned integer.
- Type** The type of a CoAP message is identified by a 2-bit unsigned integer in the message header.
- Token Length** This 4-bit field specifies the length of the Token field, which can be between 0 to 8 bytes. The token is used to correlate requests and responses.
- Code** The distinction between a request and a response in CoAP is determined by the code value. This is an 8-bit unsigned integer divided into a 3-bit class (the most significant bits) and a 5-bit detail (the least significant bits), formatted as

'c.dd'. Here, 'c' represents a digit ranging from 0 to 7 for the 3-bit subfield, and 'dd' comprises two digits from 00 to 31 for the 5-bit subfield.

Message ID This 16-bit unsigned integer field is used to detect duplicates and, optionally, enable reliable communication.

Options A variable-length sequence of options that modify the request or response. Options can specify URI paths, content formats, query strings, and more. Each option is a pair which comprises a number and a value.

Payload Maker In CoAP, the presence of a payload in a message is indicated by a specific payload marker [23, p. 17], which is a distinct byte (0xFF). This marker serves to separate the payload, which is the actual data being transmitted, from the rest of the message components, such as the header and any options.

With RFC 7959 [5], CoAP incorporates a feature known as *Block-Wise Transfers* to facilitate the transfer of large data. This mechanism breaks down a large body into a series of smaller blocks, which is particularly advantageous in constrained network environments. Such environments often contain devices with limited memory and bandwidth capacities, making the transmission of a large body in a single message impractical. A CoAP entity indicates the need for block-wise transfer through the inclusion of either a Block1 or Block2 option within the CoAP message. The Block1 option is employed in requests, particularly when a client needs to send a large body to the server. Conversely, the Block2 option is utilized in responses, allowing the server to send a large body back to the client. These options are used in managing both the size and sequence of the blocks within the exchanged messages. The value of a Block option encodes the following fields:

Block Size (SZX) Specifies the size of each individual block. **More Blocks Indicator (M)** Indicates whether additional blocks will follow the current block.

Block Number (NUM) Represents the relative number of the current block within a series of blocks of the given size.

IV. METHODOLOGY

In this section, we describe our methodology. In a nutshell, it involves creating a common test harness (§IV-A), encoding protocol requirements as assertions within the SUT (§IV-B), and systematically applying fuzzing (§IV-C) or SE (§IV-D) to detect deviations from CoAP protocol requirements.

A. Test Harness Creation

The first step is to create a test harness that interacts with a protocol entity. This harness is responsible for facilitating the testing process by sending and receiving protocol messages. Before testing can start, it is necessary to capture a sequence of packets that are sent to the protocol entity during an interaction (similar to Fig. 1). Packet capture can be achieved by configuring sample programs typically provided with the SUT to perform interactions, and then using tools like TCPdump or Wireshark to capture the packets that are sent to the protocol entity (e.g., server) during one such interaction. During testing, the test harness advances the protocol interaction step by

TABLE I
COAP REQUIREMENTS EXTRACTED FROM RFC 7252 AND RFC 7959

Requirement	Short Description	Reference
Version Validity	The version field MUST be set to 1 (01 binary). Other values are reserved for future versions. Messages with unknown version numbers MUST be silently ignored.	RFC 7252 [23, p. 16]
Matching Message Type	Confirmable messages MUST either carry a request or response, or be empty to elicit a Reset. Non-confirmable messages MUST always carry a request or response. Acknowledgement messages MUST carry a response or be empty. Reset messages MUST be empty. If these conditions are not met, the recipient MUST reject the message.	RFC 7252 [23, p. 21, 23]
Reserved Code	The class can indicate a request (0), a success response (2), a client error response (4), or a server error response (5). All other class values are reserved and MUST be rejected.	RFC 7252 [23, p. 16]
Token Length Validity	Lengths 9-15 for tokens are reserved. MUST NOT be sent, and MUST be processed as a message format error.	RFC 7252 [23, p. 16]
Token Echo	The tokens in a response and its respective request MUST match.	RFC 7252 [23, p. 35]
Message ID Echo	The message ID transmitted in a CON or NON message, MUST be echoed in the ACK or RST message by the recipient.	RFC 7252 [23, p. 24]
Repeatable Options	An option that is not repeatable MUST NOT be included more than once in a message.	RFC 7252 [23, p. 39]
Unrecognized Options	Unrecognized options in a message MUST be rejected.	RFC 7252 [23, p. 37]
Block Size Validity	If the block is not the final block, the block size implied by SZX MUST match the size of the payload in bytes.	RFC 7959 [5, p. 10]
Content Format	The Content-Format Option sent with the requests or responses MUST reflect the Content-Format of the entire body.	RFC 7959 [5, p. 12]
Further Request Block Size	The SZX block size specified in a Block1 Option in control usage in a response indicates the maximum block size preferred by the server for subsequent transfers, and the client should adhere to this size or a smaller one in all further requests within the transfer sequence.	RFC 7959 [5, p. 14]
Missing Blocks	If not all previous blocks are available at the server at the time of processing the final block, the transfer fails and error code 4.08 MUST be returned.	RFC 7959 [5, p. 14]

step using pre-captured packets. At each step, it invokes the appropriate API from the SUT to process the packet and generate the next packet from the corresponding response. The test harness should be able to run test scenarios that are relevant for all checked requirements. This can be achieved either by running a single interaction that is relevant for all requirements, or by executing several distinct interactions. In our work, the constructed test harness serves as a unified mechanism for requirements checking, and can be easily adapted either for fuzzing or for symbolic execution with minimal adjustments.

B. Encoding Protocol Requirements as Assertions

Network protocols involve particular requirements for the sequences of packets exchanged among parties. For CoAP, these requirements are located in its RFC documents including RFC 7252 [23] and RFC 7959 [5]. Once requirements, such as those listed in Table I, have been extracted from the RFC documents, they can be translated into assertions over sequences of packets exchanged between protocol entities. The assertions are then incorporated into the SUT code, in the region responsible for sending a response to a received packet. In the two implementations we tested, assertions were inserted into function `coap_session_send_pdu` for libcoap, and function `coap_server_trans_send` for FreeCoAP. In both cases, these two functions already took as argument a pointer to the memory block storing the response. Our assertions used this pointer to access fields in the response packet. Fields in the received packet were accessed via a global pointer variable which was maintained externally in the test harness.

Let us now present the encoding of two example CoAP requirements by quoting the relevant RFC text and providing the corresponding assertion that checks the requirement. The CoAP RFC [23, p. 24] specifies that:

The Message ID is a 16-bit unsigned integer that is generated by the sender of a Confirmable or Non-confirmable message and included in the CoAP header. The Message ID MUST be echoed in the Acknowledgement or Reset message by the recipient.

In Table I, we refer to this requirement as Message ID Echo. Let p_{in} represent the parsed transmitted packet and p_{out} represent the response packet. We place the following assertion at the

point where the response packet is being generated:

```
assert(((pin.type = CON ∨ pin.type = NON) ∧
  (pout.type = ACK ∨ pout.type = RST)) ⇒
  pin.message_id = pout.message_id)
```

During the execution of the test harness, this assertion is evaluated at each step to ensure that for every transmitted packet of type Confirmable (CON) or Non-confirmable (NON), the `message_id` is correctly echoed by the responder.

Some protocol requirements specify relationships between packets received in a sequence. To capture such relationships, we can formulate constraints over the fields of a sequence of packets received by a protocol party. For instance, RFC 7959 [5, p. 14] mandates:

In response to a request with a payload (e.g., a PUT or POST transfer), the block size given in the Block1 Option indicates the block size preference of the server for this resource.
...
the client SHOULD heed the preference indicated and, for all further blocks, use the block size preferred by the server or a smaller one.

In Table I, this requirement is referred to as Further Request Block Size. We check this requirement using the excerpt below:

```
if (first_response ∧ pin.code ∈ REQ_CODES)
  preferred_size := BlockSize(pout)
  first_response := false
else
  assert((BlockSize(pin) > preferred_size) ⇒
    pout.code ∈ ERR_CODES)
```

In this code, `first_response` is a boolean variable that is true if p_{out} is the first packet generated by the recipient, and `preferred_size` is a variable that stores the server's preferred block size. `BlockSize` is a function that takes a packet and returns the block size given in the Block1 option. Finally, `REQ_CODES` represents the set of codes used for requests, while `ERR_CODES` denotes the set of error codes. The above code stores the preferred size communicated in the first block transmitted by the server. It then checks that subsequent requests that exceed this size are rejected by the server, prompting it to generate an error response. Note that the RFC does not explicitly define a suggested response when a transmitted block fails to meet the preferred size requirement. In cases where the RFC is unclear on the expected behavior, we can use common sense. Here, we require that the response somehow flags an error.

Once the SUT’s code has been extended with such assertions, we can employ fuzzing or SE to test whether these assertions are triggered (i.e., whether the requirements can be violated).

C. Testing Requirements Using Fuzzing

To test using fuzzing, we employed AFL++ (4.10c), a state-of-the-art fuzzer [10] incorporating contributions from the latest research. As initial seeds, we utilized valid pre-captured packets obtained from interactions executed using the sample programs provided with each implementation. The fuzzer begins by executing the test harness, which performs a test scenario. It then provides the incoming packet, intended to be processed by a protocol party, by mutating these seeds. For some requirements concerning block-wise transfer, multiple packets need to be processed before an assertion can be checked, as a state needs to be built. Since standard fuzzers cannot handle these stateful scenarios, we load all the required packets for the interaction into a single buffer at different offsets. During a block-wise transfer, for each packet, the recipient reads from the appropriate offsets within the buffer. Consequently, the fuzzer can mutate this single buffer during a fuzzing campaign. All the assertions are checked simultaneously. To account for the inherent randomness of fuzzing, fuzzing campaigns are repeated multiple times. When the fuzzer detects an assertion violation, it stores the mutated test case that caused the violation, along with metadata such as the timestamp and number of mutations before the test case is generated. Analyzing the generated test case can help identify the reason for the assertion violation.

D. Testing Requirements Using Symbolic Execution

To test using symbolic execution, we employed KLEE (v3.0), a mature symbolic execution engine [6] built on top of the LLVM compiler infrastructure. During each symbolic execution run, only one requirement is checked, necessitating separate runs for each requirement. In each run, KLEE executes the test harness, which performs a test scenario using pre-captured valid packets. For each requirement, before a packet is processed by a protocol party, the relevant fields specified in the requirement are treated as symbolic using a KLEE-specific API, while other fields in the packet remain unchanged. To facilitate the process of treating a field as symbolic, we used a CoAP parser to convert loaded packets into data structures. We also used a serializer to convert the data structure back into a buffer, which is then processed by the protocol party. For instance, to check the Message ID Echo requirement, we treated the `message_id` and the type in the packet as symbolic. During each SE run, if a path exists where the checked assertion is violated, KLEE, with the help of an SMT solver, generates a concrete test case that violates the assertion. In this context, the test case refers to the specific values of the fields that were made symbolic.

V. EVALUATION

In this section, we present results from applying our methodology to two well-known CoAP implementations: libcoap and FreeCoAP. We begin by comparing the effectiveness of the two techniques (§V-A), followed by a description of the bugs and non-conformances they discovered (§V-B).

A. Effectiveness of Fuzzing and Symbolic Execution

In terms of effectiveness in detecting requirement violations, we did not notice any significant difference between the two techniques. On this set of twelve CoAP requirements, both techniques identified non-conformances for seven of them in libcoap—all of them have been fixed—and for two of them in FreeCoAP which have been reported (cf. Table II). Also, all violations were found quickly: We ran five fuzzing campaigns, and AFL++ consistently found inputs that violate the assertions in less than two seconds and 900 mutations on average, with the exception of one case in FreeCoAP where it needed about 40 seconds and 36 824 mutations on average to find the violation. Symbolic execution using KLEE was also able to find inputs that violate the assertions in few seconds for most requirements, with the exception of the Repeatable Options requirement for which it needed about one minute for libcoap and three and a half for FreeCoAP to find inputs that trigger the violation.

For the protocol requirements that an implementation respects, the two techniques differ in a fundamental way. Fuzzing, being a random testing technique, cannot give any guarantee other than “No violation was found after trying N input mutations in time T .” Hence, Table II does not show any numbers for fuzzing for requirements that are not violated; any such number depends on the time limit T used for the fuzzing campaign. In contrast, symbolic execution systematically explores the program paths and can in principle provide a guarantee that a requirement is not violated. The Time and Paths columns of Table II report the time which KLEE took to explore the corresponding number of paths. For libcoap, we can see that KLEE explored the search space of most requirements in few seconds or minutes with the exception of the Block Size Validity requirement for which it needed to explore about 18 000 paths in two and a half hours. Exploring the search space of FreeCoAP’s code proved more challenging: for four of the requirements KLEE took more than an hour to complete its exploration and for two other requirements the exploration did not complete within the 24 hour time limit that we used.

Taking the above into account, our experience is that fuzzing has a slight edge when one is interested in finding specification violations fast, but investing on SE pays off in the long run. In most cases, symbolic execution is able to trigger violations equally fast as fuzzing, and it can also provide guarantees of the absence of bugs in cases where the SMT solver used by the SE engine does not time out. Our overall recommendation to implementors is to adopt the assertion-based methodology to testing protocol requirements we have described in this paper.

B. Bugs Discovered

We describe the non-conformances grouped by the corresponding requirement.

Version Validity: A violation of the Version Validity requirement occurs when a server, upon receiving a message m with an invalid version, responds with a Reset message (RST) rather than ignoring m . This deviation from the RFC can cause interoperability issues, increase unnecessary network traffic, and potentially be exploited to fingerprint the server.

TABLE II
RESULTS FROM TESTING THE CoAP REQUIREMENTS ON LIBCOAP AND FREECoAP USING FUZZING AND SYMBOLIC EXECUTION

Requirement	libcoap (version 4.3.1)						FreeCoAP (commit ffc87fd)					
	Bug Status ¹	Fuzzing		Symbolic Execution			Bug Status ¹	Fuzzing		Symbolic Execution		
		TTE ²	MTE ³	TTE ²	Time ⁴	Paths		TTE ²	MTE ³	TTE ²	Time ⁴	Paths
Version Validity	Fixed (#1376)	<1s	193	1s	1s	2	—	—	—	—	43s	6
Matching Message Type	Fixed (#1295)	<1s	371	4s	16s	38	—	—	—	—	2h51m4s	7844
Reserved Code	Fixed (#1300)	<2s	546	4s	17s	41	—	—	—	—	2h55m32s	7844
Token Length Validity	—	—	—	—	12m42s	31	—	—	—	—	3h00m14s	7083
Token Echo	—	—	—	—	1s	2	—	—	—	—	20s	4
Message ID Echo	—	—	—	—	1s	1	—	—	—	—	11s	32
Repeatable Options	Fixed (#1389)	<1s	205	56s	2m29s	213	Reported (#41)	<2s	789	3m30s	⊙	28949
Unrecognized Options	—	—	—	—	2m36s	196	—	—	—	—	⊙	28917
Block Size Validity	Fixed (#1284)	<1s	286	1s	2h25m28s	17971	—	—	—	—	37s	203
Content Format	—	—	—	—	1s	4	—	—	—	—	1s	1
Further Request Block Size	Fixed (#1290)	<2s	891	2s	19m14s	27	Reported (#42)	≈40s	36824	3s	45m13s	8497
Missing Blocks	Fixed (#1287)	<1s	255	6s	3m51s	359	—	—	—	—	1h24m17s	22979

¹ **Bug Status:** non-empty entries report on the bug's status at the time of this writing; in parentheses is the issue number in the GitHub repositories of libcoap and FreeCoAP.

² **TTE (Time to Exposure):** the time it takes to expose the requirement violation (average across five AFL++ fuzzing campaigns or KLEE runs).

³ **MTE (Mutations to Exposure):** the number of mutations it takes for AFL++ to expose a requirement violation (average across five fuzzing campaigns).

⁴ **Time:** the time required for symbolic execution to complete its exploration, with a timeout set at 24 hours (denoted ⊙).

Matching Message Type: This requirement is violated when there is a discrepancy between the message Type and Code fields. Our testing revealed that a libcoap server failed to reject an erroneous message of type ACK that carries a request. Instead, it responded with the requested resource in a non-confirmable message. This non-conformance can also be used for fingerprinting or cause other problems.

Reserved Code: Another non-conformance arises when entities do not reject messages containing reserved codes. Accepting messages with these codes can cause interoperability issues among different CoAP implementations.

Repeatable Options: This non-conformance arises when a server cannot properly handle an (erroneous) message m containing more than one instance of an unrepeatable option. In such cases, the server proceeded with a response rather than rejecting m . Failing to satisfy this CoAP requirement can lead to interoperability issues and potential erroneous data interpretation. Both libcoap and FreeCoAP implementations exhibited this non-conformance.

The remaining three non-conformances are related to block-wise transfers.

Block Size Validity: A significant bug was detected in libcoap when there is a mismatch between the block size suggested by the SZX value and the actual size of a message's payload. In such scenarios, the CoAP entity processes the message based exclusively on the SZX value, disregarding the actual payload size. This situation leads to two different problems:

- If the SZX value indicates a block size *larger* than the actual payload, arbitrary data can be stored on the server. Such a situation might cause buffer overwrites, overreads, and introduce various security vulnerabilities due to the processing of unintended data.
- Conversely, if the SZX value denotes a block size *smaller* than the actual payload, only a fraction of the payload gets stored on the server. This leads to the partial loss of transmitted data, as part of the payload is discarded.

After reporting the bug that our testing revealed, libcoap's development team addressed it through Pull Request (PR) #1286. Subsequent testing of the SUT with the changes in this PR revealed that the fix was only partial. Specifically, the PR resolved the problem only in scenarios where a server receives a request from a client, not in cases where a client receives a response from a server. The issue was fully resolved by another PR (#1294) which was later merged into libcoap's code base.

Further Request Block Size: In the CoAP block-wise transfer process, once the block size has been negotiated, it is expected that all entities will adhere to this agreed-upon size for the remainder of the data exchange. A non-conformance arises when an entity encounters a message m with a block size that deviates from the one previously established. Instead of ignoring m or reporting an error, the entity adjusted its own block size to align with the block size of m . Both libcoap and FreeCoAP implementations exhibited this non-conformance.

Missing Blocks: A non-conformance in libcoap was detected when a server fails to recognize the absence of one or more blocks prior to receiving the final block in a block-wise transfer. Specifically, the issue occurs under the following circumstances: When a CoAP server receives a block with the 'More' (M) bit set to 0, it interprets this as the final block of the transfer. At this point, the server is required to have all preceding blocks available in order to successfully reassemble and process the complete data. However, in certain scenarios, the server does not adhere to this requirement. For instance, if data is partitioned into three blocks and the server receives blocks numbered 0, 2, and 3. In such cases, the server is expected to detect this discrepancy and return an error. Nonetheless, the libcoap server erroneously proceeded without recognizing that a block is missing, failing to return the anticipated error code.

VI. CONCLUSIONS

In this paper, we investigated the use of two major testing techniques, fuzzing and symbolic execution, to test IoT protocol implementations for conformance to the requirements in their

specifications, and applied them to two widely used implementations of the CoAP protocol. Our results demonstrated the effectiveness of these two techniques when combined with the assertion-based methodology we advocate. Most notably, we have shown that both fuzzing and SE are able to uncover requirement violations that have remained unnoticed by their developers, and that this can be done quickly.

We hold that our work offers practical insights for developers and researchers in the area of IoT protocol requirement testing. By highlighting the strengths and limitations of fuzzing and symbolic execution, we provide guidance on how to improve the security and reliability testing of IoT networks.

ACKNOWLEDGMENTS

We thank Sabor Amini for his initial work on using SE to test CoAP implementations [1]. Although our testing infrastructure differs significantly from his, some of the libcoap bugs of Table II were originally discovered and reported by him.

This research was partially funded by the Swedish Foundation for Strategic Research (SSF) through project aSSIsT and by grants from the Swedish Research Council (Vetenskapsrådet).

REFERENCES

- [1] S. Amini, "Using requirement-driven symbolic execution to test implementations of the CoAP and EDHOC network protocols," Master's thesis, Uppsala University, Department of Information Technology, Sep. 2023.
- [2] H. Asadian, P. Fiterău-Broștean, B. Jonsson, and K. Sagonas, "Applying symbolic execution to test implementations of a network protocol against its specification," in *IEEE Conference on Software Testing, Verification and Validation*, ser. ICST 2022. IEEE, Apr. 2022, pp. 70–81. [Online]. Available: <https://ieeexplore.ieee.org/document/9787883>
- [3] —, "Monitor-based testing of network protocol implementations using symbolic execution," in *Proceedings of the 19th International Conference on Availability, Reliability and Security*, ser. ARES '24. ACM, Jul. 2024. [Online]. Available: <https://doi.org/10.1145/3664476.3664521>
- [4] J. Ba, M. Böhme, Z. Mirzamomen, and A. Roychoudhury, "Stateful greybox fuzzing," in *31st USENIX Security Symposium*, ser. USENIX Security 2022. USENIX Association, Aug. 2022, pp. 3255–3272. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/ba>
- [5] C. Bormann and Z. Shelby, "Block-Wise Transfers in the Constrained Application Protocol (CoAP)," RFC 7959, Aug. 2016. [Online]. Available: <https://www.rfc-editor.org/info/rfc7959>
- [6] C. Cadar, D. Dunbar, and D. Engler, "KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI '08. Berkeley, CA, USA: USENIX Association, 2008, pp. 209–224. [Online]. Available: <http://dl.acm.org/citation.cfm?id=1855741.1855756>
- [7] C. Cadar and K. Sen, "Symbolic execution for software testing: three decades later," *Commun. ACM*, vol. 56, no. 2, pp. 82–90, 2013. [Online]. Available: <https://doi.org/10.1145/2408776.2408795>
- [8] M. M. Carvalho, J. DeMott, R. Ford, and D. A. Wheeler, "Heartbleed 101," *IEEE Secur. Priv.*, vol. 12, no. 4, pp. 63–67, 2014. [Online]. Available: <https://doi.org/10.1109/MSP.2014.66>
- [9] K. Cullen, "FreeCoap." [Online]. Available: <https://github.com/keith-cullen/FreeCoAP>
- [10] A. Fioraldi, D. C. Maier, H. Eibfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *14th USENIX Workshop on Offensive Technologies*, ser. WOOT 2020. USENIX Association, Aug. 2020. [Online]. Available: <https://www.usenix.org/conference/woot20/presentation/fioraldi>
- [11] J. C. King, "Symbolic execution and program testing," *Commun. ACM*, vol. 19, no. 7, pp. 385–394, Jul. 1976. [Online]. Available: <https://doi.org/10.1145/360248.360252>
- [12] "libcoap." [Online]. Available: <https://www.libcoap.net>
- [13] F. Liljedahl, "Exploring the possibilities of robustness testing of CoAP implementations using evolutionary fuzzing," Master thesis, KTH, Stockholm, Sweden, 2019.
- [14] B. Melo and P. Geus, "Robustness testing of CoAP server-side implementations through black-box fuzzing techniques," in *Proceedings of the 17th Brazilian Symposium on Information and Computational Systems Security*. SBC, 2017, pp. 533–540. [Online]. Available: <https://sol.sbc.org.br/index.php/sbsseg/article/view/19528>
- [15] R. Meng, Z. Dong, J. Li, I. Beschastnikh, and A. Roychoudhury, "Linear-time temporal logic guided greybox fuzzing," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: ACM, May 2022, pp. 1343–1355. [Online]. Available: <https://doi.org/10.1145/3510003.3510082>
- [16] B. Möller, T. Duong, and K. Kotowicz, "This POODLE bites: exploiting the SSL 3.0 fallback," 2014. [Online]. Available: <https://www.openssl.org/~bodo/ssl-poodle.pdf>
- [17] R. Natella, "StateAFL: Greybox fuzzing for stateful network servers," *Empir. Software Eng.*, vol. 27, no. 7, p. 191, 2022. [Online]. Available: <https://doi.org/10.1007/s10664-022-10233-3>
- [18] L. Pedrosa, A. Fogel, N. Kothari, R. Govindan, R. Mahajan, and T. D. Millstein, "Analyzing protocol implementations for interoperability," in *12th USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI 15. USENIX Association, May 2015, pp. 485–498. [Online]. Available: <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/pedrosa>
- [19] V.-T. Pham, M. Böhme, and A. Roychoudhury, "AFLNET: A greybox fuzzer for network protocols," in *IEEE 13th International Conference on Software Testing, Validation and Verification*, ser. ICST 2020. IEEE, Oct. 2020, pp. 460–465. [Online]. Available: <https://ieeexplore.ieee.org/document/9159093>
- [20] C. Poncelet, K. Sagonas, and N. Tsiiftes, "So many fuzzers, so little time*: Experience from evaluating fuzzers on the Contiki-NG network (hay)stack," in *37th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '22. New York, NY, USA: ACM, 2023. [Online]. Available: <https://doi.org/10.1145/3551349.3556946>
- [21] F. Rath, D. Schemmel, and K. Wehrle, "Interoperability-guided testing of QUIC implementations using symbolic execution," in *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC*, ser. EPIQ/CoNEXT 2018. ACM, Dec. 2018, pp. 15–21. [Online]. Available: <https://doi.org/10.1145/3284850.3284853>
- [22] R. Sasnauskas, O. Landsiedel, M. H. Alizai, C. Weise, S. Kowalewski, and K. Wehrle, "KleeNet: Discovering insidious interaction bugs in wireless sensor networks before deployment," in *Proceedings of the 9th International Conference on Information Processing in Sensor Networks*, ser. IPSN 2010. ACM, Apr. 2010, pp. 186–196. [Online]. Available: <https://doi.org/10.1145/1791212.1791235>
- [23] Z. Shelby, K. Hartke, and C. Bormann, "The Constrained Application Protocol (CoAP)," RFC 7252, Jun. 2014. [Online]. Available: <https://www.rfc-editor.org/info/rfc7252>
- [24] J. Song, C. Cadar, and P. R. Pietzuch, "SYMBEXNET: Testing network protocol implementations with symbolic execution and rule-based specifications," *IEEE Trans. Software Eng.*, vol. 40, no. 7, pp. 695–709, 2014. [Online]. Available: <https://doi.org/10.1109/TSE.2014.2323977>
- [25] S. Tempel, V. Herdt, and R. Drechsler, "Specification-based symbolic execution for stateful network protocol implementations in the IoT," *IEEE Internet of Things Journal*, 2023. [Online]. Available: <https://doi.org/10.1109/IJOT.2023.3236694>
- [26] S. Wen, Q. Meng, C. Feng, and C. Tang, "A model-guided symbolic execution approach for network protocol implementations and vulnerability detection," *PLoS one*, vol. 12, no. 11, p. e0188229, 2017. [Online]. Available: <https://doi.org/10.1371/journal.pone.0188229>
- [27] M. Zalewski, "American fuzzy lop," <http://lcamtuf.coredump.cx/afl/>, 2013.
- [28] Y. Zeng, M. Lin, S. Guo, Y. Shen, T. Cui, T. Wu, Q. Zheng, and Q. Wang, "MultiFuzz: A coverage-based multiparty-protocol fuzzer for IoT publish/subscribe protocols," *Sensors*, vol. 20, no. 18, 2020. [Online]. Available: <https://www.mdpi.com/1424-8220/20/18/5194>

